JSPM's

RAJARSHI SHAHU COLLEGE OF ENGINEERING

TATHAWADE, PUNE-33

(An Autonomous Institute Affiliated to Savitribai Phule Pune University,Pune)

LAB MANUAL

OF

Machine Learning Lab Manual

(IT-3206L)

T.Y.B.Tech

INFORMATION TECHNOLOGY

Authors:

1. **Prof. Dipmala Salunkhe**

© JSPM Group of Institutes, Pune. All Rights Reserved. All the information in this Course Manual is confidential. Participants shall refrain from copying, distributing, misusing/ or disclosing the content to any third parties any circumstances whatsoever.

INDEX

Sr. No.		Topic	Page. No.
1		**How to Use This Manual…….**	**4**
2		**Laboratory Details……**	**5**
	2.1	Laboratory Objective	**9**
	2.2	Laboratory Equipment	**10**
		PART A	
	2.4	Assignment No.A01	**12**
		Practical Session Plan	**12**
		Title, Objectives, Aim/ProblemStatement,Apparatus,Theory,Output etc…	**13-20**
	2.5	Assignment No. A02	**21**
		Practical Session Plan	**21**
		Title, Objectives, Aim/ProblemStatement,Apparatus,Theory,Output etc…	**22-31**
	2.6	Assignment No. A03	**32**
		Practical Session Plan	**32**
		Title, Objectives, Aim/ProblemStatement,Apparatus,Theory,Output etc…	**33-48**
	2.7	Assignment No. A04	**49**
		Practical Session Plan	**49**
		Title,Objectives,Aim/ProblemStatement,Apparatus,Theory,Output etc…	**50-57**
	2.8	Assignment No.A05	**58**
		Practical Session Plan	**58**
		Title, Objectives,Aim/ProblemStatement,Apparatus,Theory,Output etc…	**59-89**
	2.9	Assignment No. A06	**90**
		Practical Session Plan	**90**
		Title, Objectives,Aim/ProblemStatement,Apparatus,Theory,Output etc…	**91-114**
	2.10	Assignment No. A07	**115**
		Practical Session Plan	**115**
		Title, Objectives, Aim/ProblemStatement,Apparatus,Theory,Output etc…	**116-126**
		PART-B	
	2.11	Assignment No. B01	**127**

		Practical Session Plan	**127**
		Title,Objectives,Aim/ProblemStatement, Apparatus, Theory, Output etc…	**128-140**
	2.12	Assignment No. B02	**141**
		Practical Session Plan	**141**
		Title, Objectives, Aim/ProblemStatement,Apparatus,Theory,Output etc…	**142-151**
	2.13	Assignment No. B03	**151**
		Practical Session Plan	**151**
		Title, Objectives,Aim/ProblemStatement,Apparatus,Theory,Output etc…	**152-160**
	2.14	Assignment No. B04	**161**
		Practical Session Plan	**161**
		Title, Objectives,Aim/ProblemStatement,Apparatus,Theory,Output etc…	**162-180**
	2.15	Assignment No. B05 **(Content Beyond Syllabus)**	**181**
		Practical Session Plan	**181**
		Title, Objectives,Aim/ProblemStatement,Apparatus,Theory,Output etc…	**182-194**

RSCOE.IT

1. How to Use This Manual

This manual assumes that the facilitators are aware of collaborative learning methodologies.

This manual will provide a tool to facilitate the session on Python libraries and built-in functions required for computational statistics in collaborative learning environment.

The facilitator is expected to refer this manual before the session.

RSCOE-IT

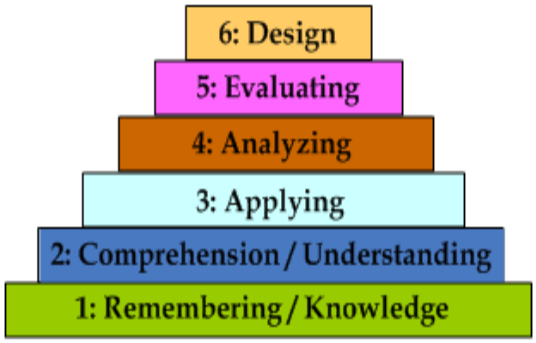
Icon of Graduate Attributes

K Applying Knowledge	**A** Problem Analysis	**D** Design & Development	**I** Investigation of problems
M Modern Tool Usage	**E** Engineer & Society	**E** Environment Sustainability	**T** Ethics
T Individual & Team work	**O** Communication	**M** Project Management & Finance	**I** Life Long Learning

Disk Approach- Digital Blooms Taxonomy



1: Remembering / Knowledge
2: Comprehension / Understanding
3: Applying
4: Analyzing
5: Evaluating
6: Creating / Design



This Manual uses icons as visual cues to the interactivities during the session.

	Blooms Taxonomy
	Remembering
	Understanding
	Applying
	Analyzing
	Evaluating
	Creating

☞	This icon is used to indicate instructions for faculties.
📣	This icon is used to indicate a statement to be made by faculty.
💻	This icon is used to indicate a list of additional resources.
🏋	This icon indicates an activity to be conducted.
⁉	This icon indicates questions to be asked by faculty.

VISION of Institute

To satisfy the aspirations of youth force, who want to lead the nation towards prosperity through techno economic development.

MISSION of Institute

To provide, nurture and maintain an environment of high academic excellence, research and entrepreneurship for all aspiring students, which will prepare them to face global challenges maintaining high ethical and moral standards.

VISION of Department

To create quality information technology professionals through superior academic environment.

MISSION of Department

To incorporate the IT fundamentals in students to be successful in their career. To motivate the students for higher studies, research and entrepreneurship. To provide IT services to society.

PSO's of Department

PSO1: Demonstrate the ability to apply discrete principles of mathematics along with programming paradigms to expedite solution building in the IT domain.

PSO2: Apply computational techniques using core aspects of network and system programming to deliver secured applications in the arena of analytics and computing.

PSO3: Demonstrate project management skills to handle multidisciplinary complex tasks proficiently and utilize these skills for entrepreneurship.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

PO 1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2. Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3. Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of Assignment s, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Prerequisites: Programming Fundamentals, Problem solving skills, Python programming

Laboratory Outcomes

LO1: Demonstrate Use of built-in-functins available in libraries like NumPy , Panda's, sklearn, scikit learn, seaborn, Sci-py

LO2: Compute Estimators of the main statistical measures like mean, variance, standard Deviations, Covariance, normal Distribution

LO3: Apply the concept of Dimension reduction for sample dataset using PCA Algorithm

LO4: Compare different analysis tools for Data visualization and feature Extraction.

2.1 Laboratory Objectives

Upon completion of Advanced Python Programming Lab student should able to:

1. To understand different built-in-functions available in libraries like NumPy , Panda's, sklearn, scikit learn, seaborn, Sci-py
2. To learn different built-in-functions required for computational statistics

RSCOE-IT

2. Lab Details

2.2 Laboratory Equipment list

Hardware	Specification
Processor	Intel Pentium 4 or Higher
Hard Drive	64 GB
Memory	4 GB or Higher
Internet Browser	Microsoft Internet Explorer 7.0 or Higher, Google Chrome or Mozilla Firefox or Apple Safari
Software	Google Colab,Anaconda, Python 3.7 and above

2.3 Laboratory Plan

Exp. No.	Title of Assignment
	Part A
1.	Study and implement functions of python NumPy library.
2.	Study and implement functions of python Panda's library.
3.	Study and implement functions of python Matplotlib library.
4.	Study and implement functions of python Scikit-learn library.
5.	Study and implement functions of python SciPy library.
6.	Study and implement functions of Seaborn library.
7.	Study and implement functions of TensorFlow library.
	Part B
8.	Compute Estimators of the main statistical measures like Mean, Variance, Standard Deviation, Covariance, Correlation and Standard error with respect to any example. Display graphically the distribution of samples.

9.	Plot the Normal Distribution for class test result of a particular subject. Identify the Skewness and Kurtosis
10.	Demonstrates how to do data correlation, rescaling and dimensionality reduction is done through PCA
11.	Using a suitable tool, perform data visualization, feature extraction analysis
12.	Study and implement Convolutional Neural Networks and Recurrent Neural Networks (**Content Beyond Syllabus**)
	End semester assessment Mock Practical Examination

RSCOE-IT

ASSIGNMENT NO. A01

Study and implement functions of python NumPy library.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Python NumPy Library

Objectives: To study different built-in functions available in library numpy.

Aim/ Problem Statement: Write a Python Program for Import Numpy and Perform Basic Operation On List.

Apparatus: Computer

Theory: Theory:

NumPy is a Python package. It stands for 'Numerical Python'. It is a library consisting of multidimensional array objects and a collection of routines for processing of array. Numeric, the ancestor of NumPy, was developed by Jim Hugunin. Another package Numarray was also developed, having some additional functionality.

Standard Python distribution doesn't come bundled with NumPy module. A lightweight alternative is to install NumPy using popular Python package installer, pip.

syntax: pip install numpy

The best way to enable NumPy is to use an installable binary package specific to your operating system. These binaries contain full SciPy stack (inclusive of NumPy, SciPy, matplotlib, IPython, SymPy and nose packages along with core Python).

NumPy package is imported using the following

syntax: import numpy as np

NUMPY − NDARRAY OBJECT:- The most important object defined in NumPy is an N-dimensional array type called ndarray. It describes the collection of items of the same type. Items in the collection can be accessed using a zero-based index.

It creates an ndarray from any object exposing array interface, or from any method that returns an array.

Syntax: numpy.array(object, dtype=None, copy=True, order=None, subok=False, ndmin=0)

The above constructor takes the following parameters:

object	Any object exposing the array interface method returns an array, or any (nested) sequence
dtype	Desired data type of array, optional
copy	Optional. By default (true), the object is copied
order	C (row major) or F (column major) or A (any) (default)
subok	By default, returned array forced to be a base class array. If true, sub-classes passed through
ndimin	Specifies minimum dimensions of resultant array

Example

```
import numpy as np
a=np.array([1,2,3])
print a
```

The output is as follows:

```
[1, 2, 3]
```

Example - more than one dimensions

```
import numpy as np
NumPy 12 a = np.array([[1, 2], [3, 4]])
print a
```

The output is as follows:

```
[[1, 2] [3, 4]]
```

Example - minimum dimensions

```
import numpy as np
a=np.array([1, 2, 3,4,5], ndmin=2)
print a
```

The output is as follows:

The output is as follows: [[1, 2, 3, 4, 5]]

NUMPY − DATA TYPES

NumPy supports a much greater variety of numerical types than Python does. The following table shows different scalar data types defined in NumPy.

bool_	Boolean (True or False) stored as a byte
int_	Default integer type (same as C long; normally either int64 or int32)
intc	Identical to C int (normally int32 or int64)
intp	Integer used for indexing (same as C ssize_t; normally either int32 or int64)
int8	Byte (-128 to 127)
int16	Integer (-32768 to 32767)
uint16	Unsigned integer (0 to 65535)
float_	Shorthand for float64
complex_	Shorthand for complex128

Data Type Objects(dtype)

A data type object describes interpretation of fixed block of memory corresponding to an array, depending on the following aspects:

- Type of data (integer, float or Python object)
- Size of data
- Byte order (little-endian or big-endian)
- In case of structured type, the names of fields, data type of each field and part of the memory block taken by each field
- If data type is a subarray, its shape and data type,

The byte order is decided by prefixing '<' or '>' to data type. '<' means that encoding is littleendian (least significant is stored in smallest address). '>' means that encoding is big-endian (most significant byte is stored in smallest address).

A dtype object is constructed using the following

syntax: numpy.dtype(object, align, copy)

The parameters are:

- Object: To be converted to data type object
- Align: If true, adds padding to the field to make it similar to C-struct

- Copy: Makes a new copy of dtype object. If false, the result is reference to built-in data type object

Example - using array-scalar type

```
import numpy as np
dt=np.dtype(np.int32)
print dt
```

The output is as follows:

int32

NUMPY − ARRAY ATTRIBUTES

ndarray.shape

This array attribute returns a tuple consisting of array dimensions. It can also be used to resize the array. Example 1

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print a.shape
```

The output is as follows:

(2, 3)

ndarray.ndim This array attribute returns the number of array dimensions.

Example 4 #

An array of evenly spaced numbers

```
import numpy as np a = np.arange(24)
print a
```

The output is as follows: [0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]

- A new ndarray object can be constructed by any of the following array creation routines or using a low-level ndarray constructor.
- **numpy.itemsize** This array attribute returns the length of each element of array in bytes. Example 6 # dtype of array is int8 (1 byte)

  ```
  import numpy as np x = np.array([1,2,3,4,5],
  dtype=np.int8) print x.itemsize
  ```

  The output is as follows:

  1

- **numpy.empty** It creates an uninitialized array of specified shape and dtype.

- **numpy.zeros** Returns a new array of specified size, filled with zeros.

  Syntax:- numpy.zeros(shape, dtype=float, order='C')

  The constructor takes the following parameters

  The constructor takes the following parameters.

Shape	Shape of an empty array in int or sequence of int
Dtype	Desired output data type. Optiona
Order	'C' for C-style row-major array, 'F' for FORTRAN style column-major array

  Example - array of five zeros. Default dtype is float

```
import numpy as np
x = np.zeros(5)
print x
```

  The output is as follows:

```
[ 0. 0. 0. 0. 0.]
```

  Example –

```
import numpy as np
x = np.ones([2,2], dtype=int)
print x
```

  the output would be as follows:

```
[1 1]
[1 1]
```

ARRAY FROM EXISTING DATA

numpy.asarray - This function is similar to numpy.array except for the fact that it has fewer parameters. This routine is useful for converting Python sequence into ndarray.

Syntax - numpy.asarray(a, dtype=None, order=None)

The constructor takes the following parameters.

a	Input data in any form such as list, list of tuples, tuples, tuple of tuples or tuple of lists
dtype	By default, the data type of input data is applied to the resultant ndarray
order	C (row major) or F (column major). C is default

Example - Convert list to ndarray

import numpy as np

x = [1,2,3]

a = np.asarray(x)

print a

Its output would be as follows:

[1 2 3]

- **numpy.frombuffer** This function interprets a buffer as one-dimensional array. Any object that exposes the buffer interface is used as parameter to return an

The constructor takes the following parameters.

buffer	Any object that exposes buffer interface
dtype	Data type of returned ndarray. Defaults to float
count	The number of items to read, default -1 means all data
offset	The starting position to read from. Default is 0

Sytanx :- ndarray. numpy.frombuffer(buffer, dtype=float, count=-1, offset=0)

Example - The following examples demonstrate the use of frombuffer function.

import numpy as np s = 'Hello World'

a = np.frombuffer(s, dtype='S1')

print a

Here is its output: ['H' 'e' 'l' 'l' 'o' ' ' 'W' 'o' 'r' 'l' 'd']

- **numpy.fromiter** - This function builds an ndarray object from any iterable object.

A new one-dimensional array is returned by this function.

Sytanx- numpy.fromiter(iterable, dtype, count=-1)

the constructor takes the following parameters.

iterable	Any iterable object
dtype	Data type of resultant array
count	The number of items to be read from iterator. Default is -1 which means all data to be read

The following examples show how to use the built-in range() function to return a list object. An iterator of this list is used to form an ndarray object.

Example - create list object using range function

```
import numpy as np
list = range(5)
print list
```

Its output is as follows: [0, 1, 2, 3, 4]

Example - Obtain iterator object from list

```
import numpy as np
list = range(5)
it = iter(list)
# use iterator to create ndarray
x = np.fromiter(it, dtype=float)
print x
```

Now, the output would be as follows:

[0. 1. 2. 3. 4.]

ARRAY FROM NUMERICAL RANGES

numpy.arange - This function returns an ndarray object containing evenly spaced values within a given range. The format of the function is as follows:

Syntax - numpy.arange(start, stop, step, dtype)

The constructor takes the following parameters

start	The start of an interval. If omitted, defaults to 0
stop	The end of an interval (not including this number)
step	Spacing between values, default is 1
dtype	Data type of resulting ndarray. If not given, data type of input is used

Example

```
import numpy as np
x = np.arange(5)
print x
```

Its output would be as follows:

[0 1 2 3 4]

Example

```
import numpy as np
# dtype set
x = np.arange(5, dtype=float)
print x
```

Here, the output would be:

```
[0. 1. 2. 3. 4.]
```

numpy.linspace - This function is similar to arange() function. In this function, instead of step size, the number of evenly spaced values between the interval is specified.

The usage of this function is as follows:

Sytanx :- numpy.linspace(start, stop, num, endpoint, retstep, dtype)

The constructor takes the following parameters.

start	The starting value of the sequence
stop	The end value of the sequence, included in the sequence if endpoint set to true
num	The number of evenly spaced samples to be generated. Default is 50
endpoint	True by default, hence the stop value is included in the sequence. If false, it is not included
retstep	If true, returns samples and step between the consecutive numbers
dtype	Data type of output ndarray

Conclusion- Thus, we have studied and implemented functions of python NumPy library.

ASSIGNMENT NO. A02

Study and implement functions of python Pandas library.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Python Pandas Library

Objectives: To study different built-in functions available in library Pandas.

Aim/ Problem Statement: Write a Python Program for Import Pandas and Perform Basic Operation.

Apparatus: Computer

Theory: Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures.

Prior to Pandas, Python was majorly used for data munging and preparation. It had very little contribution towards data analysis.

Standard Python distribution doesn't come bundled with Pandas module. A lightweight alternative is to install NumPy using popular Python package installer, **pip.**

Sytanx - pip install pandas

Pandas deals with the following three data structures −

- Series
- DataFrame
- Panel

These data structures are built on top of Numpy array, which means they are fast.

Dimension & Description

The best way to think of these data structures is that the higher dimensional data structure is a container of its lower dimensional data structure. For example, DataFrame is a container of Series, Panel is a container of DataFrame.

Data Structure	Dimensions	Description
Series	1	1D labeled homogeneous array, sizeimmutable.

Data Frames	2	General 2D labeled, size-mutable tabular structure with potentially heterogeneously typed columns.
Panel	3	General 3D labeled, size-mutable array.

Building and handling two or more dimensional arrays is a tedious task, burden is placed on the user to consider the orientation of the data set when writing functions. But using Pandas data structures, the mental effort of the user is reduced.

example, with tabular data (DataFrame) it is more semantically helpful to think of the **index** (the rows) and the **columns** rather than axis 0 and axis 1.

Mutability

All Pandas data structures are value mutable (can be changed) and except Series all are size mutable. Series is size immutable.

Note − DataFrame is widely used and one of the most important data structures. Panel is used much less.

Series

Series is a one-dimensional array like structure with homogeneous data. For example, the following series is a collection of integers 10, 23, 56, …

10	23	56	17	52	61	73	90	26	72

pandas.Series

A pandas Series can be created using the following constructor −

Syntax- pandas.Series(data, index, dtype, copy)

Example

```
#import the pandas library and aliasing as pd
import pandas as pd
s = pd.Series()
```

```
print s
```

Its **output** is as follows −

Series([], dtype: float64)

Create a Series from ndarray

If data is an ndarray, then index passed must be of the same length. If no index is passed, then by default index will be **range(n)** where **n** is array length, i.e., [0,1,2,3…. **range(len(array))-1].**

Example

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
data = np.array(['a','b','c','d'])
s = pd.Series(data)
print s
```

Its **output** is as follows –

0 a
1 b
2 c
3 d
dtype: object

Example

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
data = np.array(['a','b','c','d'])
s = pd.Series(data,index=[100,101,102,103])
print s
```

Its **output** is as follows −

100 a

101 b

102 c

103 d

dtype: object

pandas.DataFrame

A pandas DataFrame can be created using the following constructor −

pandas.DataFrame(data, index, columns, dtype, copy)

Create DataFrame

A pandas DataFrame can be created using various inputs like −

- Lists
- dict
- Series
- Numpy ndarrays
- Another DataFrame

In the subsequent sections of this chapter, we will see how to create a DataFrame using these inputs.

Create an Empty DataFrame

A basic DataFrame, which can be created is an Empty Dataframe.

Example

```
#import the pandas library and aliasing as pd
import pandas as pd
df = pd.DataFrame()
print df
```

Its **output** is as follows −

Empty DataFrame

Columns: []

Index: []

pandas.Panel()

A Panel can be created using the following constructor −

pandas.Panel(data, items, major_axis, minor_axis, dtype, copy)

The parameters of the constructor are as follows −

Parameter	Description
data	Data takes various forms like ndarray, series, map, lists, dict, constants and also another DataFrame
items	axis=0
major_axis	axis=1
minor_axis	axis=2
dtype	Data type of each column
copy	Copy data. Default, **false**

Create Panel

A Panel can be created using multiple ways like −

- From ndarrays
- From dict of DataFrames

From 3D ndarray

```
# creating an empty panel
import pandas as pd
import numpy as np
data = np.random.rand(2,4,5)
p = pd.Panel(data)
```

```
print p
```

Its **output** is as follows −

```
<class 'pandas.core.panel.Panel'>
Dimensions: 2 (items) x 4 (major_axis) x 5 (minor_axis)
Items axis: 0 to 1
Major_axis axis: 0 to 3
Minor_axis axis: 0 to 4
```

Python Pandas - Basic Functionality

Series Basic Functionality

Sr.No.	Attribute or Method & Description
1	**axes** Returns a list of the row axis labels
2	**dtype** Returns the dtype of the object.
3	**empty** Returns True if series is empty.
4	**ndim** Returns the number of dimensions of the underlying data, by definition 1.
5	**size** Returns the number of elements in the underlying data.

6	**values** Returns the Series as ndarray.
7	**head()** Returns the first n rows.
8	**tail()** Returns the last n rows.

Let us now create a Series and see all the above tabulated attributes operation.

Example

```
import pandas as pd
import numpy as np

#Create a series with 100 random numbers
s = pd.Series(np.random.randn(4))
print s
```

Its **output** is as follows −

```
0 0.967853
1 -0.148368
2 -1.395906
3 -1.758394
dtype: float64
```

size

Returns the size(length) of the series.

```
import pandas as pd
import numpy as np
```

```
#Create a series with 4 random numbers
s = pd.Series(np.random.randn(2))
print s
print ("The size of the object:")
print s.size
```

Its **output** is as follows −

```
0 3.078058
1 -1.207803
dtype: float64

The size of the object:
2
```

Python Pandas - Descriptive Statistics

A large number of methods collectively compute descriptive statistics and other related operations on DataFrame. Most of these are aggregations like **sum(), mean(),** but some of them, like **sumsum()**, produce an object of the same size. Generally speaking, these methods take an **axis** argument, just like *ndarray.{sum, std, ...},* but the axis can be specified by name or integer

DataFrame − "index" (axis=0, default), "columns" (axis=1)

Let us create a DataFrame and use this object throughout this chapter for all the operations.

Example

```
import pandas as pd
import numpy as np

#Create a Dictionary of series
d = {'Name':pd.Series(['Tom','James','Ricky','Vin','Steve','Smith','Jack',
```

```
   'Lee','David','Gasper','Betina','Andres']),
   'Age':pd.Series([25,26,25,23,30,29,23,34,40,30,51,46]),
   'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,4.6,3.8,3.78,2.98,4.80,4.10,3.65])
}

#Create a DataFrame
df = pd.DataFrame(d)
print df
```

Its **output** is as follows −

```
Age Name Rating
0 25 Tom 4.23
1 26 James 3.24
2 25 Ricky 3.98
3 23 Vin 2.56
4 30 Steve 3.20
5 29 Smith 4.60
6 23 Jack 3.80
7 34 Lee 3.78
8 40 David 2.98
9 30 Gasper 4.80
10 51 Betina 4.10
11 46 Andres 3.65
```

RSCOE-IT

Functions & Description

Let us now understand the functions under Descriptive Statistics in Python Pandas. The following table list down the important functions −

Sr.No.	Function	Description
1	count()	Number of non-null observations
2	sum()	Sum of values
3	mean()	Mean of Values
4	median()	Median of Values
5	mode()	Mode of values
6	std()	Standard Deviation of the Values
7	min()	Minimum Value
8	max()	Maximum Value
9	abs()	Absolute Value
10	prod()	Product of Values
11	cumsum()	Cumulative Sum
12	cumprod()	Cumulative Product

Conclusion- Thus, we have studied and implemented functions of python Pandas library**.**

RSCOE IT

ASSIGNMENT NO. A03

Study and implement functions of python Matplotlib library

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
40	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
50	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

RSCOE,IT

Title: Python Matplotlib library

Objectives: To demonstrate use of built in functions of python Matplotlib library

Aim/ Problem Statement: Write a Python program demonstrate functions of python Matplotlib library

Apparatus: Computer

Theory:

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib makes easy things easy and hard things possible.

- Create publication quality plots.
- Make interactive figures that can zoom, pan, update.
- Customize visual style and layout.
- Export to many file formats.
- Embed in JupyterLab and Graphical User Interfaces.
- Use a rich array of third-party packages built on Matplotlib.

Installation quick-start

Install using pip:

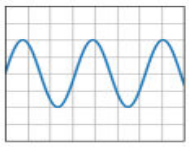
```
pip install matplotlib
```

Install using conda:

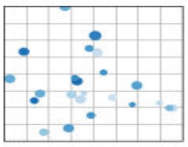
```
conda install matplotlib
```

Plot types

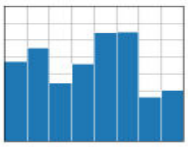
Basic



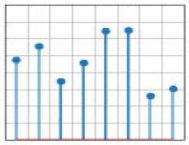
plot(x, y)



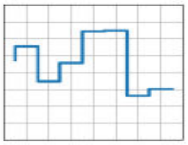
scatter(x, y)



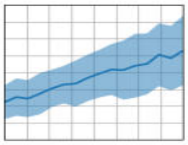
bar(x, height) / barh(y, width)



stem(x, y)



step(x, y)



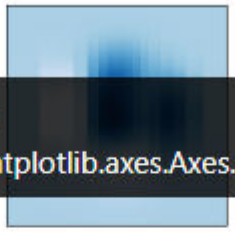
fill_between(x, y1, y2)

Plots of arrays and fields

Plotting for arrays of data `Z(x, y)` and fields `U(x, y)`, `V(x, y)`.



imshow(Z)



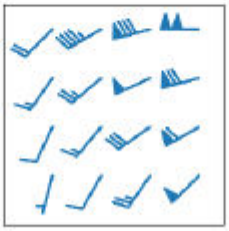
pcolormesh(X, Y, Z)



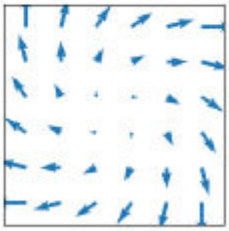
contour(X, Y, Z)



contourf(X, Y, Z)



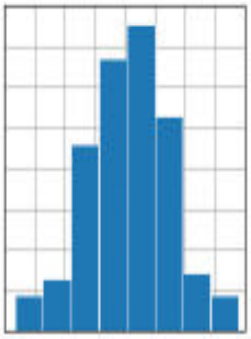
barbs(X, Y, U, V)



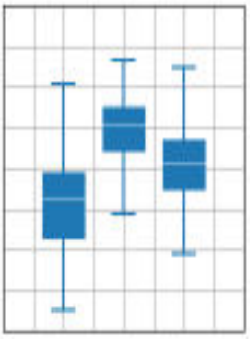
quiver(X, Y, U, V)

Statistics plots

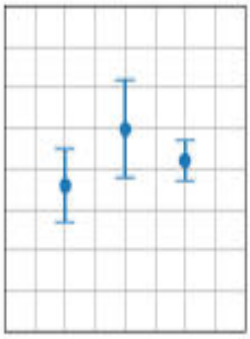
Plots for statistical analysis.



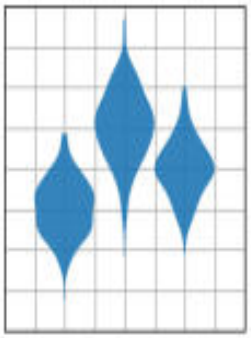
hist(x)



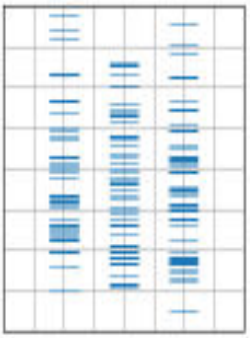
boxplot(X)



errorbar(x, y, yerr, xerr)



violinplot(D)



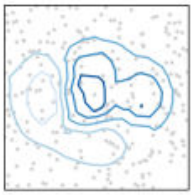
eventplot(D)



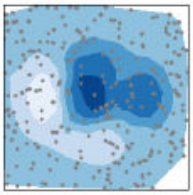
hist2d(x, y)

Unstructured coordinates

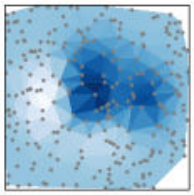
Sometimes we collect data `z` at coordinates `(x,y)` and want to visualize as a contour. Instead of gridding the data and then using `contour`, we can use a triangulation algorithm and fill the triangles.



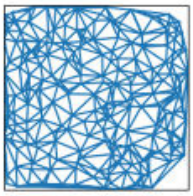
tricontour(x, y, z)



tricontourf(x, y, z)



tripcolor(x, y, z)



triplot(x, y)

Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

```
import matplotlib.pyplot as plt
```

Now the Pyplot package can be referred to as plt.

Basic plot types, usually y versus x.

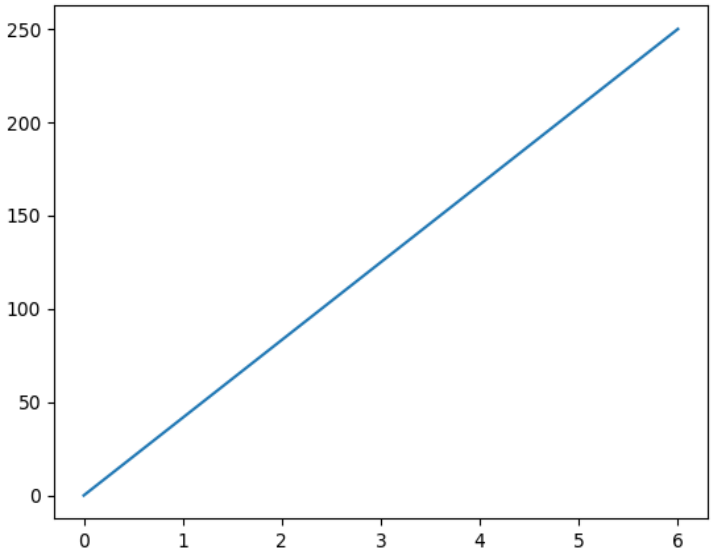
Example

1. Draw a line in a diagram from position (0,0) to position (6,250):

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([0, 6])
ypoints = np.array([0, 250])
```

```
plt.plot(xpoints, ypoints)
plt.show()
```

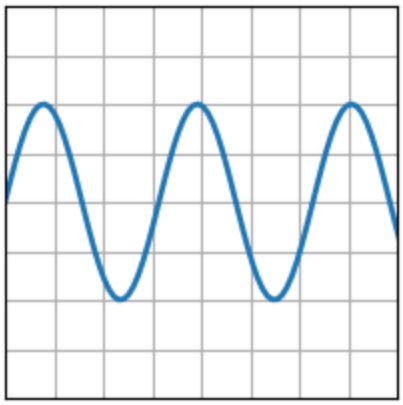
Output:



2. **Program to plot(x, y)**

```
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('_mpl-gallery')
# make data
x = np.linspace(0, 10, 100)
y = 4 + 2 * np.sin(2 * x)
# plot
fig, ax = plt.subplots()
ax.plot(x, y, linewidth=2.0)
ax.set(xlim=(0, 8), xticks=np.arange(1, 8),
       ylim=(0, 8), yticks=np.arange(1, 8))
plt.show()
```

Output:



Plotting x and y points

The plot() function is used to draw points (markers) in a diagram.

By default, the plot() function draws a line from point to point.

The function takes parameters for specifying points in the diagram.

Parameter 1 is an array containing the points on the **x-axis**.

Parameter 2 is an array containing the points on the **y-axis**.

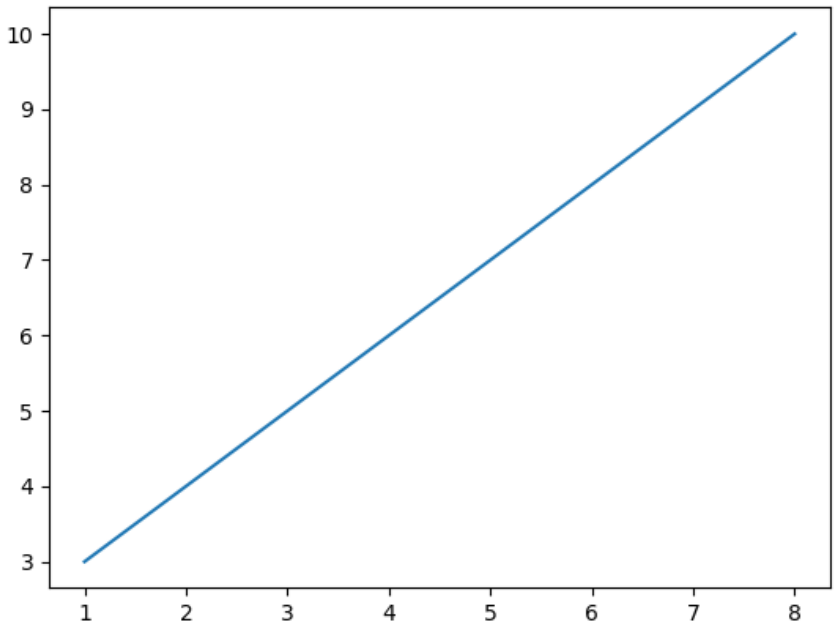
If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.

Example

Draw a line in a diagram from position (1, 3) to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])
plt.plot(xpoints, ypoints)
plt.show()
```

Result:



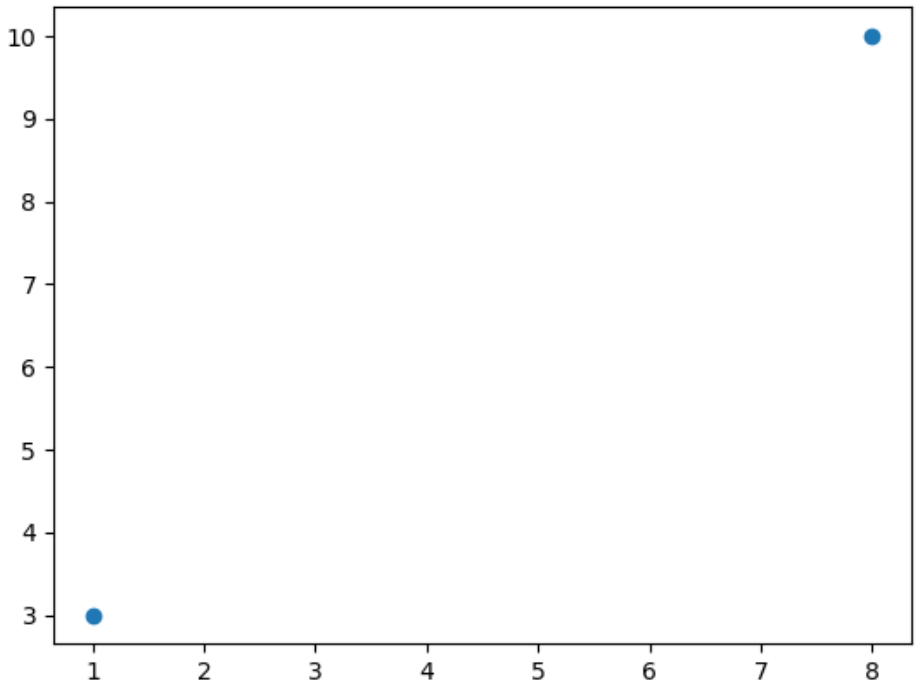
Plotting Without Line

To plot only the markers, you can use *shortcut string notation* parameter 'o', which means 'rings'.

Example

Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])
plt.plot(xpoints, ypoints, 'o')
plt.show()
```



Result:

Multiple Points

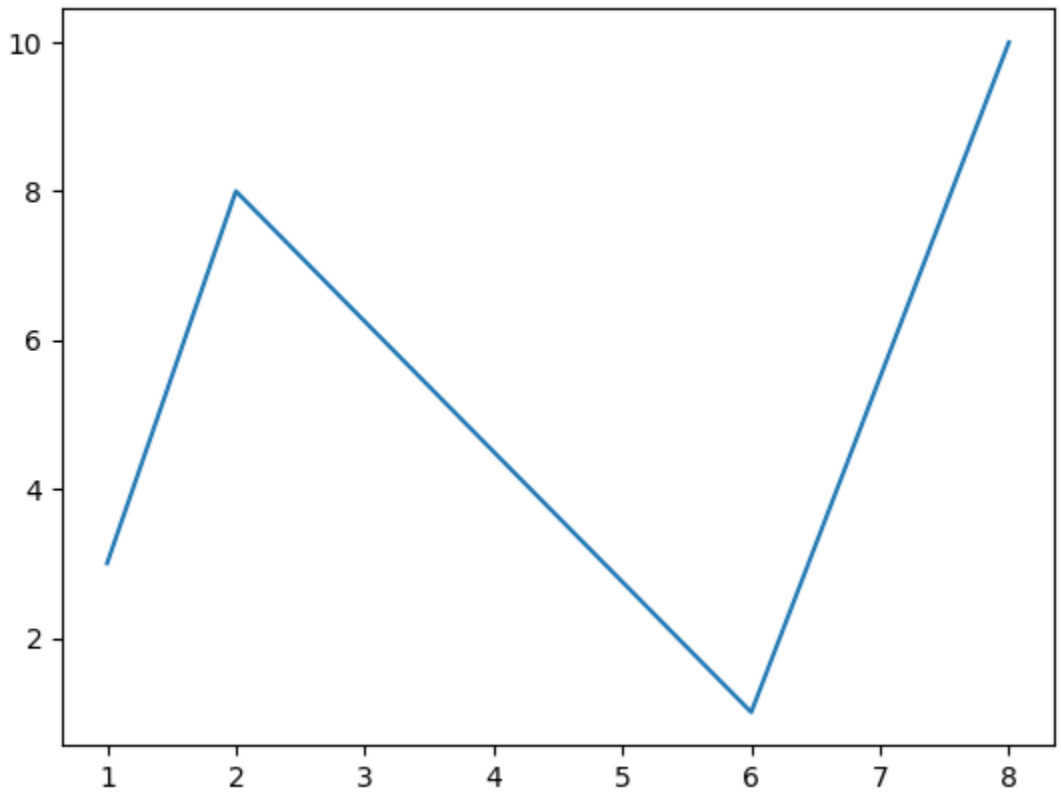
You can plot as many points as you like, just make sure you have the same number of points in both axis.

Example

Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])
plt.plot(xpoints, ypoints)
plt.show()
```

Result:



Default X-Points

If we do not specify the points in the x-axis, they will get the default values 0, 1, 2, 3, (etc. depending on the length of the y-points.

So, if we take the same example as above, and leave out the x-points, the diagram will look like this:

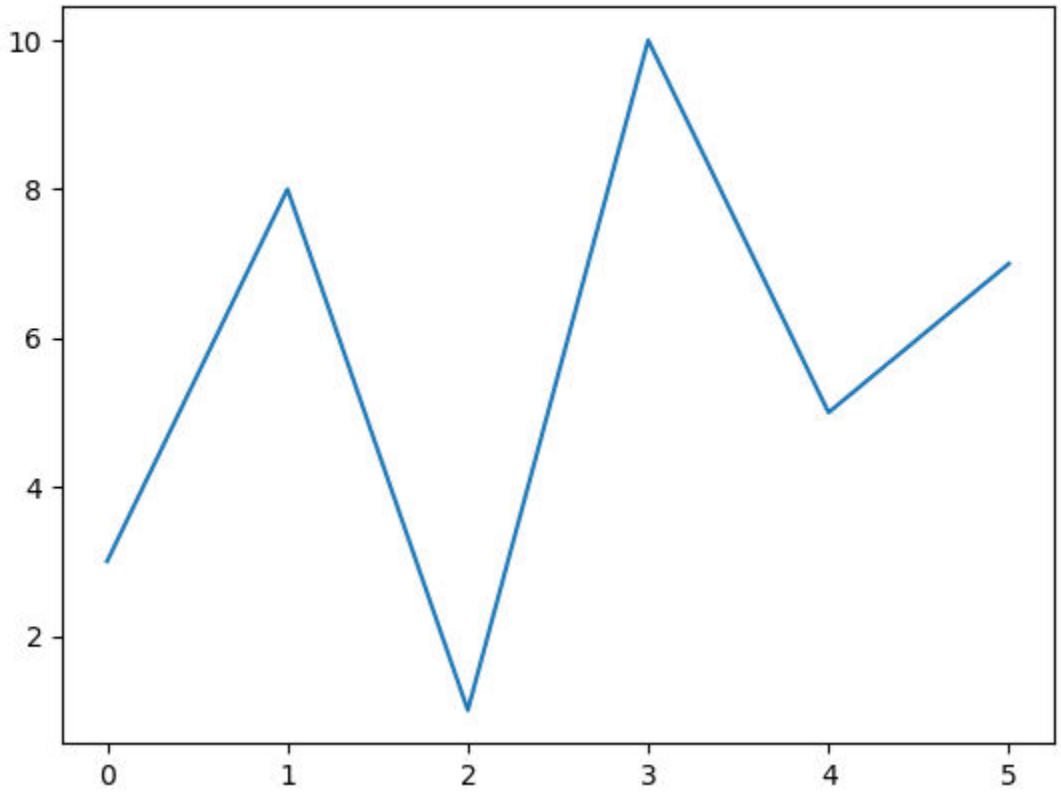
Example

Plotting without x-points:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
ypoints = np.array([3, 8, 1, 10, 5, 7])
plt.plot(ypoints)
plt.show()
```

Result:



Create Labels for a Plot

With Pyplot, you can use the xlabel() and ylabel() functions to set a label for the x- and y-axis.

Example

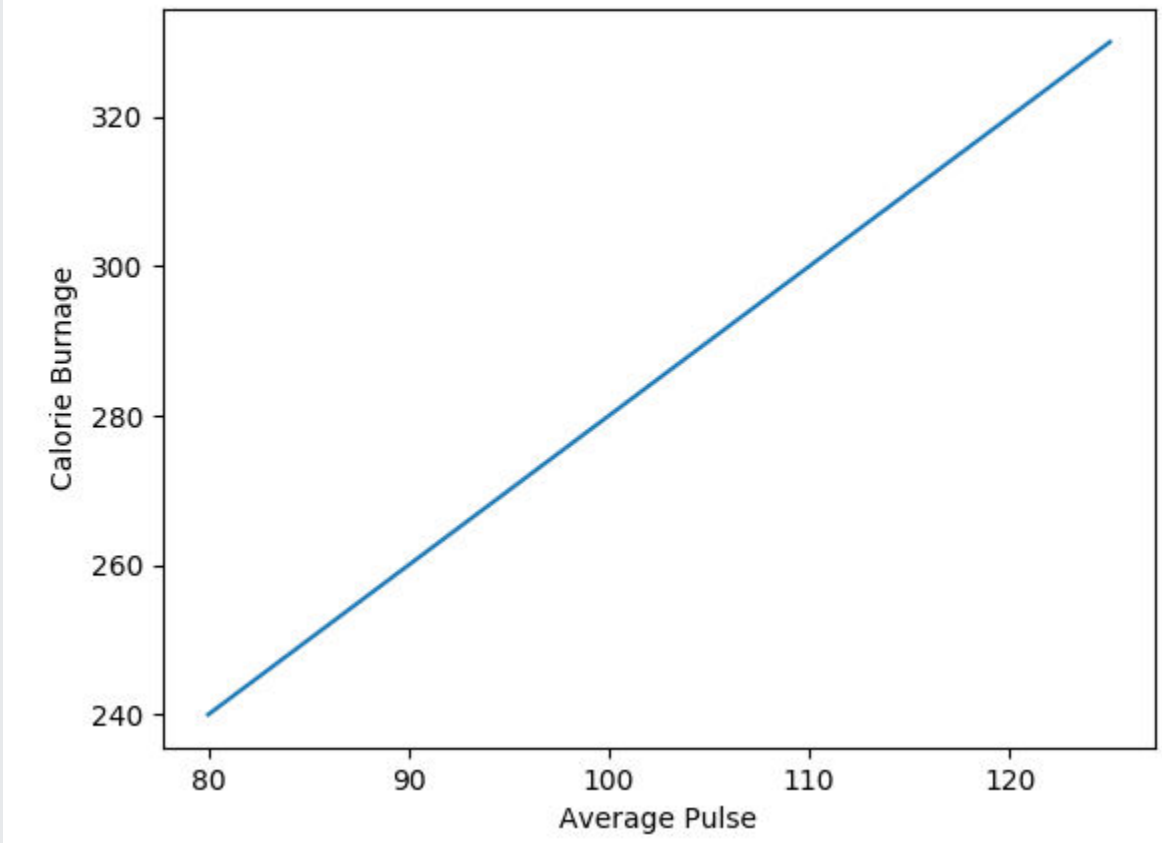
Add labels to the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])
plt.plot(x, y)
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

Result:



Create a Title for a Plot

With Pyplot, you can use the title() function to set a title for the plot.

Example

Add a plot title and labels for the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

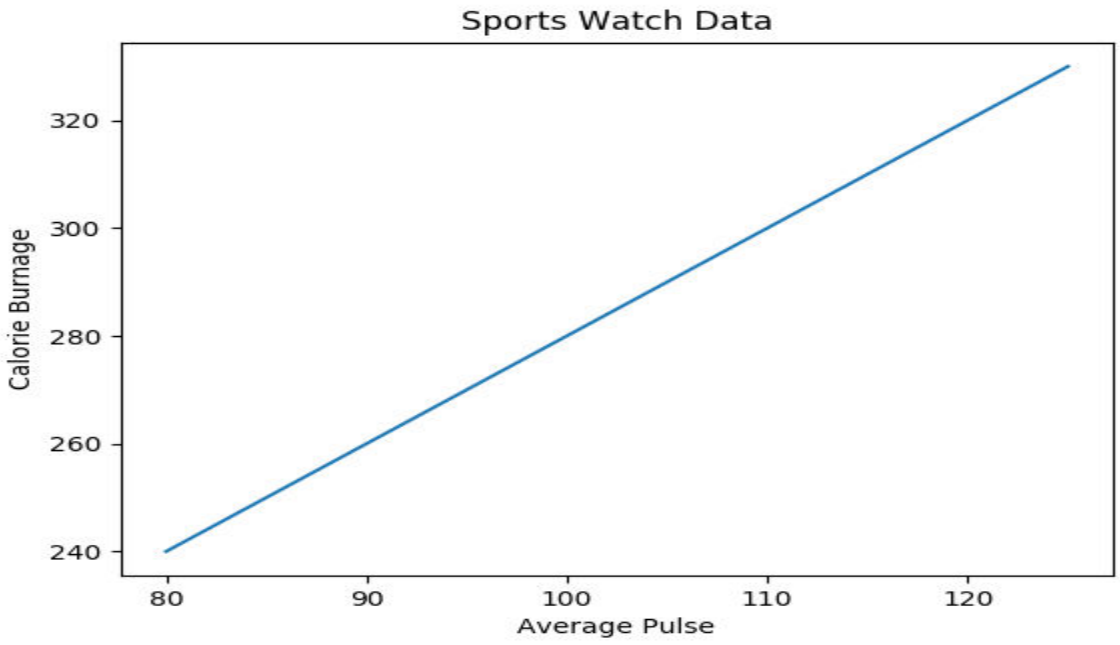
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

Result:



Add Grid Lines to a Plot

With Pyplot, you can use the grid() function to add grid lines to the plot.

Example

Add grid lines to the plot:

```
import numpy as np

import matplotlib.pyplot as plt


x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])

y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")

plt.xlabel("Average Pulse")

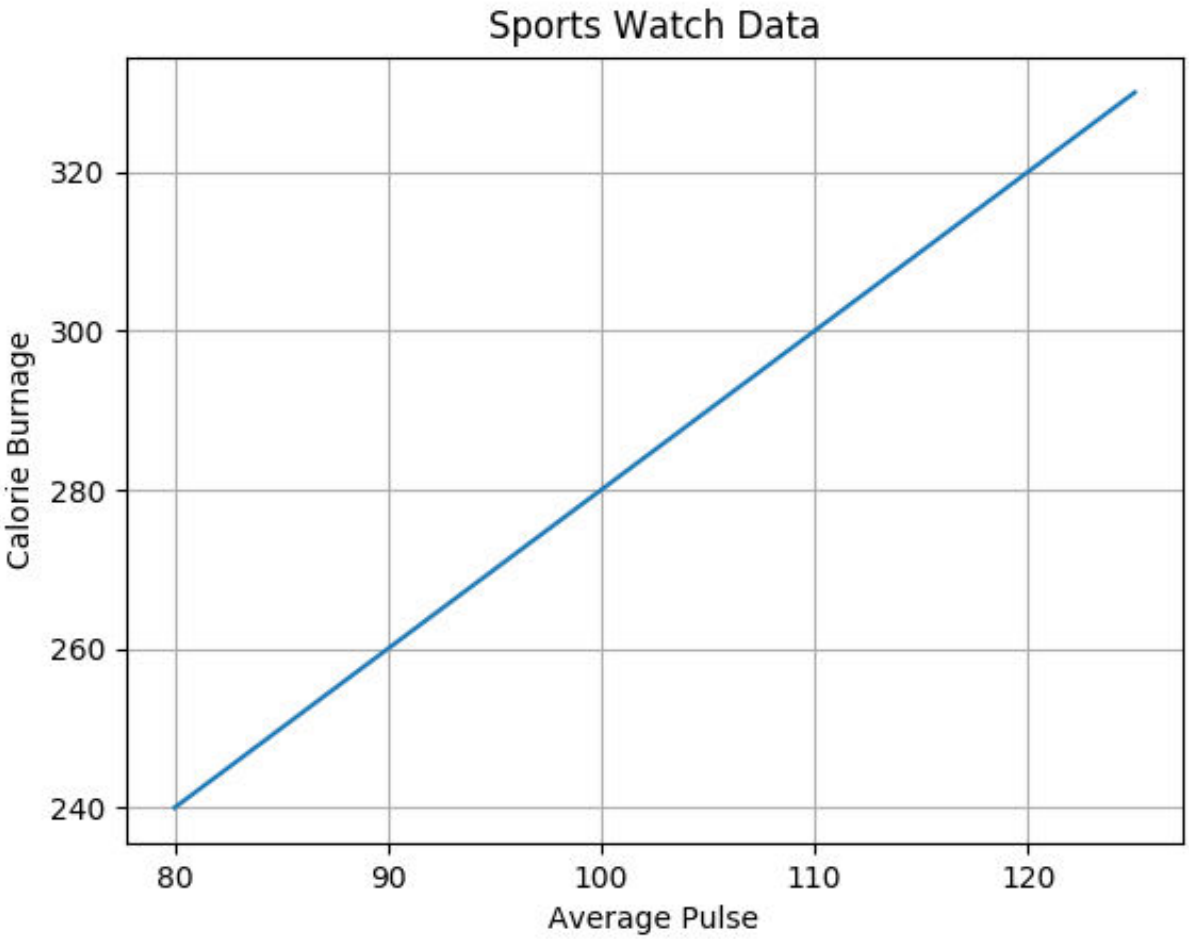
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid()

plt.show()
```

Result:

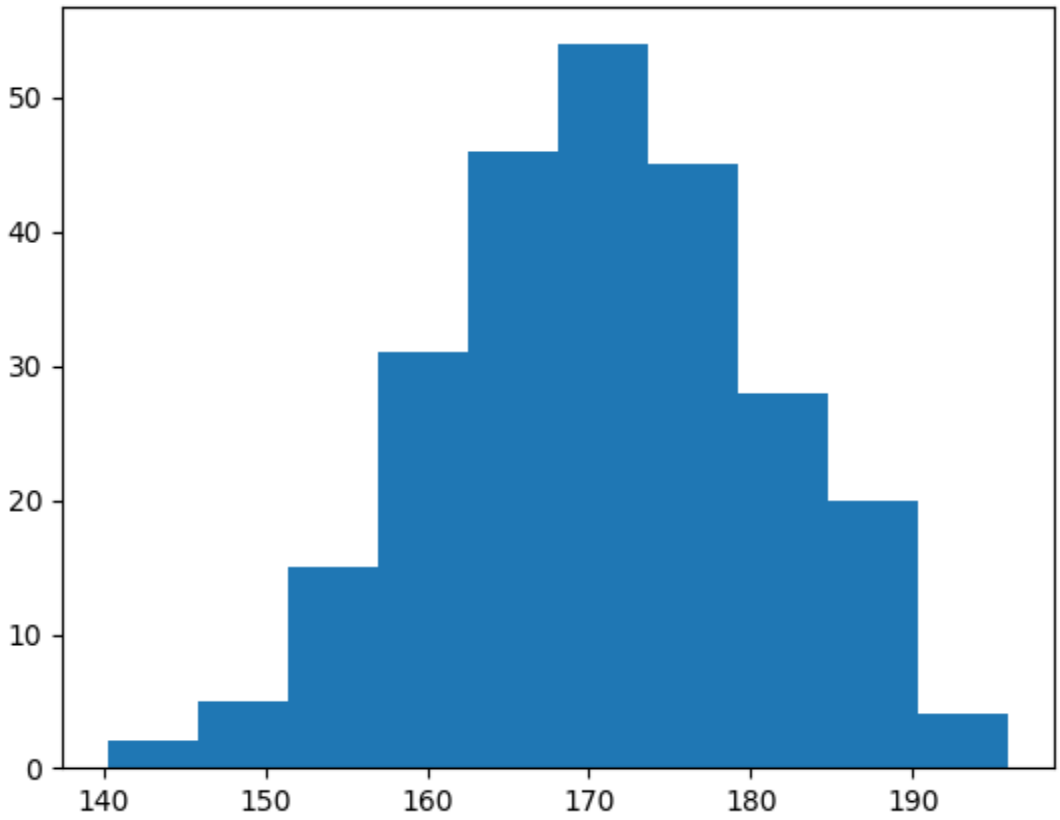


Histogram

A histogram is a graph showing *frequency* distributions.

It is a graph showing the number of observations within each given interval.

Example: Say you ask for the height of 250 people, you might end up with a histogram like this:



You can read from the histogram that there are approximately:

2 people from 140 to 145cm

5 people from 145 to 150cm

15 people from 151 to 156cm

31 people from 157 to 162cm

46 people from 163 to 168cm

53 people from 168 to 173cm

45 people from 173 to 178cm

28 people from 179 to 184cm

21 people from 185 to 190cm

4 people from 190 to 195cm

Creating Pie Charts

With Pyplot, you can use the pie() function to draw pie charts:

Example

A simple pie chart:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
plt.pie(y)
plt.show()
```

Result:



Conclusion: Thus we have studied and implemented functions of python Matplotlib library

ASSIGNMENT NO. A04

Study and implement functions of python Scikit-learn library.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
40	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
50	Coding of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Python Scikit-learn library

Objectives: To demonstrate use of built in functions of python Scikit-learn library.

Aim/ Problem Statement: Write a Python program to Study and implement functions of python Scikit-learn library

Apparatus: Computer

Theory:

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, and clustering and dimensionality reduction via a consistence interface in Python. This library, which is largely written in Python, is built upon **NumPy, SciPy** and **Matplotlib**.

Prerequisites

Before we start using scikit-learn latest release, we require the following −

- Python (>=3.5)
- NumPy (>= 1.11.0)
- Scipy (>= 0.17.0)li
- Joblib (>= 0.11)
- Matplotlib (>= 1.5.1) is required for Sklearn plotting capabilities.
- Pandas (>= 0.18.0) is required for some of the scikit-learn examples using data structure and analysis.

Installation

If you already installed NumPy and Scipy, following are the two easiest ways to install scikit-learn −

Using pip

Following command can be used to install scikit-learn via pip −

pip install -U scikit-learn

Using conda

Following command can be used to install scikit-learn via conda −

conda install scikit-learn

Features

Rather than focusing on loading, manipulating and summarising data, Scikit-learn library is focused on modeling the data. Some of the most popular groups of models provided by Sklearn are as follows −

Supervised Learning algorithms − Almost all the popular supervised learning algorithms, like Linear Regression, Support Vector Machine (SVM), Decision Tree etc., are the part of scikit-learn.

Unsupervised Learning algorithms − On the other hand, it also has all the popular unsupervised learning algorithms from clustering, factor analysis, PCA (Principal Component Analysis) to unsupervised neural networks.

Clustering − This model is used for grouping unlabeled data.

Cross Validation − It is used to check the accuracy of supervised models on unseen data.

Dimensionality Reduction − It is used for reducing the number of attributes in data which can be further used for summarisation, visualisation and feature selection.

Ensemble methods − As name suggest, it is used for combining the predictions of multiple supervised models.

Feature extraction − It is used to extract the features from data to define the attributes in image and text data.

Feature selection − It is used to identify useful attributes to create supervised models.

Open Source − It is open source library and also commercially usable under BSD license.

Dataset Loading

A collection of data is called dataset. It is having the following two components −

Features − The variables of data are called its features. They are also known as predictors, inputs or attributes.

- **Feature matrix** − It is the collection of features, in case there are more than one.
- **Feature Names** − It is the list of all the names of the features.

Response − It is the output variable that basically depends upon the feature variables. They are also known as target, label or output.

- **Response Vector** − It is used to represent response column. Generally, we have just one response column.
- **Target Names** − It represent the possible values taken by a response vector.

Scikit-learn have few example datasets like **iris** and **digits** for classification and the **Boston house prices** for regression.

Example

Following is an example to load **iris** dataset –

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names
print("Feature names:", feature_names)
print("Target names:", target_names)
print("\nFirst 10 rows of X:\n", X[:10])
```

Output:

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']
First 10 rows of X:
[
[5.1 3.5 1.4 0.2]
[4.9 3. 1.4 0.2]
```

```
[4.7 3.2 1.3 0.2]
[4.6 3.1 1.5 0.2]
[5. 3.6 1.4 0.2]
[5.4 3.9 1.7 0.4]
[4.6 3.4 1.4 0.3]
[5. 3.4 1.5 0.2]
[4.4 2.9 1.4 0.2]
[4.9 3.1 1.5 0.1]
]
```

Splitting the dataset

To check the accuracy of our model, we can split the dataset into two pieces-**a training set** and **a testing set**. Use the training set to train the model and testing set to test the model. After that, we can evaluate how well our model did.

Example

The following example will split the data into 70:30 ratio, i.e. 70% data will be used as training data and 30% will be used as testing data. The dataset is iris dataset as in above example.

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size = 0.3, random_state = 1
)
print(X_train.shape)
print(X_test.shape)
```

```
print(y_train.shape)

print(y_test.shape)
```

Output

(105, 4)

(45, 4)

(105,)

(45,)

As seen in the example above, it uses **train_test_split()** function of scikit-learn to split the dataset. This function has the following arguments −

- **X, y** − Here, **X** is the **feature matrix** and y is the **response vector**, which need to be split.
- **test_size** − This represents the ratio of test data to the total given data. As in the above example, we are setting **test_data = 0.3** for 150 rows of X. It will produce test data of 150*0.3 = 45 rows.
- **random_size** − It is used to guarantee that the split will always be the same. This is useful in the situations where you want reproducible results.

Train the Model

Next, we can use our dataset to train some prediction-model. As discussed, scikit-learn has wide range of **Machine Learning (ML) algorithms** which have a consistent interface for fitting, predicting accuracy, recall etc.

Example

In the example below, we are going to use KNN (K nearest neighbors) classifier. Don't go into the details of KNN algorithms, as there will be a separate chapter for that. This example is used to make you understand the implementation part only.

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
```

```
   X, y, test_size = 0.4, random_state=1
)
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
classifier_knn = KNeighborsClassifier(n_neighbors = 3)
classifier_knn.fit(X_train, y_train)
y_pred = classifier_knn.predict(X_test)
# Finding accuracy by comparing actual response values(y_test)with predicted response
value(y_pred)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
# Providing sample data and the model will make prediction out of that data
sample = [[5, 5, 3, 2], [2, 4, 3, 5]]
preds = classifier_knn.predict(sample)
pred_species = [iris.target_names[p] for p in preds] print("Predictions:", pred_species)
```

Output

```
Accuracy: 0.9833333333333333
Predictions: ['versicolor', 'virginica']
```

Model Persistence

Once you train the model, it is desirable that the model should be persist for future use so that we do not need to retrain it again and again. It can be done with the help of **dump** and **load** features of ***joblib*** package.

Consider the example below in which we will be saving the above trained model (classifier_knn) for future use −

```
from sklearn.externals import joblib
joblib.dump(classifier_knn, 'iris_classifier_knn.joblib')
```

The above code will save the model into file named iris_classifier_knn.joblib. Now, the object can be reloaded from the file with the help of following code −

```
joblib.load('iris_classifier_knn.joblib')
```

Preprocessing the Data

As we are dealing with lots of data and that data is in raw form, before inputting that data to machine learning algorithms, we need to convert it into meaningful data. This process is called preprocessing the data. Scikit-learn has package named **preprocessing** for this purpose. The **preprocessing** package has the following techniques −

Mean Removal

This technique is used to eliminate the mean from feature vector so that every feature centered on zero.

Example

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array(
  [2.1, -1.9, 5.5],
  [-1.5, 2.4, 3.5],
  [0.5, -7.9, 5.6],
  [5.9, 2.3, -5.8]]
)
#displaying the mean and the standard deviation of the input data
print("Mean =", input_data.mean(axis=0))
print("Stddeviation = ", input_data.std(axis=0))
#Removing the mean and the standard deviation of the input data

data_scaled = preprocessing.scale(input_data)
print("Mean_removed =", data_scaled.mean(axis=0))
print("Stddeviation_removed =", data_scaled.std(axis=0))
```

Output

Mean = [1.75 -1.275 2.2]

Stddeviation = [2.71431391 4.20022321 4.69414529]

Mean_removed = [1.11022302e-16 0.00000000e+00 0.00000000e+00]

Stddeviation_removed = [1. 1. 1.]

Scaling

We use this preprocessing technique for scaling the feature vectors. Scaling of feature vectors is important, because the features should not be synthetically large or small.

Example

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array(   [
    [2.1, -1.9, 5.5],
    [-1.5, 2.4, 3.5],
    [0.5, -7.9, 5.6],
    [5.9, 2.3, -5.8]
  ])
data_scaler_minmax = preprocessing.MinMaxScaler(feature_range=(0,1))
data_scaled_minmax = data_scaler_minmax.fit_transform(input_data)
print ("\nMin max scaled data:\n", data_scaled_minmax)
```

Output

Min max scaled data:

[

[0.48648649 0.58252427 0.99122807]

[0. 1. 0.81578947]

[0.27027027 0. 1.]

[1. 0.99029126 0.]

]

Conclusion: Thus we have studied and implemented functions of python Scikit-learn library

ASSIGNMENT NO. A05

Study and implement functions of python SciPy library.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
40	Explanation of Assignment and concepts	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
50	Coding of python SciPy library	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Study and implement functions of python SciPy library.

Objectives: To demonstrate python SciPy library functions

Aim/ Problem Statement: Write program for functions of python Scipy Library

Apparatus: Computer

Theory:

- Basics
- Sub-Packages
- Basic Functionality
- SciPy – Constants
- SciPy – FFTpack
- SciPy – Integrate
- SciPy – Interpolate
- SciPy - Input & Output
- SciPy – Linalg
- SciPy – Ndimage
- SciPy – CSGraph
- SciPy - ODR

Basics -:

SciPy, a scientific library for Python is an open source, BSD-licensed library for mathematics, science and engineering. The SciPy library depends on NumPy, which provides convenient and fast N-dimensional array manipulation. The main reason for building the SciPy library is that, it should work with NumPy arrays. It provides many user-friendly and efficient numerical practices such as routines for numerical integration and optimization

SciPy, pronounced as Sigh Pi, is a scientific python open source, distributed under the BSD licensed library to perform Mathematical, Scientific and Engineering Computations.

The SciPy library depends on NumPy, which provides convenient and fast N-dimensional array manipulation. The SciPy library is built to work with NumPy arrays and provides many user-friendly and efficient numerical practices such as routines for numerical integration and optimization

SciPy Sub-packages-:

SciPy is organized into sub-packages covering different scientific computing domains. These are summarized in the following table

scipy.cluster	Vector quantization / Kmeans
scipy.constants	Physical and mathematical constants
scipy.fftpack	Fourier transform
scipy.integrate	Integration routines
scipy.interpolate	Interpolation
scipy.io	Data input and output
scipy.linalg	Linear algebra routines
scipy.ndimage	n-dimensional image package
scipy.odr	Orthogonal distance regression
scipy.optimize	Optimization
scipy.signal	Signal processing
scipy.sparse	Sparse matrices
scipy.spatial	Spatial data structures and algorithms
scipy.special	Any special mathematical functions
scipy.stats	Statistics

SciPy - Environment Setup

Standard Python distribution does not come bundled with any SciPy module. A lightweight alternative is to install SciPy using the popular Python package installer,

pip install pandas

If we install the Anaconda Python package, Pandas will be installed by default. Following are the packages and links to install them in different operating systems.

Windows

Anaconda (from https://www.continuum.io) is a free Python distribution for the SciPy stack. It is also available for Linux and Mac.

Canopy (https://www.enthought.com/products/canopy/) is available free, as well as for commercial distribution with a full SciPy stack for Windows, Linux and Mac.

Python (x,y) − It is a free Python distribution with SciPy stack and Spyder IDE for Windows OS. (Downloadable from https://python-xy.github.io/)

Linux

Package managers of respective Linux distributions are used to install one or more packages in the SciPy stack.

Ubuntu

We can use the following path to install Python in Ubuntu.

sudo apt-get install python-numpy python-scipy

python-matplotlibipythonipython-notebook python-pandas python-sympy python-nose

Fedora

We can use the following path to install Python in Fedora.

sudo yum install numpyscipy python-matplotlibipython python-pandas

sympy python-nose atlas-devel

Basic Functionality-:

By default, all the NumPy functions have been available through the SciPy namespace. There is no need to import the NumPy functions explicitly, when SciPy is imported. The main object of NumPy is the homogeneous multidimensional array. It is a table of elements (usually numbers), all of the same type, indexed by a tuple of positive integers. In NumPy, dimensions are called as axes. The number of **axes** is called as **rank**. As SciPy is built on top of NumPy arrays, understanding of NumPy basics is necessary.

NumPy Vector

A Vector can be created in multiple ways. Some of them are described below.

Converting Python array-like objects to NumPy

```
import numpy as np

list = [1,2,3,4]

arr = np.array(list)

print arr
```

Output/Result : [1 2 3 4]

Intrinsic NumPy Array Creation

NumPy has built-in functions for creating arrays from scratch. Some of these functions are explained below.

Using zeros()

The zeros(shape) function will create an array filled with 0 values with the specified shape. The default dtype is float64. import numpy as np

```
print np.zeros((2, 3))
```

Output/Result : array([[0., 0., 0.],

[0., 0., 0.]])

Using ones()

The ones(shape) function will create an array filled with 1 values. It is identical to zeros in all the other respects. import numpy as np

print np.ones((2, 3))

Output/Result :

array([[1., 1., 1.],

[1., 1., 1.]])

Using arange()

The arange() function will create arrays with regularly incrementing values. import numpy as np

print np.arange(7)

Output/Result :array([0, 1, 2, 3, 4, 5, 6])

Defining the data type of the values

import numpy as np
arr = np.arange(2, 10, dtype = np.float)
print arr
print "Array Data Type :",arr.dtype
Output/Result : [2. 3. 4. 5. 6. 7. 8. 9.]
Array Data Type : float64

Using linspace()

The linspace() function will create arrays with a specified number of elements, which will be spaced equally between the specified beginning and end values.

import numpy as np

print np.linspace(1., 4., 6)

Output/Result :

array([1. , 1.6, 2.2, 2.8, 3.4, 4.])

Matrix

A matrix is a specialized 2-D array that retains its 2-D nature through operations. It has certain special operators, such as * (matrix multiplication) and ** (matrix power).

```
import numpy as np
print np.matrix('1 2; 3 4')
```

Output/Result :

matrix([[1, 2],
[3, 4]])

Conjugate Transpose of Matrix

This feature returns the (complex) conjugate transpose of self.

```
import numpy as np
mat = np.matrix('1 2; 3 4')
print mat.H
```

Output/Result :

matrix([[1, 3],
[2, 4]])

Transpose of Matrix

This feature returns the transpose of self.

```
import numpy as np
mat = np.matrix('1 2; 3 4')
mat.T
```

Output/Result :

matrix([[1, 3],
[2, 4]])

When we transpose a matrix, we make a new matrix whose rows are the columns of the original. A conjugate transposition, on the other hand, interchanges the row and the column index for each matrix element. The inverse of a matrix is a matrix that, if multiplied with the original matrix, results in an identity matrix.

SciPy - Constants

The scipy.constants package provides various constants. We have to import the required constant and use them as per the requirement.

```
#Import pi constant from both the packages

from scipy.constants import pi

from math import pi

print("sciPy - pi = %.16f"%scipy.constants.pi)

print("math - pi = %.16f"%math.pi)
```

Output/Result :

```
sciPy - pi = 3.1415926535897931

math - pi = 3.1415926535897931
```

SciPy – FFTpack -:

Fourier Transformation is computed on a time domain signal to check its behavior in the frequency domain. Fourier transformation finds its application in disciplines such as signal and noise processing, image processing, audio signal processing, etc. SciPy offers the fftpack module, which lets the user compute fast Fourier transforms.

Following is an example of a sine function, which will be used to calculate Fourier transform using the fftpack module.

One Dimensional Discrete Fourier Transform

The FFT y[k] of length N of the length-N sequence x[n] is calculated by fft() and the inverse transform is calculated using ifft().

```
#Importing the fft and inverse fft functions from fftpackage

from scipy.fftpack import fft

#create an array with random n numbers

x = np.array([1.0, 2.0, 1.0, -1.0, 1.5])

#Applying the fft function

y = fft(x)

print y
```

Output/Result-:

```
[ 4.50000000+0.j 2.08155948-1.65109876j -1.83155948+1.60822041j

-1.83155948-1.60822041j 2.08155948+1.65109876j ]
```

Discrete Cosine Transform

A Discrete Cosine Transform (DCT) expresses a finite sequence of data points in terms of a sum of cosine functions oscillating at different frequencies. SciPy provides a DCT with the function dct and a corresponding IDCT with the function idct.

```
from scipy.fftpack import dct

print dct(np.array([4., 3., 5., 10., 5., 3.]))
```

Output/Result :

```
array([ 60., -3.48476592, -13.85640646, 11.3137085, 6., -6.31319305])
```

The inverse discrete cosine transform reconstructs a sequence from its discrete cosine transform (DCT) coefficients. The idct function is the inverse of the dct function. Let us understand this with the following example.

```
from scipy.fftpack import dct

print idct(np.array([4., 3., 5., 10., 5., 3.]))
```

Output/Result :

```
array([ 39.15085889, -20.14213562, -6.45392043, 7.13341236,

8.14213562, -3.83035081])
```

SciPy – Integrate

When a function cannot be integrated analytically, or is very difficult to integrate analytically, one generally turns to numerical integration methods. SciPy has a number of routines for performing numerical integration. Most of them are found in the same scipy.integrate library.

The following table lists some commonly used functions.

Sr No.	Function & Description
1	**quad** Single integration
2	**dblquad** Double integration
3	**tplquad** Triple integration
4	**nquad** *n*-fold multiple integration
5	**fixed_quad**

	Gaussian quadrature, order n
6	**quadrature** Gaussian quadrature to tolerance
7	**romberg** Romberg integration
8	**trapz** Trapezoidal rule
9	**cumtrapz** Trapezoidal rule to cumulatively compute integral
10	**simps** Simpson's rule
11	**romb** Romberg integration
12	**polyint** Analytical polynomial integration (NumPy)
13	**poly1d** Helper function for polyint (NumPy)

SciPy – Interpolate

What is Interpolation?

Interpolation is the process of finding a value between two points on a line or a curve. To help us remember what it means, we should think of the first part of the word, 'inter,' as meaning 'enter,' which reminds us to look 'inside' the data we originally had. This tool, interpolation, is not only

useful in statistics, but is also useful in science, business, or when there is a need to predict values that fall within two existing data points.

Let us create some data and see how this interpolation can be done using the scipy.interpolate package.

```
import numpy as np

from scipy import interpolate

import matplotlib.pyplot as plt

x = np.linspace(0, 4, 12)

y = np.cos(x**2/3+4)

print x,y
```

Output/Result :

```
(

array([0., 0.36363636, 0.72727273, 1.09090909, 1.45454545, 1.81818182,

2.18181818, 2.54545455, 2.90909091, 3.27272727, 3.63636364, 4.]),

array([-0.65364362, -0.61966189, -0.51077021, -0.31047698, -0.00715476,

0.37976236, 0.76715099, 0.99239518, 0.85886263, 0.27994201,

-0.52586509, -0.99582185])

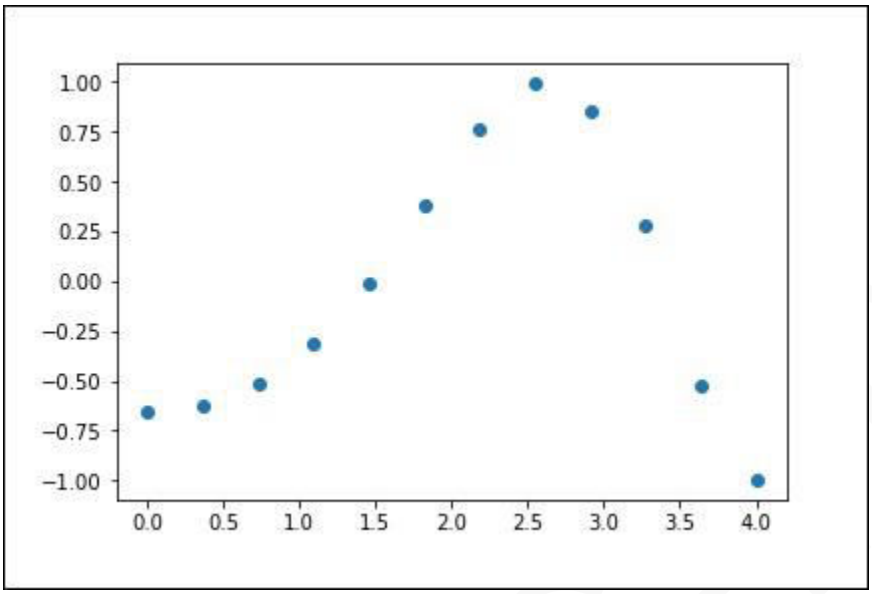
)
```

Now, we have two arrays. Assuming those two arrays as the two dimensions of the points in space, let us plot using the following program and see how they look like.

```
plt.plot(x, y,'o')

plt.show()
```

Output/Result-:



1-D Interpolation

The interp1d class in the scipy.interpolate is a convenient method to create a function based on fixed data points, which can be evaluated anywhere within the domain defined by the given data using linear interpolation.

By using the above data, let us create a interpolate function and draw a new interpolated graph.

f1 = interp1d(x, y,kind = 'linear')

f2 = interp1d(x, y, kind = 'cubic')

Using the interp1d function, we created two functions f1 and f2. These functions, for a given input x returns y. The third variable kind represents the type of the interpolation technique. 'Linear', 'Nearest', 'Zero', 'Slinear', 'Quadratic', 'Cubic' are a few techniques of interpolation.

Now, let us create a new input of more length to see the clear difference of interpolation. We will use the same function of the old data on the new data.

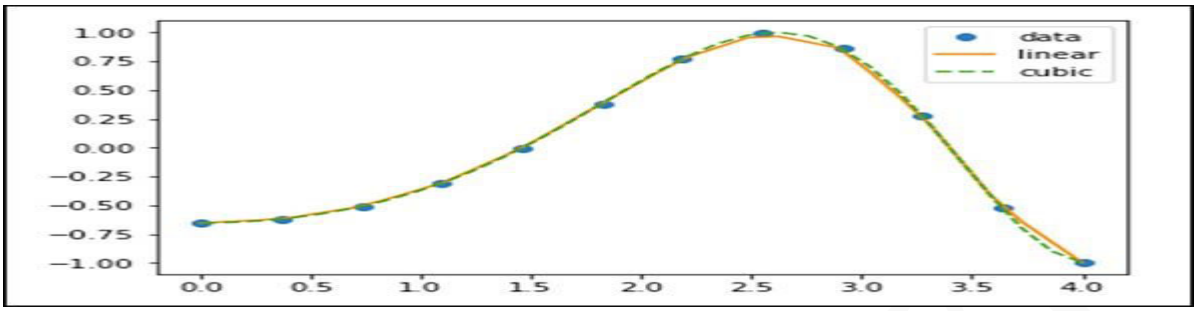
xnew = np.linspace(0, 4,30)

plt.plot(x, y, 'o', xnew, f(xnew), '-', xnew, f2(xnew), '--')

plt.legend(['data', 'linear', 'cubic','nearest'], loc = 'best')

plt.show()

Output/Result :



SciPy - Input & Output

The Scipy.io (Input and Output) package provides a wide range of functions to work around with different format of files. Some of these formats are −

Matlab,IDL,Matrix Market,Wave,Arff,Netcdf, etc.

Let us discuss in detail about the most commonly used file formats −

MATLAB

Following are the functions used to load and save a .mat file.

Sr No.	Function & Description
1	**loadmat** Loads a MATLAB file
2	**savemat** Saves a MATLAB file
3	**whosmat** Lists variables inside a MATLAB file

```
import scipy.io as sio

import numpy as np

#Save a mat file

vect = np.arange(10)

sio.savemat('array.mat', {'vect':vect})

#Now Load the File

mat_file_content = sio.loadmat('array.mat')

Print mat_file_content
```

Output/Result :

```
{

'vect': array([[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]]), '__version__': '1.0',

'__header__': 'MATLAB 5.0 MAT-file Platform: posix, Created on: Sat Sep 30

09:49:32 2017', '__globals__': []

}
```

We can see the array along with the Meta information. If we want to inspect the contents of a MATLAB file without reading the data into memory, use the whosmat command as shown below.

```
import scipy.io as sio

mat_file_content = sio.whosmat('array.mat')

print mat_file_content
```

Output/Result : [('vect', (1, 10), 'int64')]

SciPy - Linalg

SciPy is built using the optimized ATLAS LAPACK and BLAS libraries. It has very fast linear algebra capabilities. All of these linear algebra routines expect an object that can be converted into a two-dimensional array. The output of these routines is also a two-dimensional array.

SciPy.linalg vs NumPy.linalg

A scipy.linalg contains all the functions that are in numpy.linalg. Additionally, scipy.linalg also has some other advanced functions that are not in numpy.linalg. Another advantage of using scipy.linalg over numpy.linalg is that it is always compiled with BLAS/LAPACK support, while for NumPy this is optional. Therefore, the SciPy version might be faster depending on how NumPy was installed.

Linear Equations

The scipy.linalg.solve feature solves the linear equation a * x + b * y = Z, for the unknown x, y values.

As an example, assume that it is desired to solve the following simultaneous equations.

x + 3y + 5z = 10

2x + 5y + z = 8

2x + 3y + 8z = 3

To solve the above equation for the x, y, z values, we can find the solution vector using a matrix inverse as shown below.

$$\begin{bmatrix} x\\ y\\ z \end{bmatrix} = \begin{bmatrix} 1 & 3 & 5\\ 2 & 5 & 1\\ 2 & 3 & 8 \end{bmatrix}^{-1} \begin{bmatrix} 10\\ 8\\ 3 \end{bmatrix} = \frac{1}{25} \begin{bmatrix} -232\\ 129\\ 19 \end{bmatrix} = \begin{bmatrix} -9.28\\ 5.16\\ 0.76 \end{bmatrix}.$$

However, it is better to use the linalg.solve command, which can be faster and more numerically stable.

The solve function takes two inputs ‘a’ and ‘b’ in which ‘a’ represents the coefficients and ‘b’ represents the respective right hand side value and returns the solution array.

Let us consider the following example.

```
#importing the scipy and numpy packages

from scipy import linalg

import numpy as np

#Declaring the numpy arrays

a = np.array([[3, 2, 0], [1, -1, 0], [0, 5, 1]])

b = np.array([2, 4, -1])

#Passing the values to the solve function

x = linalg.solve(a, b)

#printing the result array

print x
```

Output/Result :

```
array([ 2., -2., 9.])
```

SciPy - Ndimage

The SciPy ndimage submodule is dedicated to image processing. Here, ndimage means an n-dimensional image.

Some of the most common tasks in image processing are as follows .

Input/Output, displaying images

Basic manipulations − Cropping, flipping, rotating, etc.

Image filtering − De-noising, sharpening, etc.

Image segmentation − Labeling pixels corresponding to different objects

Classification

Feature extraction

Registration

Opening and Writing to Image Files

The misc package in SciPy comes with some images. We use those images to learn the image manipulations. from scipy import misc

f = misc.face()

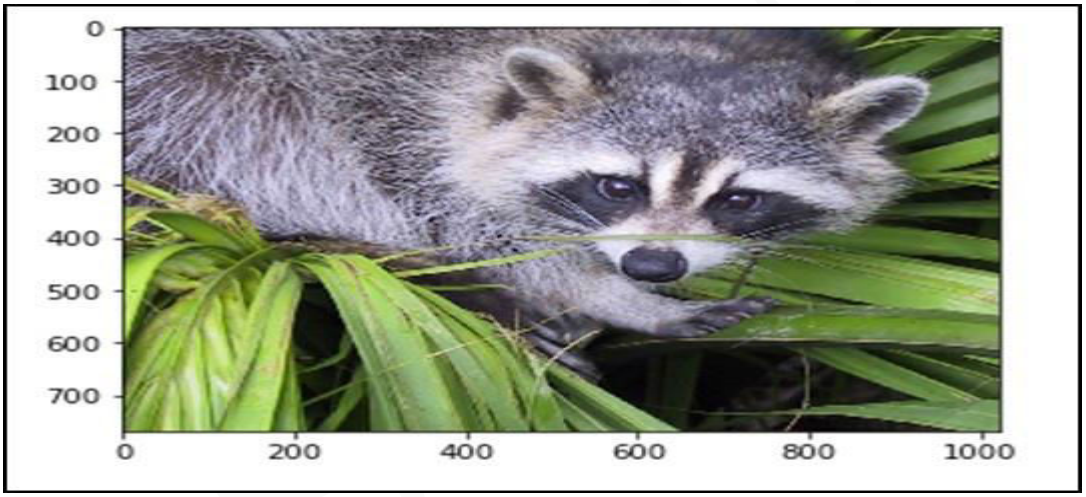
misc.imsave('face.png', f) # uses the Image module (PIL)

import matplotlib.pyplot as plt

plt.imshow(f)

plt.show()

Output/Result :



Any images in its raw format is the combination of colors represented by the numbers in the matrix format. A machine understands and manipulates the images based on those numbers only. RGB is a popular way of representation.

Let us see the statistical information of the above image.

from scipy import misc

face = misc.face(gray = False)

print face.mean(), face.max(), face.min()

The above program will generate the following output.

110.16274388631184, 255, 0

Now, we know that the image is made out of numbers, so any change in the value of the number alters the original image. Let us perform some geometric transformations on the image. The basic geometric operation is cropping

```
from scipy import misc

face = misc.face(gray = True)

lx, ly = face.shape

# Cropping

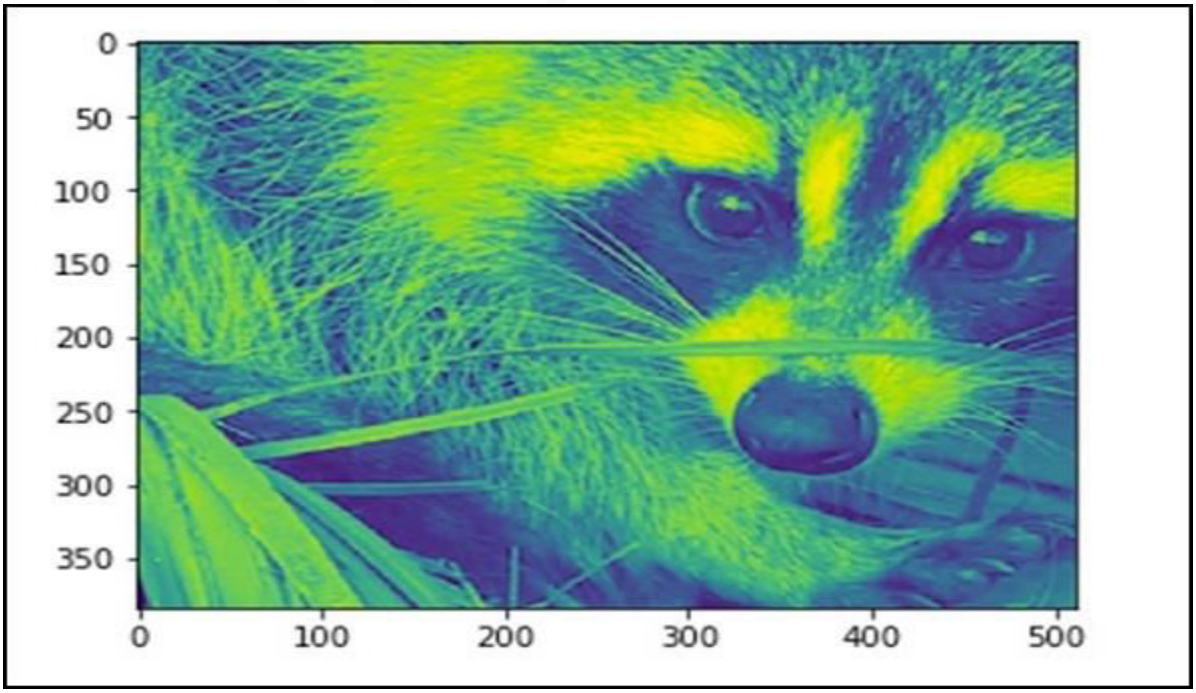
crop_face = face[lx / 4: - lx / 4, ly / 4: - ly / 4]

import matplotlib.pyplot as plt

plt.imshow(crop_face)

plt.show()
```

Output/Result :



We can also perform some basic operations such as turning the image upside down as described below.

```
# up <-> down flip

from scipy import misc

face = misc.face()

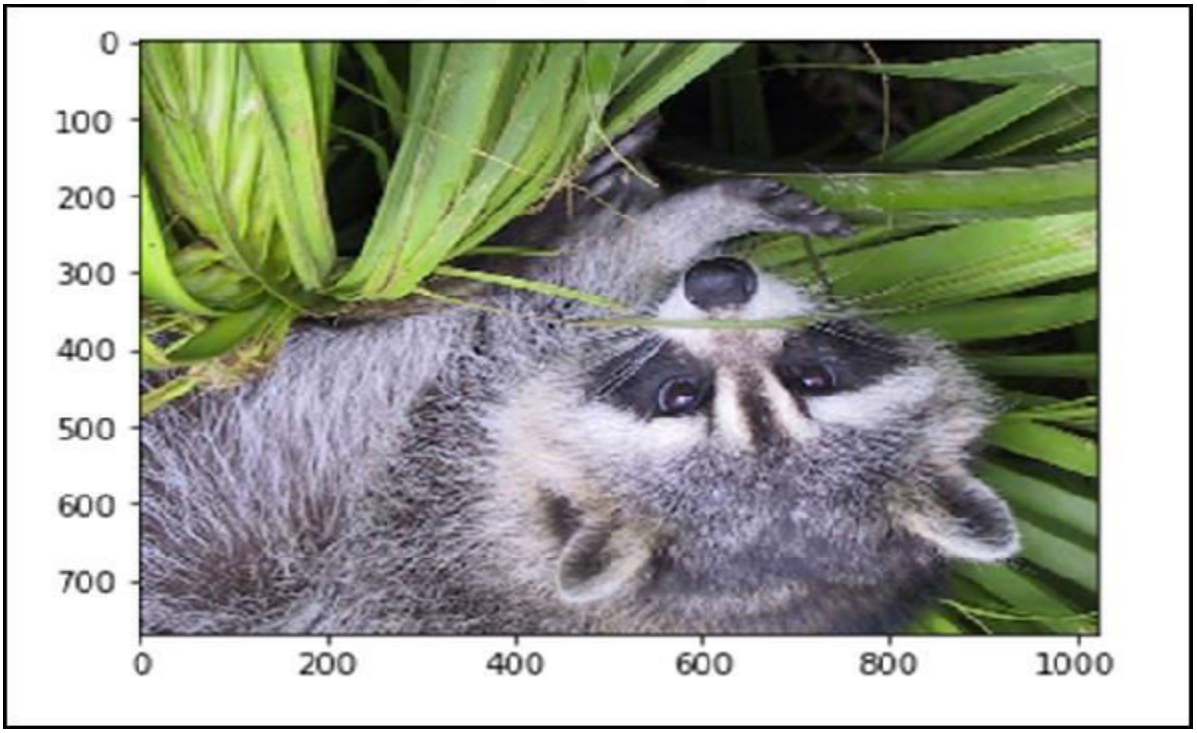
flip_ud_face = np.flipud(face)

import matplotlib.pyplot as plt

plt.imshow(flip_ud_face)

plt.show()
```

Output/Result :



Besides this, we have the rotate() function, which rotates the image with a specified angle.

```
# rotation

from scipy import misc,ndimage

face = misc.face()

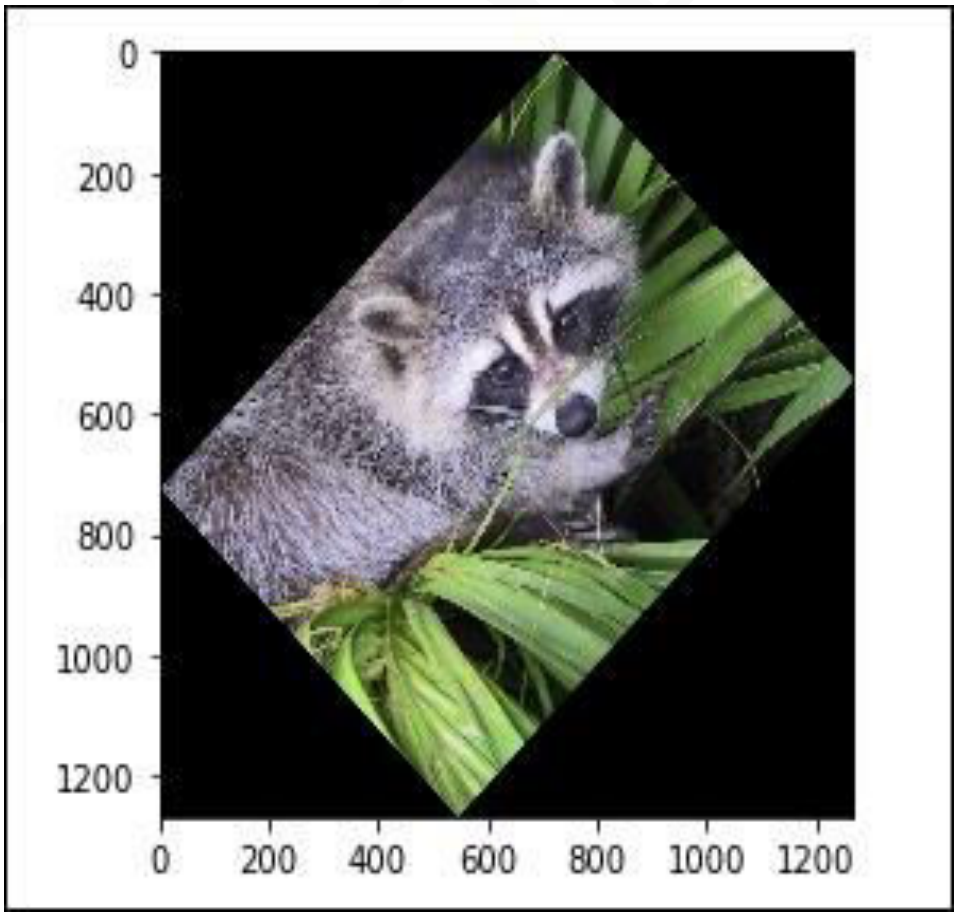
rotate_face = ndimage.rotate(face, 45)

import matplotlib.pyplot as plt

plt.imshow(rotate_face)

plt.show()
```

Output/Result :



Filters

Let us discuss how filters help in image processing.

What is filtering in image processing?

Filtering is a technique for modifying or enhancing an image. For example, you can filter an image to emphasize certain features or remove other features. Image processing operations implemented with filtering include Smoothing, Sharpening, and Edge Enhancement.

Filtering is a neighborhood operation, in which the value of any given pixel in the output image is determined by applying some algorithm to the values of the pixels in the neighborhood of the corresponding input pixel. Let us now perform a few operations using SciPy ndimage.

Blurring

Blurring is widely used to reduce the noise in the image. We can perform a filter operation and see the change in the image. Let us consider the following example.

```
from scipy import misc

face = misc.face()

blurred_face = ndimage.gaussian_filter(face, sigma=3)

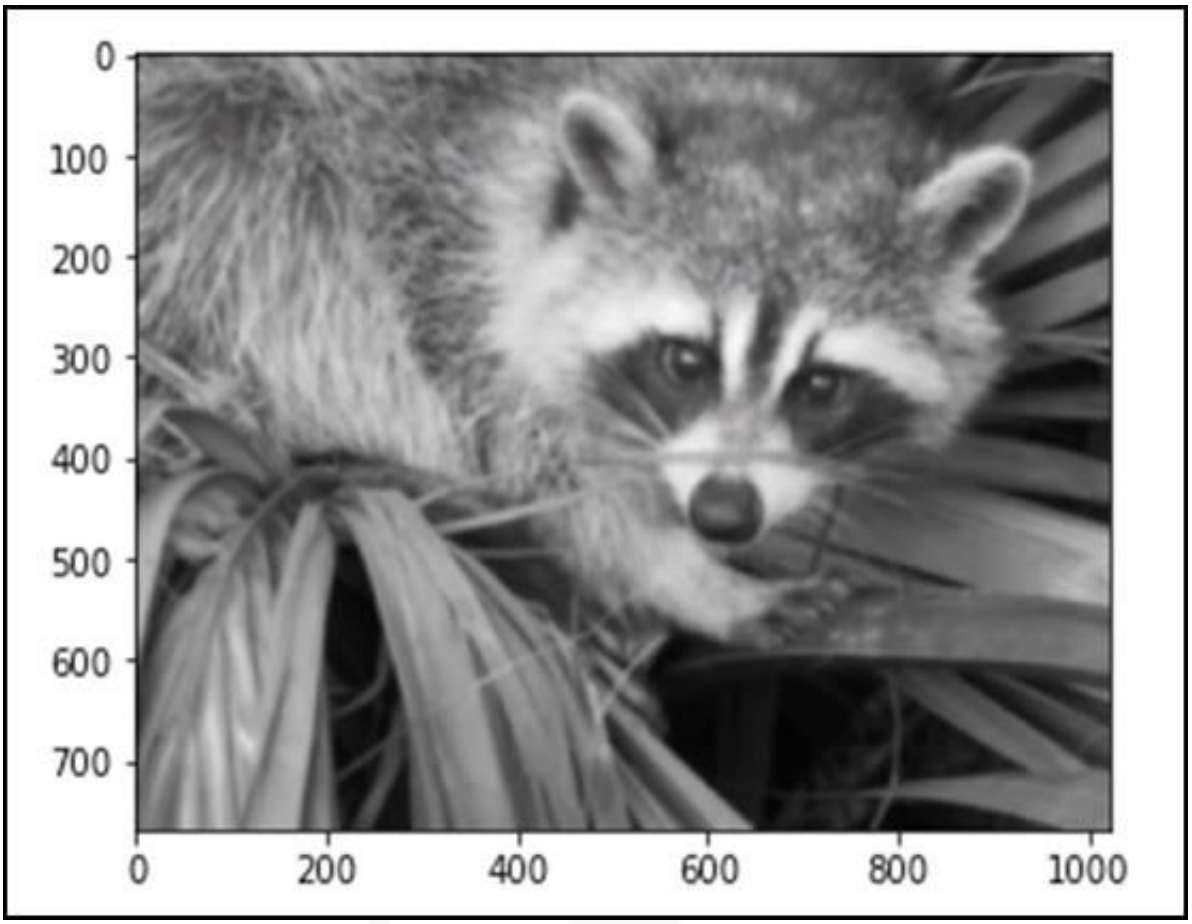
import matplotlib.pyplot as plt

plt.imshow(blurred_face)

plt.show()
```

RSCOE-IT

Output/Result :



The sigma value indicates the level of blur on a scale of five. We can see the change on the image quality by tuning the sigma value. For more details of blurring, click on → DIP (Digital Image Processing) Tutorial.

Edge Detection

Let us discuss how edge detection helps in image processing.

What is Edge Detection?

Edge detection is an image processing technique for finding the boundaries of objects within images. It works by detecting discontinuities in brightness. Edge detection is used for image segmentation and data extraction in areas such as Image Processing, Computer Vision and Machine Vision.

The most commonly used edge detection algorithms include

- Sobel
- Canny
- Prewitt
- Roberts
- Fuzzy Logic methods

Let us consider the following example.

```
import scipy.ndimage as nd

import numpy as np

im = np.zeros((256, 256))

im[64:-64, 64:-64] = 1

im[90:-90,90:-90] = 2

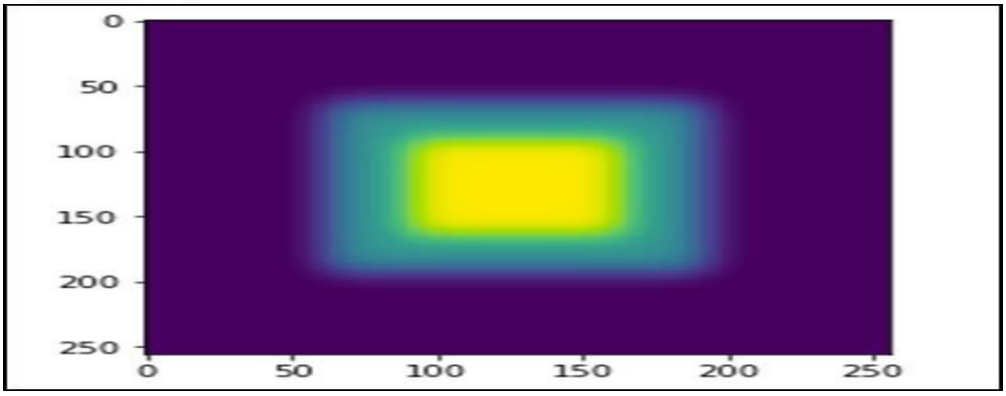
im = ndimage.gaussian_filter(im, 8)

import matplotlib.pyplot as plt

plt.imshow(im)

plt.show()
```

Output/Result-:



The image looks like a square block of colors. Now, we will detect the edges of those colored blocks. Here, ndimage provides a function called **Sobel** to carry out this operation. Whereas, NumPy provides the **Hypot** function to combine the two resultant matrices to one.

Let us consider the following example.

```
import scipy.ndimage as nd

import matplotlib.pyplot as plt

im = np.zeros((256, 256))

im[64:-64, 64:-64] = 1

im[90:-90,90:-90] = 2

im = ndimage.gaussian_filter(im, 8)

sx = ndimage.sobel(im, axis = 0, mode = 'constant')

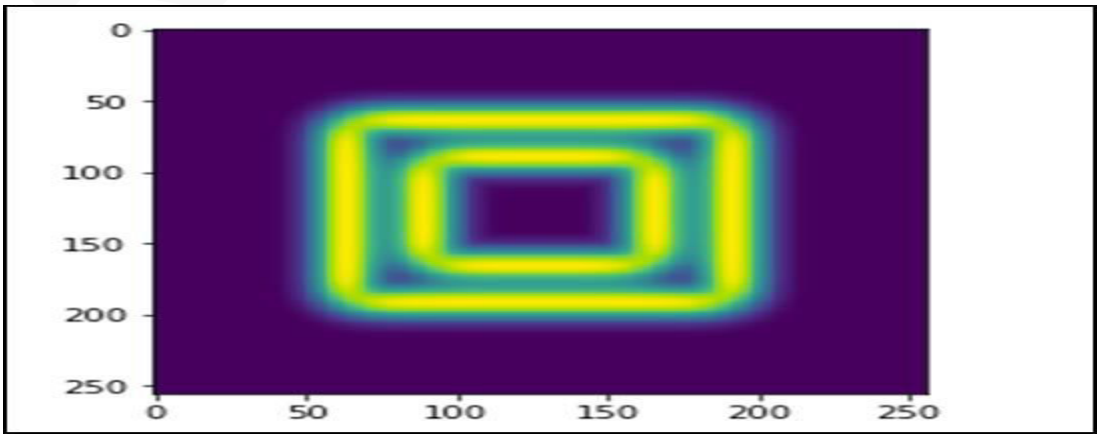
sy = ndimage.sobel(im, axis = 1, mode = 'constant')

sob = np.hypot(sx, sy)

plt.imshow(sob)

plt.show()
```

Output/Result :



SciPy - CSGraph

CSGraph stands for **Compressed Sparse Graph**, which focuses on Fast graph algorithms based on sparse matrix representations.

Graph Representations

To begin with, let us understand what a sparse graph is and how it helps in graph representations.

What exactly is a sparse graph?

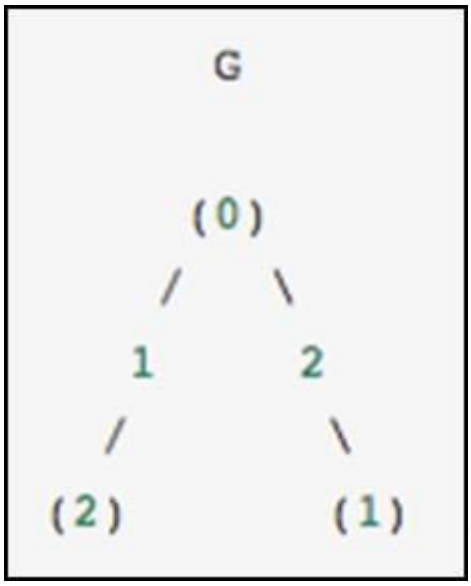
A graph is just a collection of nodes, which have links between them. Graphs can represent nearly anything − social network connections, where each node is a person and is connected to acquaintances; images, where each node is a pixel and is connected to neighboring pixels; points in a high-dimensional distribution, where each node is connected to its nearest neighbors; and practically anything else you can imagine.

One very efficient way to represent graph data is in a sparse matrix: let us call it G. The matrix G is of size N x N, and G[i, j] gives the value of the connection between node 'i' and node 'j'. A sparse graph contains mostly zeros − that is, most nodes have only a few connections. This property turns out to be true in most cases of interest.

The creation of the sparse graph submodule was motivated by several algorithms used in scikit-learn that included the following −

- **Isomap** − A manifold learning algorithm, which requires finding the shortest paths in a graph.
- **Hierarchical clustering** − A clustering algorithm based on a minimum spanning tree.
- **Spectral Decomposition** − A projection algorithm based on sparse graph laplacians.

As a concrete example, imagine that we would like to represent the following undirected graph −



This graph has three nodes, where node 0 and 1 are connected by an edge of weight 2, and nodes 0 and 2 are connected by an edge of weight 1. We can construct the dense, masked and sparse representations as shown in the following example, keeping in mind that an undirected graph is represented by a symmetric matrix.

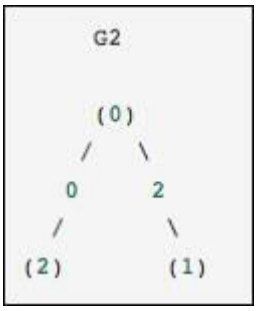
```
G_dense = np.array([ [0, 2, 1],
                     [2, 0, 0],
                     [1, 0, 0] ])


G_masked = np.ma.masked_values(G_dense, 0)
from scipy.sparse import csr_matrix


G_sparse = csr_matrix(G_dense)
print G_sparse.data
```

Output/Result:

array([2, 1, 2, 1])



This is identical to the previous graph, except nodes 0 and 2 are connected by an edge of zero weight. In this case, the dense representation above leads to ambiguities − how can non-edges be represented, if zero is a meaningful value. In this case, either a masked or a sparse representation must be used to eliminate the ambiguity.

Let us consider the following example.

```
from scipy.sparse.csgraph import csgraph_from_dense
G2_data = np.array
([
  [np.inf, 2, 0 ],
  [2, np.inf, np.inf],
  [0, np.inf, np.inf]
])
G2_sparse = csgraph_from_dense(G2_data, null_value=np.inf)
print G2_sparse.data
```

The above program will generate the following output.

array([2., 0., 2., 0.])

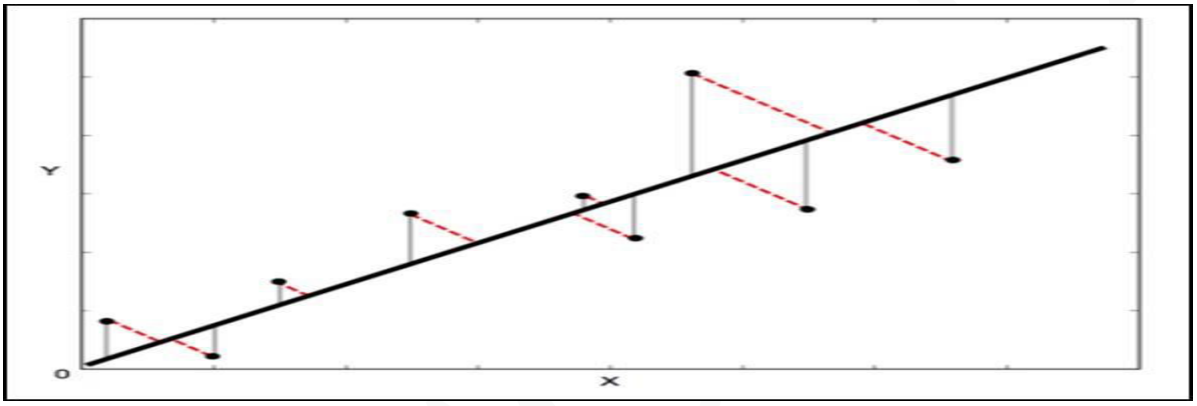
SciPy - ODR

ODR stands for **Orthogonal Distance Regression**, which is used in the regression studies. Basic linear regression is often used to estimate the relationship between the two variables **y** and **x** by drawing the line of best fit on the graph.

The mathematical method that is used for this is known as **Least Squares**, and aims to minimize the sum of the squared error for each point. The key question here is how do you calculate the error (also known as the residual) for each point?

In a standard linear regression, the aim is to predict the Y value from the X value – so the sensible thing to do is to calculate the error in the Y values (shown as the gray lines in the following image). However, sometimes it is more sensible to take into account the error in both X and Y (as shown by the dotted red lines in the following image).

For example − When you know your measurements of X are uncertain, or when you do not want to focus on the errors of one variable over another.



Orthogonal Distance Regression (ODR) is a method that can do this (orthogonal in this context means perpendicular – so it calculates errors perpendicular to the line, rather than just 'vertically').

scipy.odr Implementation for Univariate Regression

The following example demonstrates scipy.odr implementation for univariate regression.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.odr import *
import random

# Initiate some data, giving some randomness using random.random().
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([i**2 + random.random() for i in x])
```

```
# Define a function (quadratic in our case) to fit the data with.
def linear_func(p, x):
  m, c = p
  return m*x + c

# Create a model for fitting.
linear_model = Model(linear_func)

# Create a RealData object using our initiated data from above.
data = RealData(x, y)

# Set up ODR with the model and data.
odr = ODR(data, linear_model, beta0=[0., 1.])

# Run the regression.
out = odr.run()

# Use the in-built pprint method to give us results.
out.pprint()
```

Output/Result :

Beta: [5.51846098 -4.25744878]
Beta Std Error: [0.7786442 2.33126407]

Beta Covariance: [
[1.93150969 -4.82877433]
[-4.82877433 17.31417201
]]

Residual Variance: 0.313892697582
Inverse Condition #: 0.146618499389
Reason(s) for Halting:
Sum of squares convergence

SciPy - Special Package-:

The functions available in the special package are universal functions, which follow broadcasting and automatic array looping.

Let us look at some of the most frequently used special functions −

Cubic Root Function

Exponential Function

Log Sum Exponential Function

Cubic Root Function

The syntax of this cubic root function is – scipy.special.cbrt(x). This will fetch the element-wise cube root of x.

```
from scipy.special import cbrt

res = cbrt([10, 9, 0.1254, 234])

print res
```

Output/Result-:[2.15443469 2.08008382 0.50053277 6.16224015]

Exponential Function

The syntax of the exponential function is – scipy.special.exp10(x). This will compute 10**x element wise.

```
from scipy.special import exp10

res = exp10([2, 9])

print res
```

Output/Result-:[1.00000000e+02 1.00000000e+09]

Log Sum Exponential Function

The syntax for this function is – scipy.special.logsumexp(x). It helps to compute the log of the sum of exponentials of input elements.

```
from scipy.special import logsumexp

import numpy as np

a = np.arange(10)

res = logsumexp(a)

print res
```

Output/Result-:9.45862974443

Observations: SciPy, a scientific library for Python is useful for mathematics, science and engineering.

Conclusion: Thus we have studied and implement functions of python SciPy library.

RSCOE-IT

ASSIGNMENT NO. A06

Study and implement functions of Seaborn library.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
40	Explanation of Assignment and concepts	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
50	Coding of python Seaborn library	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

PROOF

Title: Study and implement functions of Seaborn library.

Objectives: To demonstrate python Seaborn library functions

Aim/ Problem Statement: Write a python program to demonstrate Seaborn library functions

Apparatus: Computer

Theory:

Introduction

Seaborn - Environment Setup

import Datasets and Libraries

Seaborn - Figure Aesthetic

Seaborn - Color Palette

Seaborn – Histogram

Seaborn - Kernel Density Estimates

Seaborn - FacetGrid

Seaborn - Pair Grid

Introduction

To analyse a set of data using Python, we make use of Matplotlib, a widely implemented 2D plotting library. Likewise, Seaborn is a visualization library in Python. It is built on top of Matplotlib.

Seaborn Vs Matplotlib

It is summarized that if Matplotlib "tries to make easy things easy and hard things possible",

Seaborn tries to make a well-defined set of hard things easy too."

Seaborn helps resolve the two major problems faced by Matplotlib; the problems are −Default Matplotlib parameters,Working with data frames

As Seaborn compliments and extends Matplotlib, the learning curve is quite gradual. If you know Matplotlib, you are already half way through Seaborn.

Important Features of Seaborn

Seaborn is built on top of Python's core visualization library Matplotlib. It is meant to serve as a complement, and not a replacement. However, Seaborn comes with some very important features. Let us see a few of them here.

The features help in −

Built in themes for styling matplotlib graphics

Visualizing univariate and bivariate data

Fitting in and visualizing linear regression models

Plotting statistical time series data

Seaborn works well with NumPy and Pandas data structures

It comes with built in themes for styling Matplotlib graphics

Seaborn - Environment Setup

Installing Seaborn and getting started

In this section, we will understand the steps involved in the installation of Seaborn.

Using Pip Installer

To install the latest release of Seaborn, you can use pip −

pip install seaborn

For Windows, Linux & Mac using Anaconda

Anaconda (from https://www.anaconda.com/ is a free Python distribution for SciPy stack. It is also available for Linux and Mac.

It is also possible to install the released version using conda −conda install seaborn

To install the development version of Seaborn directly from github

https://github.com/mwaskom/seaborn"

import Datasets and Libraries

Importing Libraries

Seaborn comes handy when dealing with DataFrames, which is most widely used data structure for data analysis.

```
# Pandas for managing datasets

import pandas as pd

# Matplotlib for additional customization

from matplotlib import pyplot as plt
```

We will import the Seaborn library with the following command −

```
# Seaborn for plotting and styling

import seaborn as sb
```

Importing Datasets

Seaborn comes with a few important datasets in the library. When Seaborn is installed, the datasets download automatically.With the help of the following function you can load the required dataset

```
load_dataset()
```

Importing Data as Pandas DataFrame

Here, will import a dataset. This dataset loads as Pandas DataFrame by default. If there is any function in the Pandas DataFrame, it works on this DataFrame.

The following line of code will help you import the dataset −

```
# Seaborn for plotting and styling
```

```
import seaborn as sb

df = sb.load_dataset('tips')

print df.head()
```

Output/Result-:total_bill tip sex smoker day time size

0 16.99 1.01 Female No Sun Dinner 2

1 10.34 1.66 Male No Sun Dinner 3

2 21.01 3.50 Male No Sun Dinner 3

3 23.68 3.31 Male No Sun Dinner 2

4 24.59 3.61 Female No Sun Dinner 4

To view all the available data sets in the Seaborn library, you can use the following command with the get_dataset_names() function as shown below −

```
import seaborn as sb

print sb.get_dataset_names()
```

Output/Result-: [u'anscombe', u'attention', u'brain_networks', u'car_crashes', u'dots',

u'exercise', u'flights', u'fmri', u'gammas', u'iris', u'planets', u'tips',

u'titanic']

Seaborn - Figure Aesthetic

Visualizing data is one step and further making the visualized data more pleasing is another step. Visualization plays a vital role in communicating quantitative insights to an audience to catch their attention.

Aesthetics means a set of principles concerned with the nature and appreciation of beauty, especially in art. Visualization is an art of representing data in effective and easiest possible way.

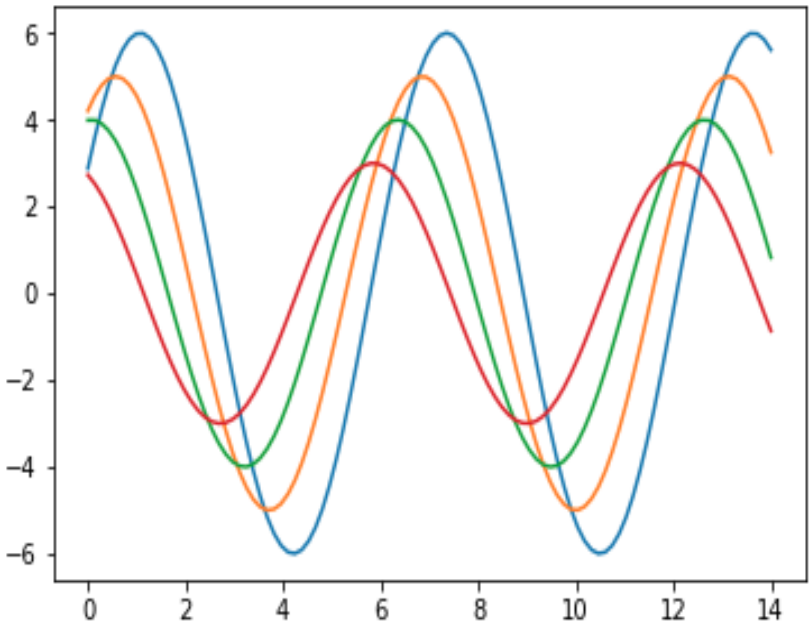
Matplotlib library highly supports customization, but knowing what settings to tweak to achieve an attractive and anticipated plot is what one should be aware of to make use of it. Unlike

Matplotlib, Seaborn comes packed with customized themes and a high-level interface for customizing and controlling the look of Matplotlib figures.

Example

```
import numpy as np
from matplotlib import pyplot as plt
def sinplot(flip = 1):
x = np.linspace(0, 14, 100)
for i in range(1, 5):
plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)
sinplot()
plt.show()
```

Output/Result-:



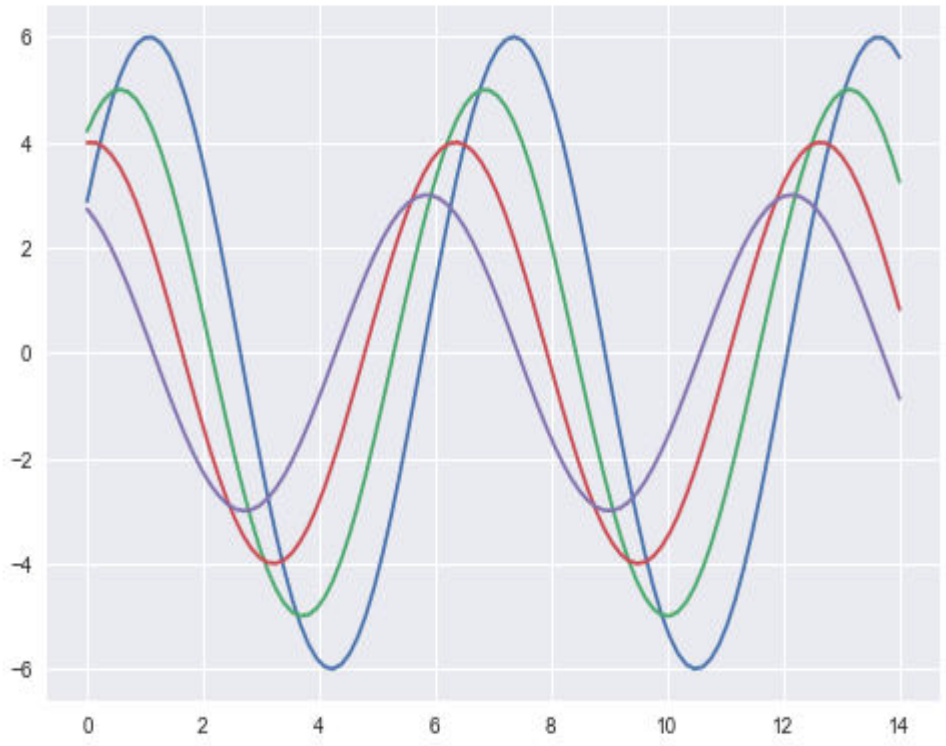
To change the same plot to Seaborn defaults, use the **set()** function −

Example

```
import numpy as np
from matplotlib import pyplot as plt
def sinplot(flip = 1):
x = np.linspace(0, 14, 100)
```

```
for i in range(1, 5):
plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)
import seaborn as sb
sb.set()
sinplot()
plt.show()
```

Output/Result-:



The above two figures show the difference in the default Matplotlib and Seaborn plots. The representation of data is same, but the representation style varies in both.

Basically, Seaborn splits the Matplotlib parameters into two groups−

- Plot styles
- Plot scale

Seaborn Figure Styles

The interface for manipulating the styles is **set_style()**. Using this function you can set the theme of the plot. As per the latest updated version, below are the five themes available.

- Darkgrid

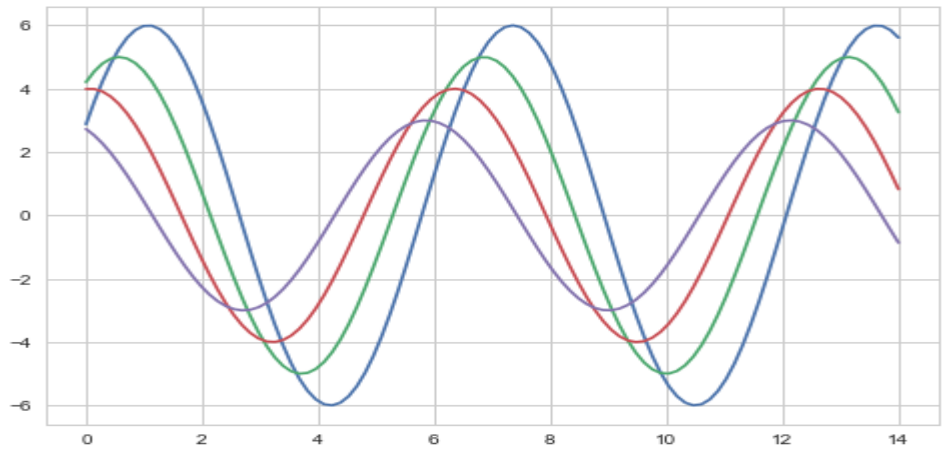
- Whitegrid
- Dark
- White
- Ticks

Let us try applying a theme from the above-mentioned list. The default theme of the plot will be **darkgrid** which we have seen in the previous example.

Example

```
import numpy as np
from matplotlib import pyplot as plt
def sinplot(flip=1):
x = np.linspace(0, 14, 100)
for i in range(1, 5):
plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)
import seaborn as sb
sb.set_style("whitegrid")
sinplot()
plt.show()
```

Output/Result-:



The difference between the above two plots is the background color

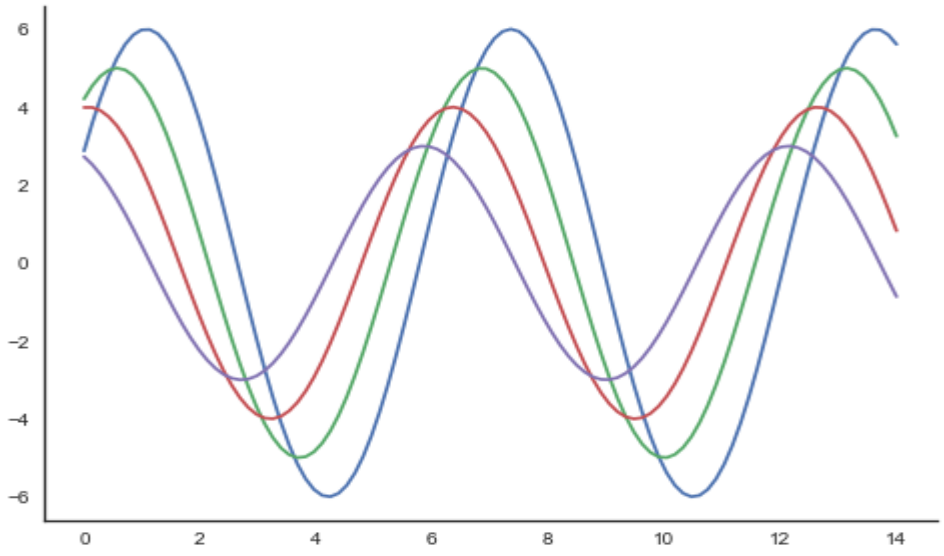
Removing Axes Spines

In the white and ticks themes, we can remove the top and right axis spines using the **despine()** function.

Example

```
import numpy as np
from matplotlib import pyplot as plt
def sinplot(flip=1):
x = np.linspace(0, 14, 100)
for i in range(1, 5):
plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)
import seaborn as sb
sb.set_style("white")
sinplot()
sb.despine()
plt.show()
```

Output/Result-:



In the regular plots, we use left and bottom axes only. Using the **despine()** function, we can avoid the unnecessary right and top axes spines, which is not supported in Matplotlib.

Overriding the Elements

If you want to customize the Seaborn styles, you can pass a dictionary of parameters to the **set_style()** function. Parameters available are viewed using **axes_style()** function.

Example

```
import seaborn as sb
```

print sb.axes_style

Output

{'axes.axisbelow' : False,

'axes.edgecolor' : 'white',

'axes.facecolor' : '#EAEAF2',

'axes.grid' : True,

'axes.labelcolor' : '.15',

'axes.linewidth' : 0.0,

'figure.facecolor' : 'white',

'font.family' : [u'sans-serif'],

'font.sans-serif' : [u'Arial', u'Liberation

Sans', u'Bitstream Vera Sans', u'sans-serif'],

'grid.color' : 'white',

'grid.linestyle' : u'-',

'image.cmap' : u'Greys',

'legend.frameon' : False,

'legend.numpoints' : 1,

'legend.scatterpoints': 1,

'lines.solid_capstyle': u'round',

'text.color' : '.15',

'xtick.color' : '.15',

'xtick.direction' : u'out',

'xtick.major.size' : 0.0,

'xtick.minor.size' : 0.0,

'ytick.color' : '.15',

'ytick.direction' : u'out',

'ytick.major.size' : 0.0,

'ytick.minor.size' : 0.0}

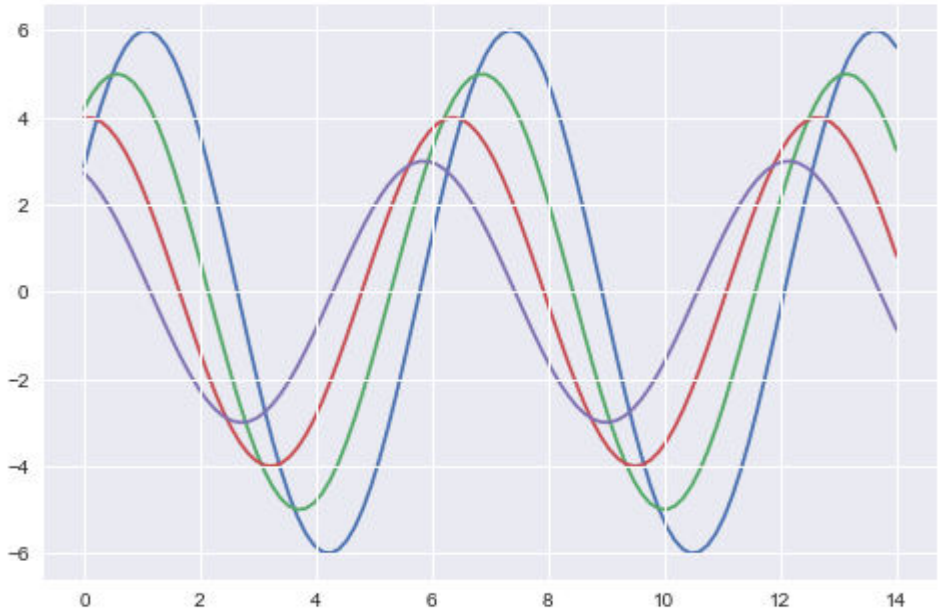
Altering the values of any of the parameter will alter the plot style.

Example

```
import numpy as np
from matplotlib import pyplot as plt
```

```
def sinplot(flip=1):
  x = np.linspace(0, 14, 100)
  for i in range(1, 5):
    plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)
import seaborn as sb
sb.set_style("darkgrid", {'axes.axisbelow': False})
sinplot()
sb.despine()
plt.show()
```

Output/Result-:



Scaling Plot Elements

We also have control on the plot elements and can control the scale of plot using the **set_context()** function. We have four preset templates for contexts, based on relative size, the contexts are named as follows

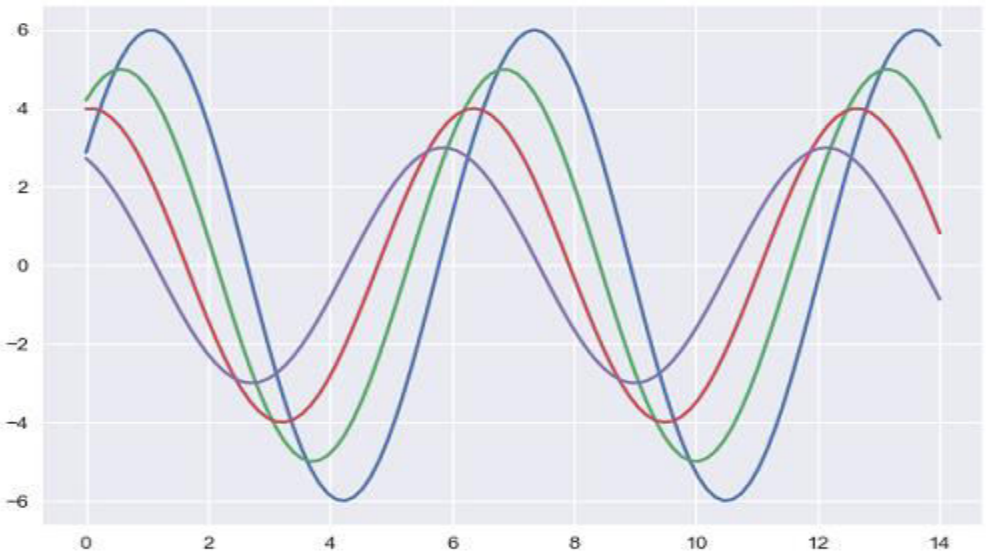
- Paper
- Notebook
- Talk
- Poster

By default, context is set to notebook; and was used in the plots above.

Example

```
import numpy as np
from matplotlib import pyplot as plt
def sinplot(flip = 1):
x = np.linspace(0, 14, 100)
for i in range(1, 5):
plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)
import seaborn as sb
sb.set_style("darkgrid", {'axes.axisbelow': False})
sinplot()
sb.despine()
plt.show()
```

Output/Result-:



The output size of the actual plot is bigger in size when compared to the above plots.

Seaborn - Color Palette

Color plays an important role than any other aspect in the visualizations. When used effectively, color adds more value to the plot. A palette means a flat surface on which a painter arranges and mixes paints.

Building Color Palette

Seaborn provides a function called **color_palette()**, which can be used to give colors to plots and adding more aesthetic value to it.

Usage

seaborn.color_palette(palette = None, n_colors = None, desat = None)

Parameter

The following table lists down the parameters for building color palette −

Sr.No.	Palatte & Description
1	**n_colors** Number of colors in the palette. If None, the default will depend on how palette is specified. By default the value of **n_colors** is 6 colors.
2	**desat** Proportion to desaturate each color.

Return

Return refers to the list of RGB tuples. Following are the readily available Seaborn palettes −

- Deep
- Muted
- Bright
- Pastel
- Dark
- Colorblind

Besides these, one can also generate new palette

It is hard to decide which palette should be used for a given data set without knowing the characteristics of data. Being aware of it, we will classify the different ways for using **color_palette()** types –

- qualitative
- sequential
- diverging

We have another function **seaborn.palplot()** which deals with color palettes. This function plots the color palette as horizontal array. We will know more regarding **seaborn.palplot()** in the coming examples.

Qualitative Color Palettes

Qualitative or categorical palettes are best suitable to plot the categorical data.

Example

```
from matplotlib import pyplot as plt
import seaborn as sb
current_palette = sb.color_palette()
sb.palplot(current_palette)
plt.show()
```

Output/Result-:



We haven't passed any parameters in **color_palette();** by default, we are seeing 6 colors. You can see the desired number of colors by passing a value to the **n_colors** parameter. Here, the **palplot()** is used to plot the array of colors horizontally.

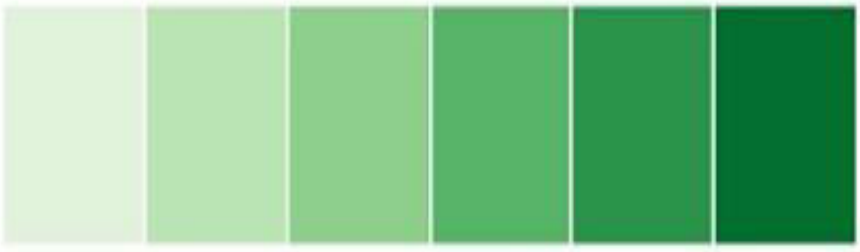
Sequential Color Palettes

Sequential plots are suitable to express the distribution of data ranging from relative lower values to higher values within a range.

Appending an additional character 's' to the color passed to the color parameter will plot the Sequential plot.

Example

```
from matplotlib import pyplot as plt
import seaborn as sb
current_palette = sb.color_palette()
sb.palplot(sb.color_palette("Greens"))
plt.show()
```



Diverging Color Palette

Diverging palettes use two different colors. Each color represents variation in the value ranging from a common point in either direction.

Assume plotting the data ranging from -1 to 1. The values from -1 to 0 takes one color and 0 to +1 takes another color.

By default, the values are centered from zero. You can control it with parameter center by passing a value.

Example

```
from matplotlib import pyplot as plt
import seaborn as sb
current_palette = sb.color_palette()
sb.palplot(sb.color_palette("BrBG", 7))
plt.show()
```

Output/Result-:



Setting the Default Color Palette

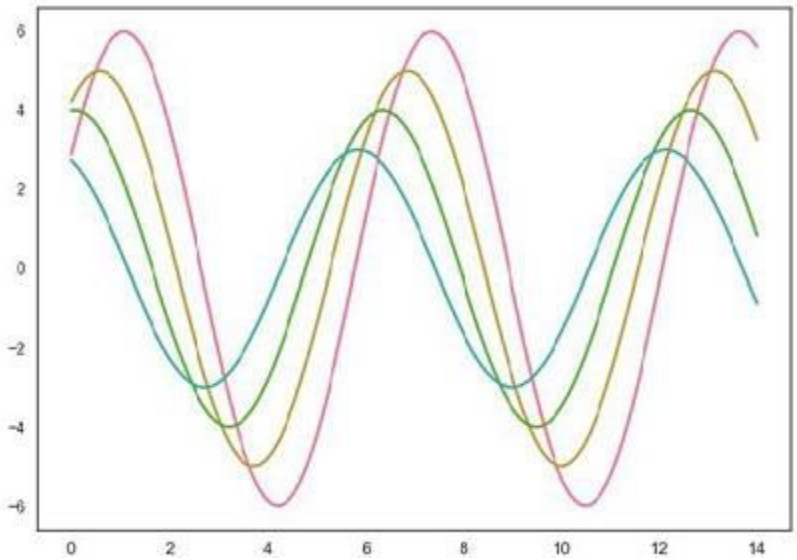
The functions **color_palette()** has a companion called **set_palette()** The relationship between them is similar to the pairs covered in the aesthetics chapter. The arguments are same for both **set_palette()** and **color_palette(),** but the default Matplotlib parameters are changed so that the palette is used for all plots.

Example

```
import numpy as np
from matplotlib import pyplot as plt
def sinplot(flip = 1):
x = np.linspace(0, 14, 100)
for i in range(1, 5):
plt.plot(x, np.sin(x + i * .5) * (7 - i) * flip)

import seaborn as sb
sb.set_style("white")
sb.set_palette("husl")
sinplot()
plt.show()
```

Output/Result-:



Plotting Univariate Distribution

Distribution of data is the foremost thing that we need to understand while analysing the data. Here, we will see how seaborn helps us in understanding the univariate distribution of the data.

Function **distplot()** provides the most convenient way to take a quick look at univariate distribution. This function will plot a histogram that fits the kernel density estimation of the data.

Usage

seaborn.distplot()

Parameters

The following table lists down the parameters and their description −

Sr.No.	Parameter & Description
1	**data** Series, 1d array or a list
2	**bins** Specification of hist bins
3	**hist** bool
4	**kde** bool

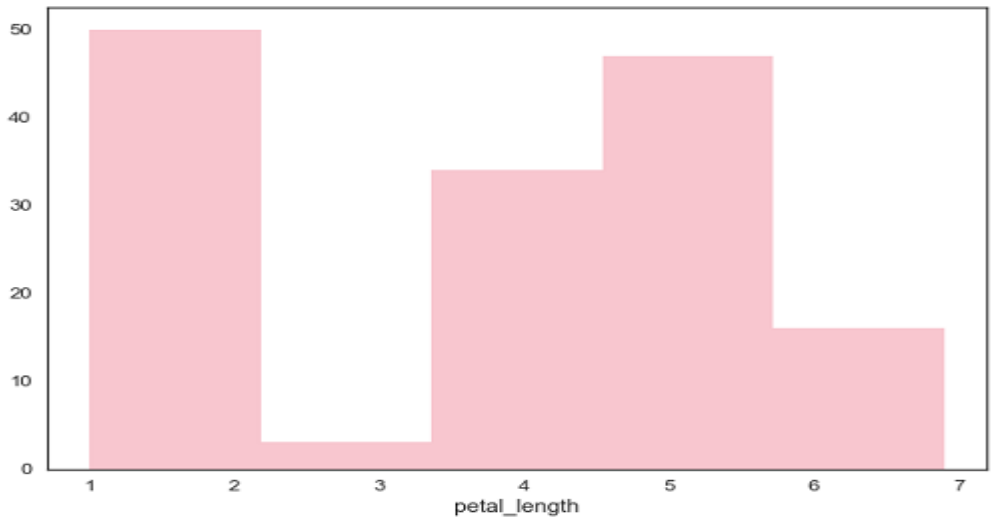
Seaborn – Histogram

Histograms represent the data distribution by forming bins along the range of the data and then drawing bars to show the number of observations that fall in each bin.

Seaborn comes with some datasets

```
import pandas as pd
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('iris')
sb.distplot(df['petal_length'],kde = False)
plt.show()
```

Output/Result-:



Here, **kde** flag is set to False. As a result, the representation of the kernel estimation plot will be removed and only histogram is plotted.

Seaborn - Kernel Density Estimates

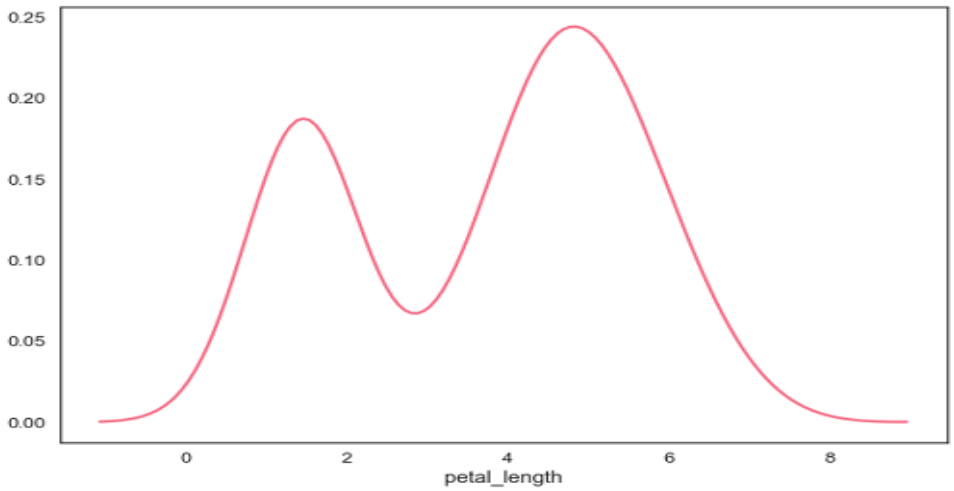
Kernel Density Estimation (KDE) is a way to estimate the probability density function of a continuous random variable. It is used for non-parametric analysis.

Setting the **hist** flag to False in **distplot** will yield the kernel density estimation plot.

```
import pandas as pd
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('iris')
sb.distplot(df['petal_length'],hist=False)
```

plt.show()

Output/Result-:



Fitting Parametric Distribution

distplot() is used to visualize the parametric distribution of a dataset.

Example

```
import pandas as pd
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('iris')
sb.distplot(df['petal_length'])

plt.show()
```

Output/Result-:

Plotting Bivariate Distribution

Bivariate Distribution is used to determine the relation between two variables. This mainly deals with relationship between two variables and how one variable is behaving with respect to the other.

The best way to analyze Bivariate Distribution in seaborn is by using the **jointplot()** function.

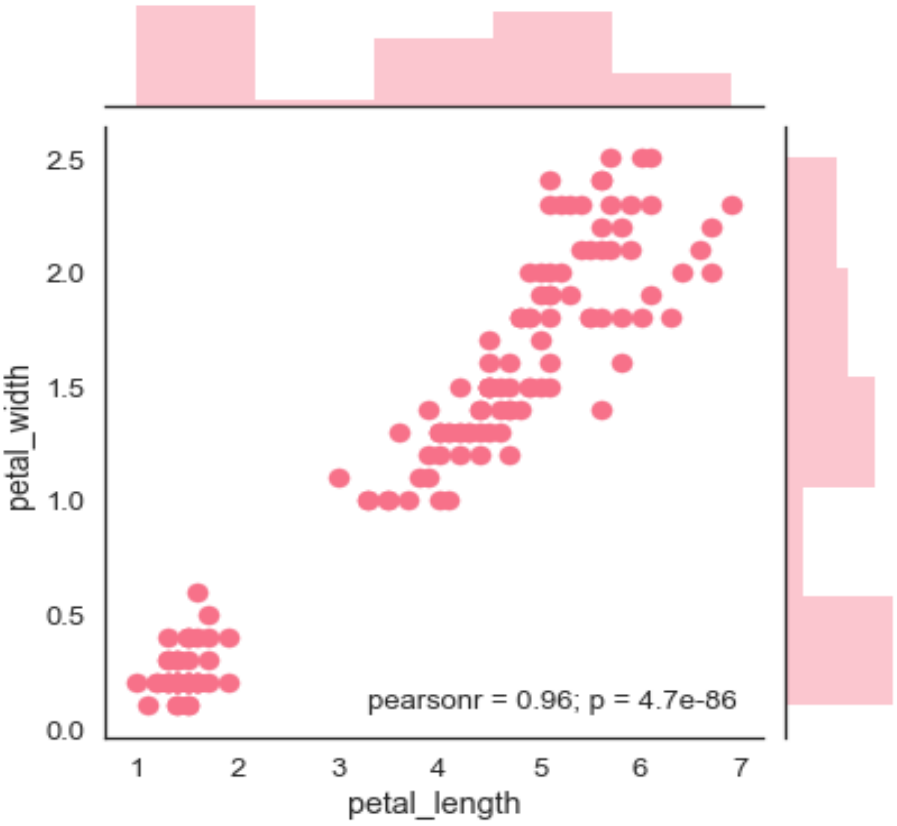
Jointplot creates a multi-panel figure that projects the bivariate relationship between two variables and also the univariate distribution of each variable on separate axes.

Scatter Plot

Scatter plot is the most convenient way to visualize the distribution where each observation is represented in two-dimensional plot via x and y axis.

```
import pandas as pd
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('iris')
sb.jointplot(x = 'petal_length',y = 'petal_width',data = df)
plt.show()
```

Output/Result-:



The above figure shows the relationship between the **petal_length** and **petal_width** in the Iris data. A trend in the plot says that positive correlation exists between the variables under study.

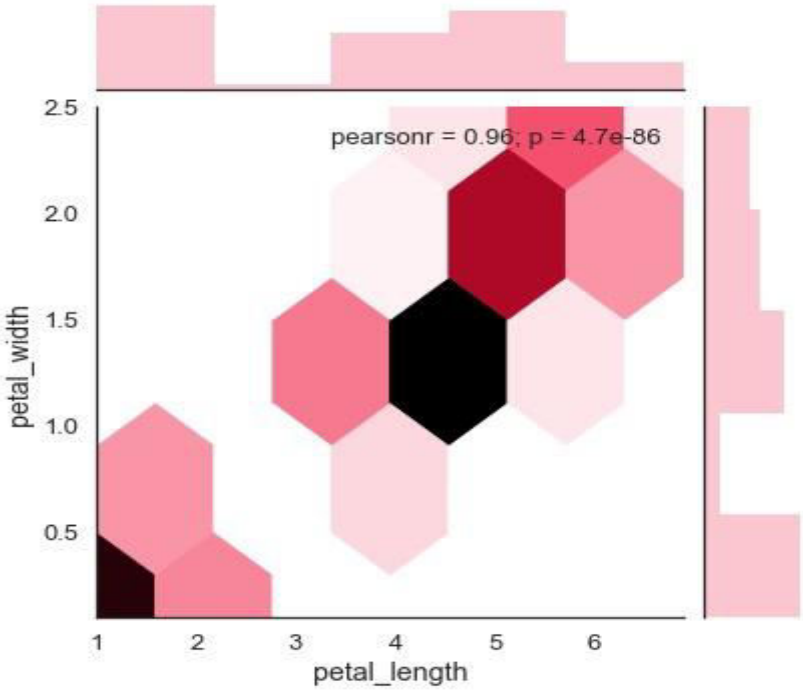
Hexbin Plot

Hexagonal binning is used in bivariate data analysis when the data is sparse in density i.e., when the data is very scattered and difficult to analyze through scatterplots.

An addition parameter called ‘kind’ and value ‘hex’ plots the hexbin plot.

```
import pandas as pd
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('iris')
sb.jointplot(x = 'petal_length',y = 'petal_width',data = df,kind = 'hex')
plt.show()
```

Output/Result-:



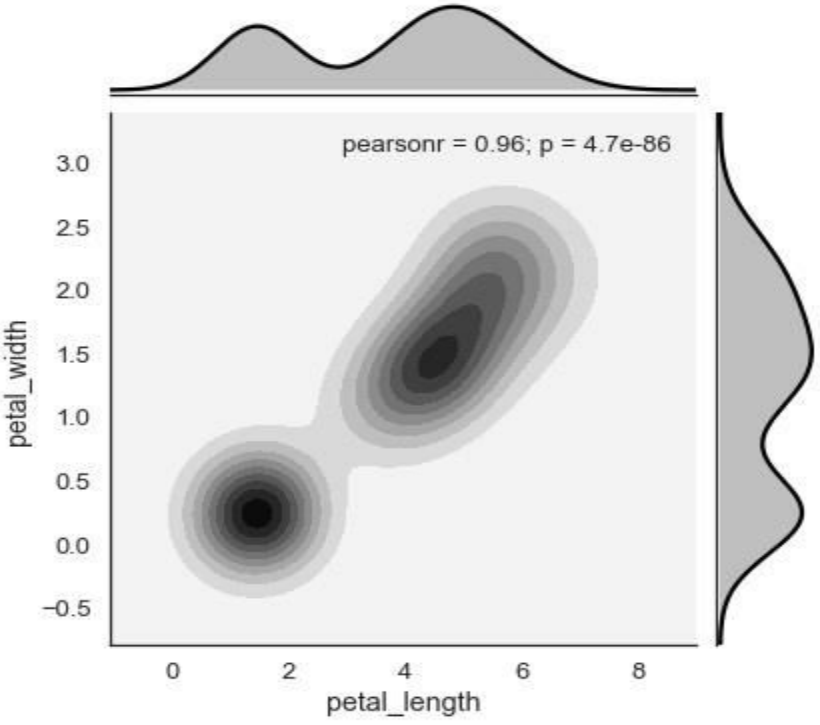
Kernel Density Estimation

Kernel density estimation is a non-parametric way to estimate the distribution of a variable. In seaborn, we can plot a kde using **jointplot().**

Pass value 'kde' to the parameter kind to plot kernel plot.

```
import pandas as pd
import seaborn as sb
from matplotlib import pyplot as plt
df = sb.load_dataset('iris')
sb.jointplot(x = 'petal_length',y = 'petal_width',data = df,kind = 'hex')
plt.show()
```

Output/Result-:



Seaborn - FacetGrid

Plotting Small Multiples of Data Subsets

A FacetGrid can be drawn with up to three dimensions − row, col, and hue. The first two have obvious correspondence with the resulting array of axes; think of the hue variable as a third dimension along a depth axis, where different levels are plotted with different colors.

FacetGrid object takes a dataframe as input and the names of the variables that will form the row, column, or hue dimensions of the grid.

The variables should be categorical and the data at each level of the variable will be used for a facet along that axis.

```
import pandas as pd

import seaborn as sb

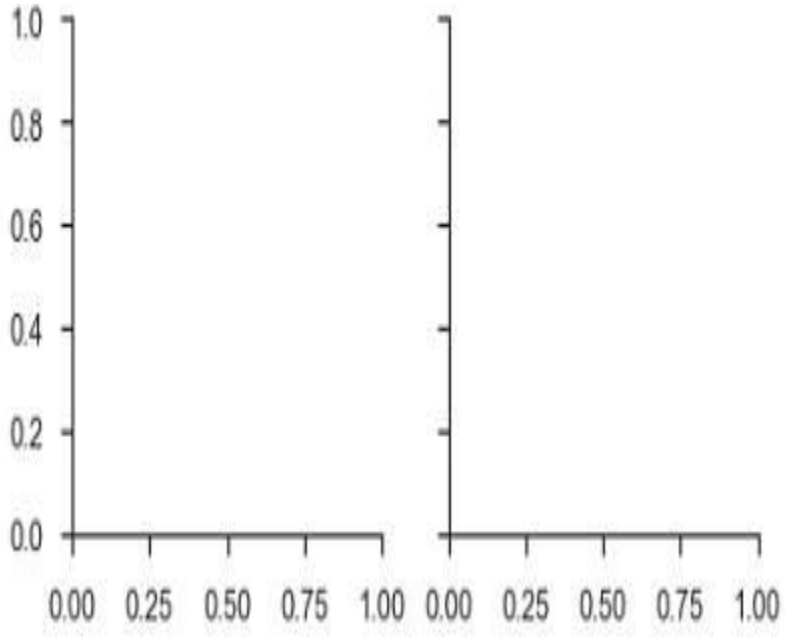
from matplotlib import pyplot as plt

df = sb.load_dataset('tips')

g = sb.FacetGrid(df, col = "time")

plt.show()
```

Output/Result-:-:



Seaborn - Pair Grid

PairGrid allows us to draw a grid of subplots using the same plot type to visualize data.

Unlike FacetGrid, it uses different pair of variable for each subplot. It forms a matrix of sub-plots. It is also sometimes called as “scatterplot matrix”.

The usage of pairgrid is similar to facetgrid. First initialise the grid and then pass the plotting function.

```
import pandas as pd

import seaborn as sb

from matplotlib import pyplot as plt

df = sb.load_dataset('iris')

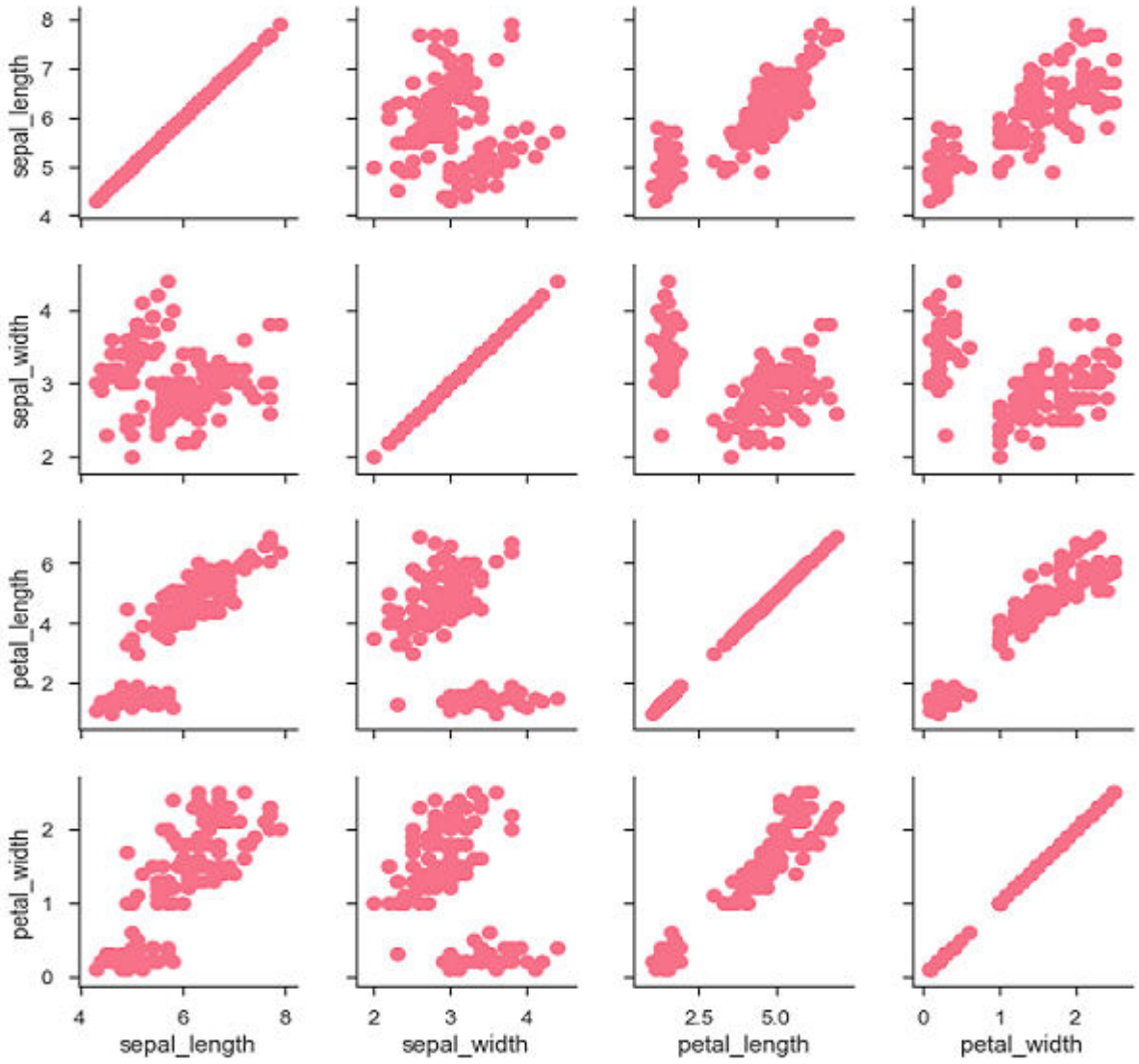
g = sb.PairGrid(df)

g.map(plt.scatter);

plt.show()
```

RSCOE-IT

Output/Result-:



Observations: Seaborn is a visualization library in Python.

Conclusion: Thus we studied and implemented functions of python Seaborn library.

ASSIGNMENT NO. A07

Study and implement functions of Tensorflow library.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
40	Explanation of Assignment and concepts	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
50	Coding of python Tensorflow library	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Study and implement functions of python Tensorflow library.

Objectives: To demonstrate python Tensorflow library functions

Aim/ Problem Statement: Write program to demonstrate Tensorflow library functions

Apparatus: Computer

Theory:

- Introduction
- Installation
- Basics of TensorFlow
- Tensor Handling and Manipulations

Introduction

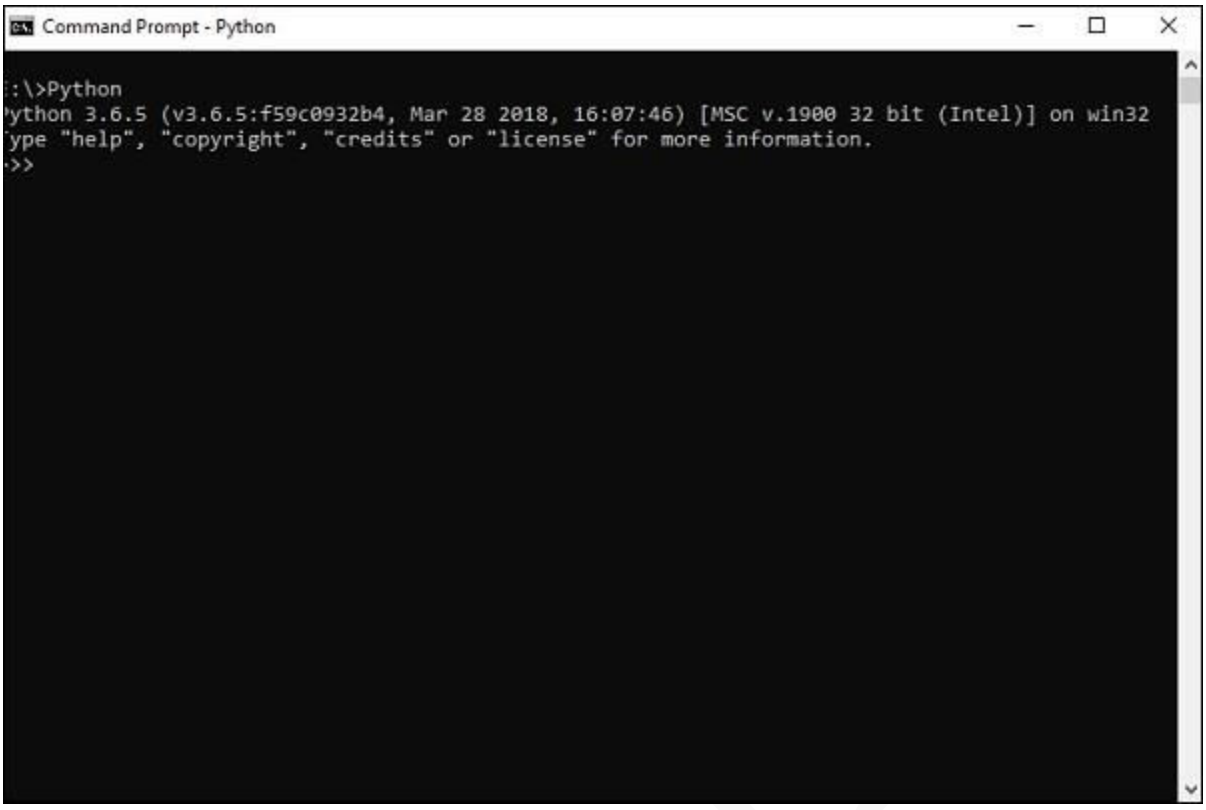
TensorFlow is a software library or framework, designed by the Google team to implement machine learning and deep learning concepts in the easiest manner. It combines the computational algebra of optimization techniques for easy calculation of many mathematical expressions.

Installation

To install TensorFlow, it is important to have "Python" installed in your system. Python version 3.4+ is considered the best to start with TensorFlow installation.

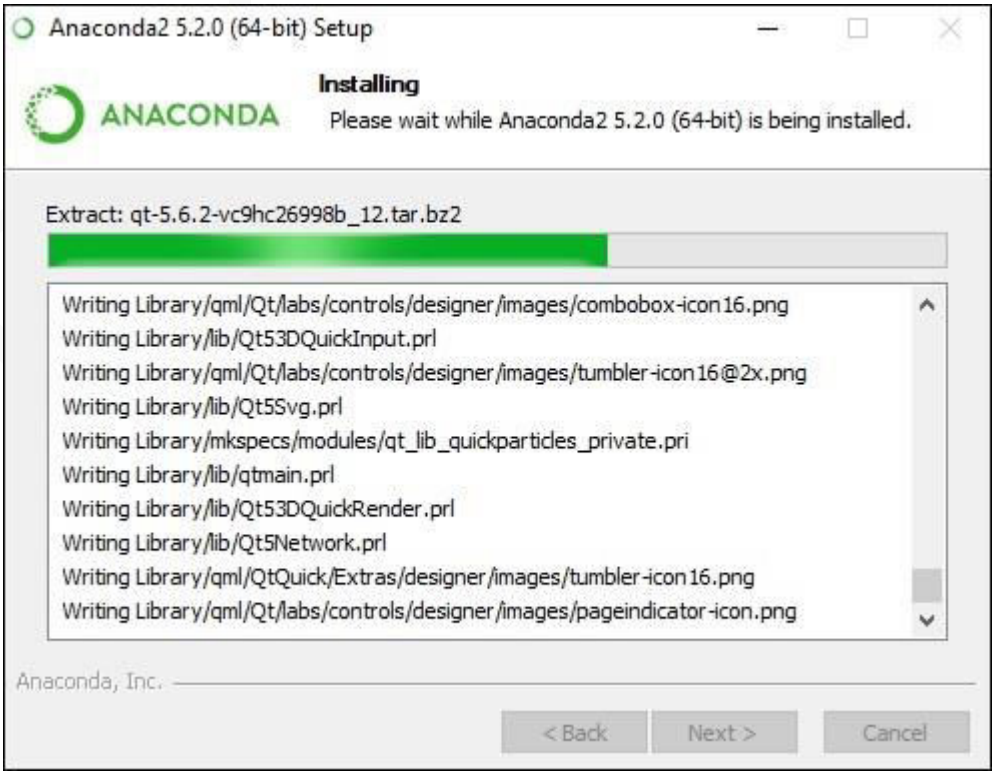
Consider the following steps to install TensorFlow in Windows operating system.

Step 1 − Verify the python version being installed.



Step 2 − A user can pick up any mechanism to install TensorFlow in the system. We recommend "pip" and "Anaconda". Pip is a command used for executing and installing modules in Python.

Before we install TensorFlow, we need to install Anaconda framework in our system.



After successful installation, check in command prompt through “conda” command. The execution of command is displayed below −

```
C:\Users\Radhika>conda
usage: conda [-h] [-V] command ...

conda is a tool for managing and deploying applications, environments and packages.

Options:

positional arguments:
  command
    clean        Remove unused packages and caches.
    config       Modify configuration values in .condarc. This is modeled
                 after the git config command. Writes to the user .condarc
                 file (C:\Users\Radhika\.condarc) by default.
    create       Create a new conda environment from a list of specified
                 packages.
    help         Displays a list of available conda commands and their help
                 strings.
    info         Display information about current conda install.
    install      Installs a list of packages into a specified conda
                 environment.
    list         List linked packages in a conda environment.
    package      Low-level conda package utility. (EXPERIMENTAL)
    remove       Remove a list of packages from a specified conda environment.
    uninstall    Alias for conda remove. See conda remove --help.
    search       Search for packages and display associated information. The
                 input is a MatchSpec, a query language for conda packages.
                 See examples below.
```

Step 3 − Execute the following command to initialize the installation of TensorFlow −

conda create --name tensorflow python = 3.5

```
Command Prompt - conda  create --name tensorflow python=3.5
    vc-14                     |        h0510ff6_3           3 KB
    wincertstore-0.2          |   py35hfebbdb8_0          13 KB
    wheel-0.31.1              |          py35_0          81 KB
    certifi-2018.4.16         |          py35_0         143 KB
    python-3.5.5              |       h0c2934d_2        18.2 MB
    ------------------------------------------------------------
                                         Total:        20.8 MB

The following NEW packages will be INSTALLED:

    certifi:        2018.4.16-py35_0
    pip:            10.0.1-py35_0
    python:         3.5.5-h0c2934d_2
    setuptools:     39.2.0-py35_0
    vc:             14-h0510ff6_3
    vs2015_runtime: 14.0.25123-3
    wheel:          0.31.1-py35_0
    wincertstore:   0.2-py35hfebbdb8_0

Proceed ([y]/n)? y


Downloading and Extracting Packages
pip-10.0.1           |  1.8 MB | ############################################################################ | 100%
setuptools-39.2.0    |  593 KB | ############################################################################ | 100%
vc-14                |    3 KB | ############################################################################ | 100%
wincertstore-0.2     |   13 KB | ############################################################################ | 100%
wheel-0.31.1         |   81 KB | ############################################################################ | 100%
certifi-2018.4.16    |  143 KB | ############################################################################ | 100%
python-3.5.5         | 18.2 MB | ##########################################################4                 |  70%
```

It downloads the necessary packages needed for TensorFlow setup.

Step 4 − After successful environmental setup, it is important to activate TensorFlow module.

activate tensorflow

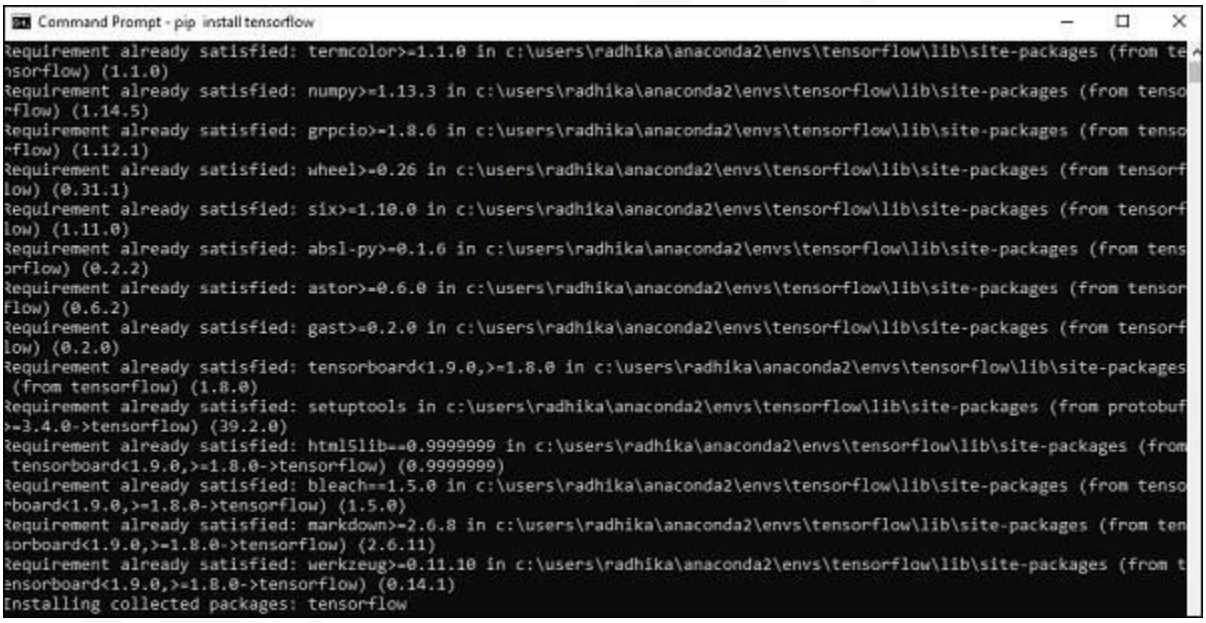


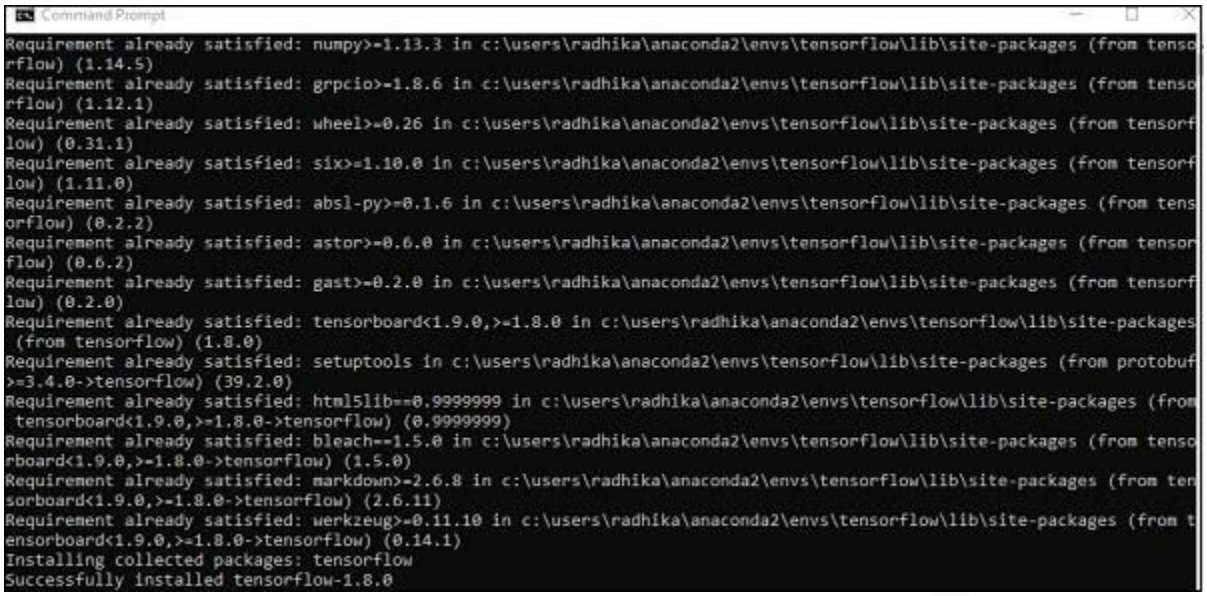
Step 5 − Use pip to install "Tensorflow" in the system. The command used for installation is mentioned as below −

pip install tensorflow

And,

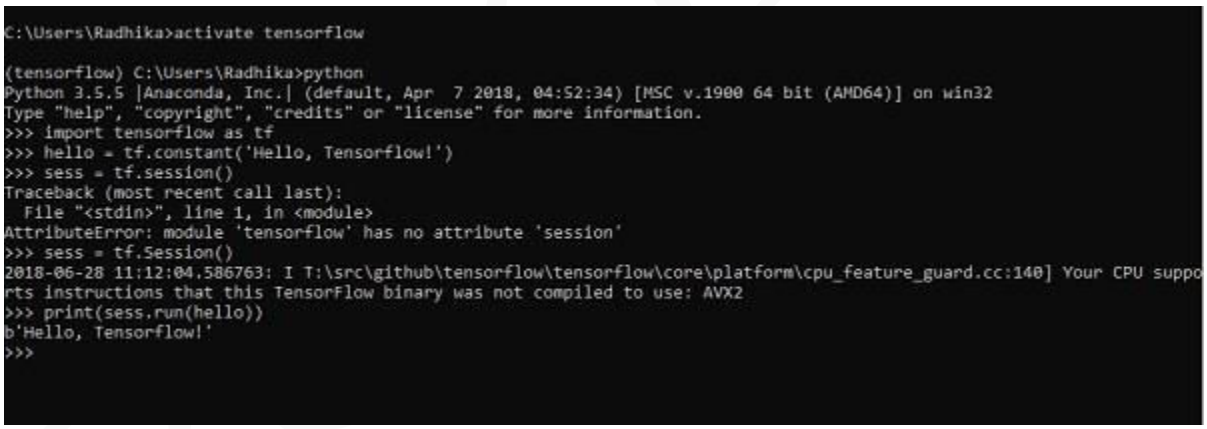
pip install tensorflow-gpu





After successful installation, it is important to know the sample program execution of TensorFlow.

Following example helps us understand the basic program creation “Hello World” in TensorFlow.



The code for first program implementation is mentioned below −

>> activate tensorflow

>> python (activating python shell)

>> import tensorflow as tf

>> hello = tf.constant(‘Hello, Tensorflow!’)

>> sess = tf.Session()

>> print(sess.run(hello))

Basics of TensorFlow

Tensor Data Structure

Tensors are used as the basic data structures in TensorFlow language. Tensors represent the connecting edges in any flow diagram called the Data Flow Graph. Tensors are defined as multidimensional array or list.

Tensors are identified by the following three parameters −

Rank

Unit of dimensionality described within tensor is called rank. It identifies the number of dimensions of the tensor. A rank of a tensor can be described as the order or n-dimensions of a tensor defined.

Shape

The number of rows and columns together define the shape of Tensor.

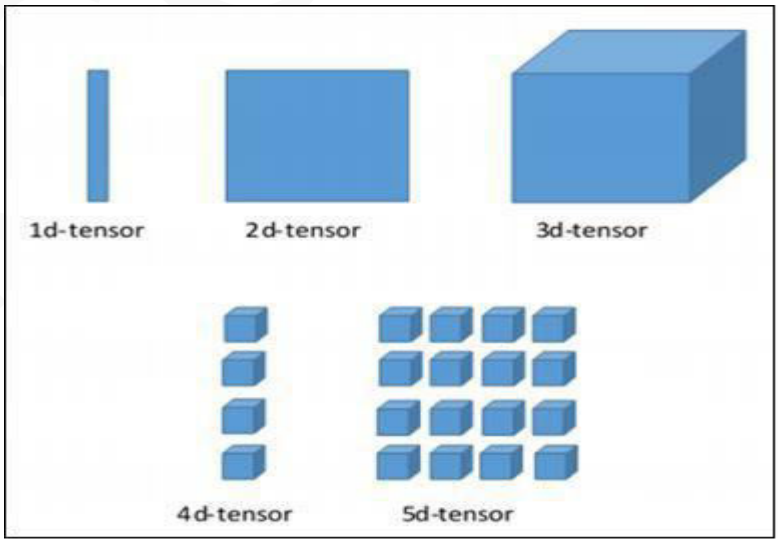
Type

Type describes the data type assigned to Tensor's elements.

A user needs to consider the following activities for building a Tensor −

Build an n-dimensional array

Convert the n-dimensional array.



Various Dimensions of TensorFlow

TensorFlow includes various dimensions. The dimensions are described in brief below −

One dimensional Tensor

One dimensional tensor is a normal array structure which includes one set of values of the same data type.

Declaration

```
>>> import numpy as np
>>> tensor_1d = np.array([1.3, 1, 4.0, 23.99])
>>> print tensor_1d
```

Output/Result :



The indexing of elements is same as Python lists. The first element starts with index of 0; to print the values through index, all you need to do is mention the index number.

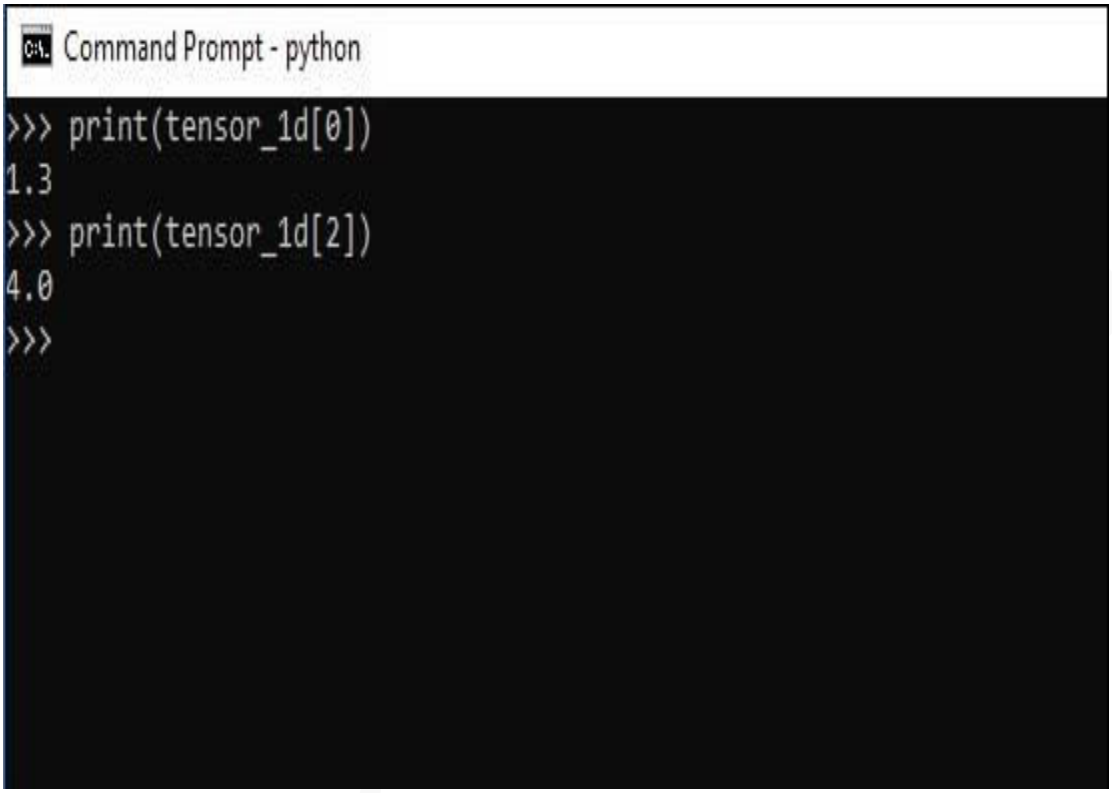
```
>>> print tensor_1d[0]
```

1.3

>>> print tensor_1d[2]

4.0

Output/Result :

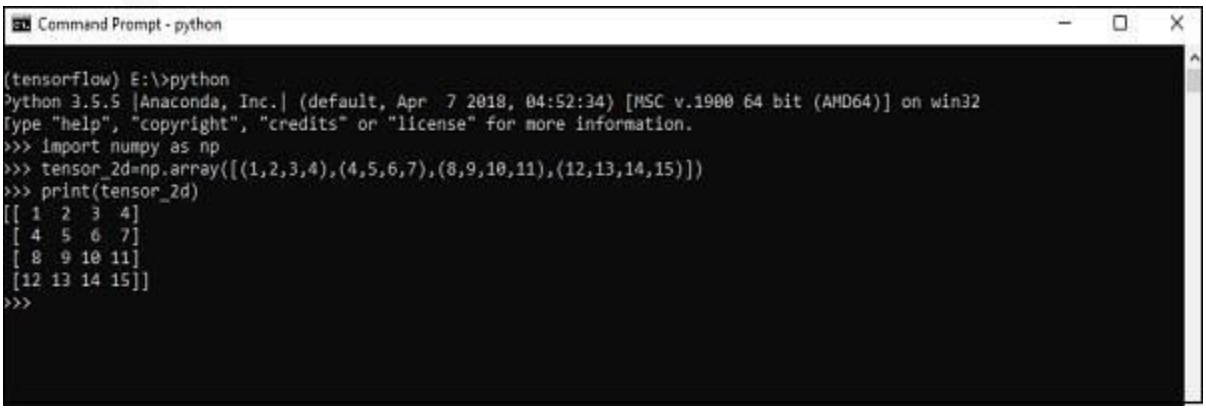


Two dimensional Tensors

Sequence of arrays are used for creating “two dimensional tensors”.

The creation of two-dimensional tensors is described below –

Output/Result :



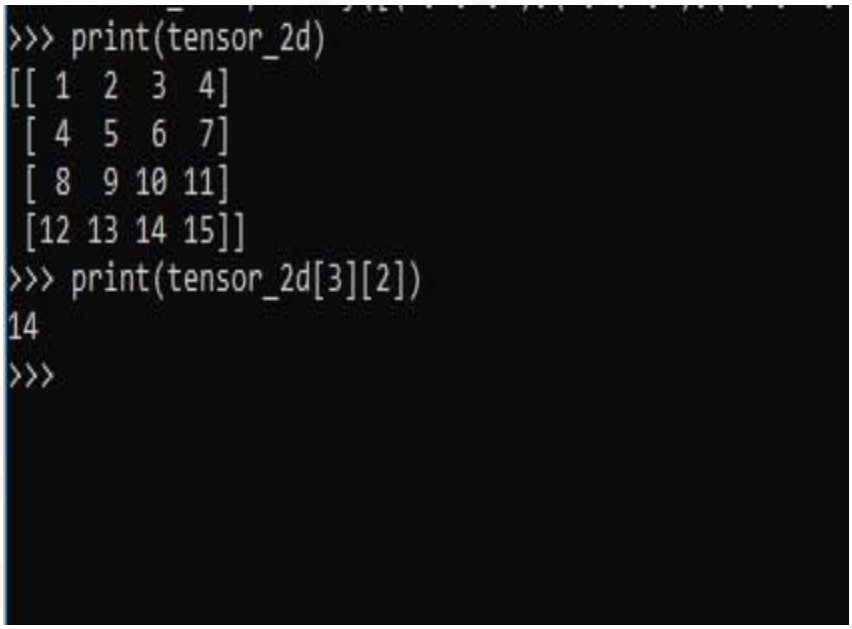
Following is the complete syntax for creating two dimensional arrays −

```
>>> import numpy as np
>>> tensor_2d = np.array([(1,2,3,4),(4,5,6,7),(8,9,10,11),(12,13,14,15)])
>>> print(tensor_2d)
[[ 1 2 3 4]
[ 4 5 6 7]
[ 8 9 10 11]
[12 13 14 15]]
>>>
```

The specific elements of two dimensional tensors can be tracked with the help of row number and column number specified as index numbers.

```
>>> tensor_2d[3][2]
14
```

Output/Result :



Tensor Handling and Manipulations

```
import tensorflow as tf
import numpy as np
```

```
matrix1 = np.array([(2,2,2),(2,2,2),(2,2,2)],dtype = 'int32')

matrix2 = np.array([(1,1,1),(1,1,1),(1,1,1)],dtype = 'int32')

print (matrix1)

print (matrix2)

matrix1 = tf.constant(matrix1)

matrix2 = tf.constant(matrix2)

matrix_product = tf.matmul(matrix1, matrix2)

matrix_sum = tf.add(matrix1,matrix2)

matrix_3 = np.array([(2,7,2),(1,4,2),(9,0,2)],dtype = 'float32')

print (matrix_3)

matrix_det = tf.matrix_determinant(matrix_3)

with tf.Session() as sess:

  result1 = sess.run(matrix_product)

  result2 = sess.run(matrix_sum)

  result3 = sess.run(matrix_det)

print (result1)

print (result2)

print (result3)
```

RSCOE-IT

Output/Result :



Observations: Tensors are used as the basic data structures in TensorFlow language

Conclusion: Thus we have studied and implemented functions of python TensorFlow library.

ASSIGNMENT NO. B01

Compute Estimators of the main statistical measures like Mean, Variance, Standard Deviation, Covariance, Correlation and Standard error with respect to any example. Display graphically the distribution of samples.

Practical Session Plan

Time (min)	Content	Learning Aid / Methodology	Faculty Approach	Typical Student Activity	Skill / Competency Developed
20	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers,Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers, Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Compute Estimators of the main statistical measures.

Objectives: To study and implement main statistical measures like Mean, Variance, Standard Deviation, Covariance, Correlation and Standard error with respect to any example.

Aim/ Problem Statement: Compute Estimators of the main statistical measures like Mean, Variance, Standard Deviation, Covariance, Correlation and Standard error with respect to any example. Display graphically the distribution of samples.

Apparatus: Computer

Theory:

Sample moments

Statistics like the sample mean, variance, skewness, and kurtosis are intimately related to the moments of a sample. If we have a sample of n random numbers, X_i, where i=1,…,n, the kth moment of the sample is

$$\mu_k = \frac{1}{n}\sum_{i=1}^{n} X_i^k$$

Sample mean

The mean (or average), mu, of the sample is the first moment:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} X_i$$

You will also see this notation sometimes

$$\bar{X} = \frac{1}{N}\sum_{i=1}^{N} X_i$$

In R, the mean() function returns the average of a vector.

The R script plot_batteries.R reads in the data in trend_corrected_chicago_batteries_2002_to_2013.csv The data include the number of daily batteries (assaults that don't involve a deadly weapon) in the City of Chicago between 2002 to 2013, along with daily temperature data from Midway airport, daily ozone data from the EPA,

and number of daylight hours from NASA. (note that I have done some pre-processing where I trend-corrected the number of batteries, and removed February 29th in leap years, such that the day-of-year variable is always between 1 to 365… also I define a "day" to be from 5am to 4:59 am, because, for instance, 1am Sunday morning is usually considered part of Saturday night).

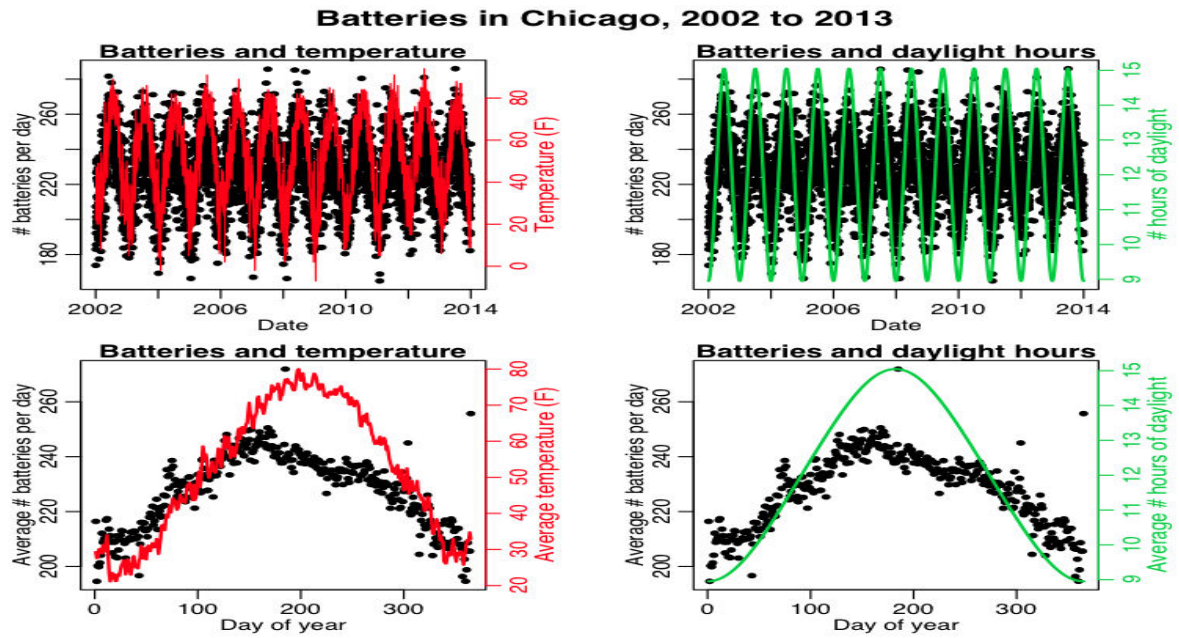
The plot_batteries.R script reads in this data and calculates the mean of the daily number of batteries, temperature, and daylight hours. It used the very useful R function aggregate() to get the average number of crimes by day of year, and the average temperature by day of year. The aggregate() function has an option by=list(<variable name>). The variable name is the variable within which you would like the data aggregated. Note that this variable needs to be discrete (ie; something like a day, month, year, gender, colour, etc). You can list more than one variable name to aggregate within, btw… here I used just day, but I could have aggregated within year and within month, using the option by=list(year,month).

The aggregate() function also has another option FUN="<function name>". If the function name is "mean" then the data are averaged within the variable levels. If the function is "sum" then the data are summed within the variable levels. If the function is "min"/"max" you will get the minimum/maximum within variable levels.

The script also gives a nice example of how to overlay plots of two entirely different quantities on the same plot, with the scale for one on the left hand axis, and the scale for the other on the right hand axis.

The R script produces the plot that shows the means of the various quantities by day of year (note that the script uses the useful R readline(prompt=) function to pause output between

plots):



What do you think those big spikes in batteries are at various times of the year? Also, what do you think that more diffuse spike is around day 75-ish of the year?

(I have a theory of crime that I call the Dr. Towers Drunk 'n Stupid Theory of Crime® that describes many of these fluctuations).

Variance

The variance, sigma^2, is a measure of the width of the distribution. It is defined by

$$\sigma^2 = \mu_2 - \mu_1^2$$

When the true mean of the distribution is known, the equation above is an unbiased estimator for the variance. However, when the mean must be estimated from the sample, it turns out that an estimate of the variance with less bias is

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \mu)^2$$

This is referred to as the "sample variance". In R, the var( ) function returns the sample variance.

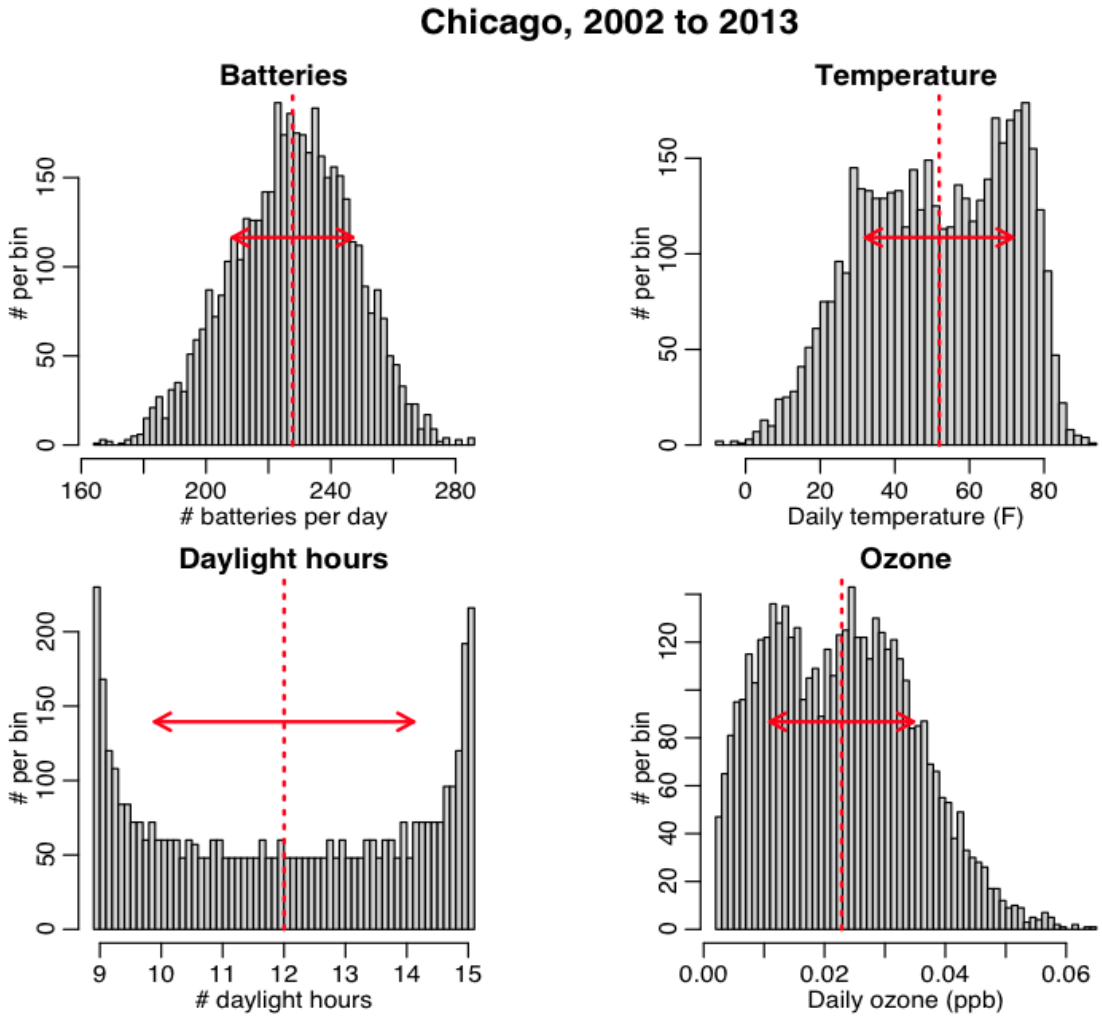
The plot_batteries.R script reads in the Chicago battery data and calculates the variance of the daily number of batteries, temperature, and daylight hours.

Standard deviation

The standard deviation of the sample is just the square root of the variance, sigma. In R, the sd( ) function returns the sample standard deviation. Note that if X has units, both the mean and the standard deviation width of X will have those units.

As an aside, you can also use the R sd function in the aggregate() function as an option via FUN="sd". This will give you the standard deviation within each of the variable levels.

The plot_batteries.R script reads in the Chicago battery data and calculates the standard deviation of the daily number of batteries, temperature, and daylight hours. It also produces this plot, that shows the mean, and standard deviation widths overlaid on the distributions:



Which of the variables are approximately Normally distributed?

Standard error on the mean

When you estimate the mean of a sample of observations, that is exactly what you are doing… estimating. That estimate may not exactly match the true mean of the probability distribution from which the X are drawn (in fact, it likely won't). Because the X are random variables, with

stochasticity, the estimate of the mean has uncertainty associated with it. The standard error on the mean is the estimate of that uncertainty:

$$SE = \frac{\sigma}{\sqrt{n}}$$

The Central Limit Theorem states that if your sample size is large enough (sample size greater than approximately 25 or so) then, no matter what probability distribution underlies the data, the estimate of the mean will be Normally distributed about the true mean of that probability distribution, with std-dev width of that Normal distribution being equal to SE.

On average 67% of the time the true value of the mean of the underlying probability distribution will lie within the range [mu-SE,mu+SE], where mu is your estimated value. Note that the larger n gets, the smaller SE gets, and the closer your estimated mean will be to the true mean.

Covariance

If we have two samples of the same size, X_i, and Y_i, where i=1,…,n, then the covariance is an estimate of how variation in X is related to variation in Y. The covariance is defined as

$$Cov(X,Y) = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \mu_X)(Y_i - \mu_Y)$$

where mu_X is the mean of the X sample, and mu_Y is the mean of the Y sample. Negative covariance means that smaller X tend to be associated with larger Y (and vice versa). Positive covariance means that larger/smaller X are associated with larger/smaller Y. Note that the covariance of X and Y is exactly the same as the covariance of Y and X. Also note that the covariance of X with itself is the variance of X. The covariance matrix for X and Y is thus

$$\mathrm{V} = \begin{pmatrix} \sigma_X^2 & Cov(X,Y) \\ Cov(X,Y) & \sigma_Y^2 \end{pmatrix}$$

The plot_batteries.R script reads in the Chicago battery data and calculates the covariance between the daily number of batteries, temperature, and daylight hours.

Correlation

The covariance has units (units of X times units of Y), and thus it can be difficult to assess how strongly related two quantities are. The correlation coefficient is a dimensionless quantity that helps to assess this.

The correlation coefficient between X and Y normalizes the covariance such that the resulting statistic lies between -1 and 1. The Pearson correlation coefficient

$$Cor(X,Y) = \frac{1}{n-1}\sum_{i=1}^{n}\frac{(X_i-\mu_X)(Y_i-\mu_Y)}{\sigma_X\sigma_Y}$$

is

The correlation matrix for X and Y is

$$C = \begin{pmatrix} 1 & Cor(X,Y) \\ Cor(X,Y) & 1 \end{pmatrix}$$

The plot_batteries.R script reads in the Chicago battery data and calculates the correlation between the daily number of batteries, and temperature, and daylight hours.

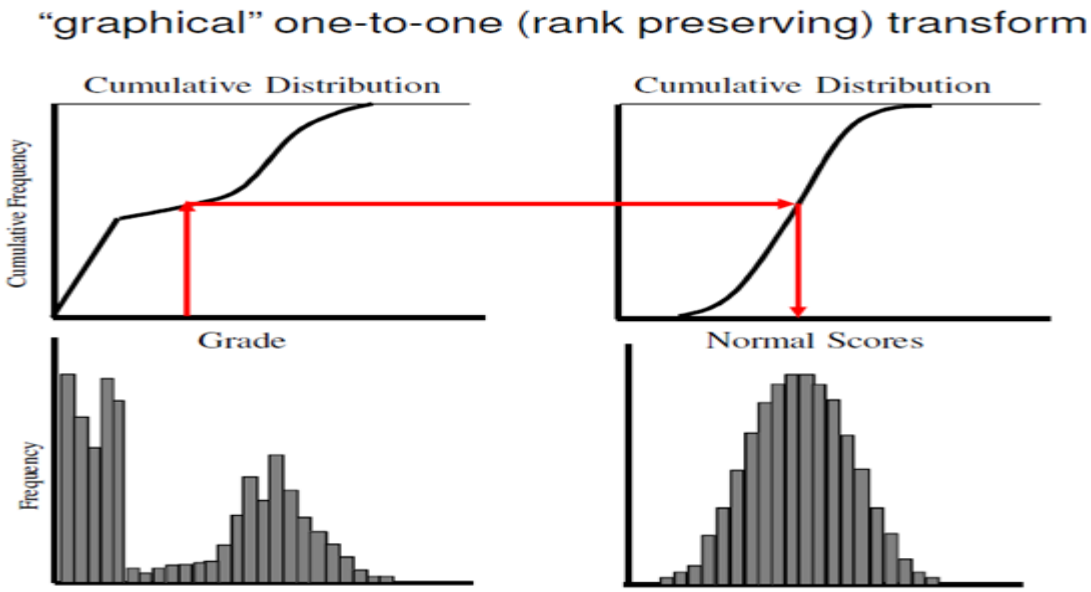
The Fishers Z-transformation can be used to assess whether or not an observed correlation between a sample of N values of X and Y is significantly different than zero.

The Vassar statistics correlation coefficient significance calculator page is a handy online tool for calculating the p-value that tests the null hypothesis that the observed correlation is consistent with zero. The non-directional "two sided test" assesses the probability that we would observe a value of r with absolute value greater than |r_observed| under the null hypothesis that the true correlation underlying the data is zero. The directional "one sided test" assesses the probability that we would observe a value of r at least as large as r_observed (if r_observed is +'ve) under the null hypothesis. If r_observed is negative, it assesses the probability that we would observe a value of r at least as small as r_observed under the null hypothesis that the true r is zero.

Unless you have a reason to expect in your alternate hypothesis that the true correlation is positive or negative, always use the two-sided test.

Are the observed correlations of the batteries to temperature or daylight hours significantly different than zero? (note that N=4379).

Note that the Fishers Z-transformation significance test assumes that X and Y are Normally distributed. If they aren't, then applying the test can lead to incorrect p-value assessment. The Spearman rho correlation coefficient helps to fix this, by first mapping the X and Y data onto a Normal distribution using a rank-Normal transformation, then calculating the correlations between the transformed variables. In R, you can obtain the Spearman rho correlation coefficient by using the method="spearman" option in the cor() method. Here's how a rank-Normal transformation works (image source):



Psedocode/Algorithm:

import numpy as np

Original array
array =np.arange(10)
print(array)

r1 =np.mean(array)
print("\nMean: ", r1)

r2 =np.std(array)
print("\nstd: ", r2)

r3 =np.var(array)
print("\nvariance: ", r3)

importnumpy as np

Original array
array =np.arange(10)
print(array)

```
r1 =np.average(array)
print("\nMean: ", r1)
  r2 =np.sqrt(np.mean((array -np.mean(array)) **2))
print("\nstd: ", r2)
  r3 =np.mean((array -np.mean(array)) **2)
print("\nvariance: ", r3)
importnumpy as np
  # Original array
x =np.arange(5)
print(x)

r11 =np.mean(x)
r12 =np.average(x)
print("\nMean: ", r11, r12)
  r21 =np.std(x)
r22 =np.sqrt(np.mean((x -np.mean(x)) **2))
print("\nstd: ", r21, r22)
  r31 =np.var(x)
r32 =np.mean((x -np.mean(x)) **2)
print("\nvariance: ", r31, r32)
```

For this example, we will be working on the well-known Iris dataset. We're only working with the setosa species to be specific, hence this will be just a sample of the dataset about some lovely purple flowers!

Let's have a look at the dataset, on which we will be performing the analysis:

Out[1]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

We are about to pick two columns, for our analysis - `sepal_length` and `sepal_width`.

In a new Python file (you can name it `covariance_correlation.py`), let's begin by creating two lists with values for the `sepal_length` and `sepal_width` properties of the flower:

```
with open('iris_setosa.csv','r') as f:
    g=f.readlines()
    # Each line is split based on commas, and the list of floats are formed
    sep_length = [float(x.split(',')[0]) for x in g[1:]]
    sep_width  = [float(x.split(',')[1]) for x in g[1:]]
```

In data science, it always helps to visualize the data you're working on. Here's a Seaborn regression plot (Scatter Plot + linear regression fit) of these setosa properties on different axes:

Visually the data points seem to be having a high correlation close to the regression line. Let's see if our observations match up to their covariance and correlation values.

Calculating Covariance in Python

The following formula computes the covariance:

$$cov_{x,y} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{N - 1}$$

In the above formula,

- x_i, y_i - are individual elements of the x and y series
- $\bar{x}$, $\bar{y}$ - are the mathematical means of the x and y series
- N - is the number of elements in the series

The denominator is `N` for a whole dataset and `N - 1` in the case of a sample. As our dataset is a small sample of the entire Iris dataset, we use `N - 1`.

With the math formula mentioned above as our reference, let's create this function in pure Python:

```
def covariance(x, y):
    # Finding the mean of the series x and y
    mean_x = sum(x)/float(len(x))
    mean_y = sum(y)/float(len(y))
    # Subtracting mean from the individual elements
    sub_x = [i - mean_x for i in x]
    sub_y = [i - mean_y for i in y]
    numerator = sum([sub_x[i]*sub_y[i] for i in range(len(sub_x))])
    denominator = len(x)-1
    cov = numerator/denominator
    return cov

with open('iris_setosa.csv', 'r') as f:
    ...
    cov_func = covariance(sep_length, sep_width)
    print("Covariance from the custom function:", cov_func)
```

We first find the mean values of our datasets. We then use a list comprehension to iterate over every element in our two series' of data and subtract their values from the mean. A for loop could have been used a well if that's your preference.

We then use those intermediate values of the two series' and multiply them with each other in another list comprehension. We sum the result of that list and store it as the `numerator`. The `denominator` is a lot easier to calculate, be sure to decraese it by 1 when you're finding the covariance for sample data!

We then return the value when the `numerator` is divided by its `denominator`, which results in the covariance.

Running our script would give us this output:

```
Covariance from the custom function: 0.09921632653061219
```

The positive value denotes that both the variables move in the same direction.

Calculating Correlation in Python

The most widely used formula to compute correlation coefficient is Pearson's 'r':

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

In the above formula,

- x_i, y_i - are individual elements of the x and y series
- The numerator corresponds to the covariance
- The denominators correspond to the individual standard deviations of x and y

```
def correlation(x, y):
    # Finding the mean of the series x and y
    mean_x = sum(x)/float(len(x))
    mean_y = sum(y)/float(len(y))
    # Subtracting mean from the individual elements
    sub_x = [i-mean_x for i in x]
    sub_y = [i-mean_y for i in y]
    # covariance for x and y
    numerator = sum([sub_x[i]*sub_y[i] for i in range(len(sub_x))])
    # Standard Deviation of x and y
    std_deviation_x = sum([sub_x[i]**2.0 for i in range(len(sub_x))])
    std_deviation_y = sum([sub_y[i]**2.0 for i in range(len(sub_y))])
    # squaring by 0.5 to find the square root
    denominator = (std_deviation_x*std_deviation_y)**0.5 # short but equivalent to (std_deviation
    cor = numerator/denominator
    return cor

with open('iris_setosa.csv', 'r') as f:
    ...
    cor_func = correlation(sep_length, sep_width)
    print("Correlation from the custom function:", cor_func)
```

As this value needs the covariance of the two variables, our function pretty much works out that value once again. Once the covariance is computed, we then calculate the standard deviation for

each variable. From there, the correlation is simply dividing the covariance with the multiplication of the squares of the standard deviation.

Running this code we get the following output, confirming that these properties have a positive (sign of the value, either +, -, or none if 0) and strong (the value is close to 1) relationship:

```
Correlation from the custom function: 0.7425466856651597
```

Expected Output/Results:

Output:

```
[0 1 2 3 4 5 6 7 8 9]

Mean:  4.5

std:  2.8722813232690143

variance:  8.25
```

Output:

```
[0 1 2 3 4 5 6 7 8 9]

Mean:  4.5

std:  2.8722813232690143

variance:  8.25
```

Output:

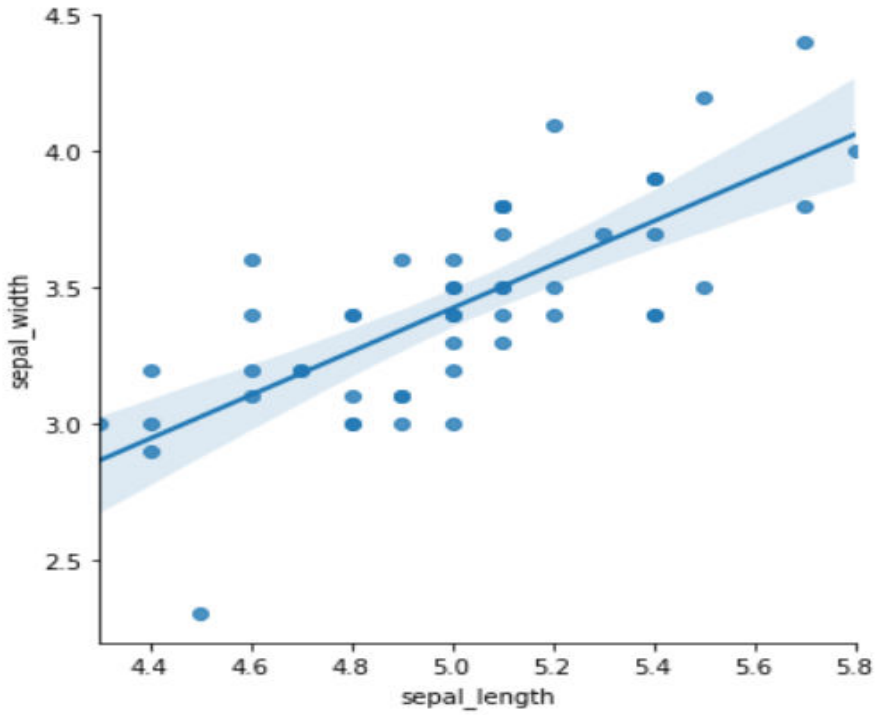
```
[0 1 2 3 4]

Mean:  2.0 2.0

std:  1.4142135623730951 1.4142135623730951

variance:  2.0 2.0
```

Out[2]: <seaborn.axisgrid.FacetGrid at 0x7fcf682320a0>



Conclusion:- Thus we have studied and implemented main statistical measures like Mean, Variance, Standard Deviation, Covariance, Correlation and Standard error for given data.

ASSIGNMENT NO. B02

Plot the Normal Distribution for class test result of a particular subject. Identify the Skewness and Kurtosis.

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers,Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers, Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

PROOF

Title: Compute Estimators of the main statistical measures.

Objectives: To study and implement Normal Distribution for class test result of a particular subject. Identify the Skewness and Kurtosis.

Aim/ Problem Statement: Plot the Normal Distribution for class test result of a particular subject. Identify the Skewness and Kurtosis.

Apparatus: Computer

Theory:

A probability distribution is a statistical function that describes the likelihood of obtaining the possible values that a random variable can take. By this, we mean the range of values that a parameter can take when we randomly pick up values from it.

A probability distribution can be discrete or continuous.

Suppose in a city we have heights of adults between the age group of 20-30 years ranging from 4.5 ft. to 7 ft.

If we were asked to pick up 1 adult randomly and asked what his/her (assuming gender does not affect height) height would be? There's no way to know what the height will be. But if we have the distribution of heights of adults in the city, we can bet on the most probable outcome.

What is Normal Distribution?

A **Normal Distribution** is also known as a **Gaussian distribution** or famously **Bell Curve**. People use both words interchangeably, but it means the same thing. It is a continuous probability distribution.

The probability density function (pdf) for Normal Distribution:

$$f(x;\mu,\sigma) = \frac{1}{\sqrt{2\pi * \sigma^2}} * e^{-\frac{1}{2} * (\frac{x-\mu}{\sigma})^2}$$

Probability Density Function Of Normal Distribution

where, μ = Mean , σ = Standard deviation , x = input value.

Terminology:

- **Mean** – The mean is the usual average. The sum of total points divided by the total number of points.
- **Standard Deviation** – Standard deviation tells us how "spread out" the data is. It is a measure of how far each observed value is from the mean.

1. Example Implementation of Normal Distribution

Let's have a look at the code below. We'll use numpy and matplotlib for this demonstration:

```
# Importing required libraries

import numpy as np
import matplotlib.pyplot as plt

# Creating a series of data of in range of 1-50.
x =np.linspace(1,50,200)

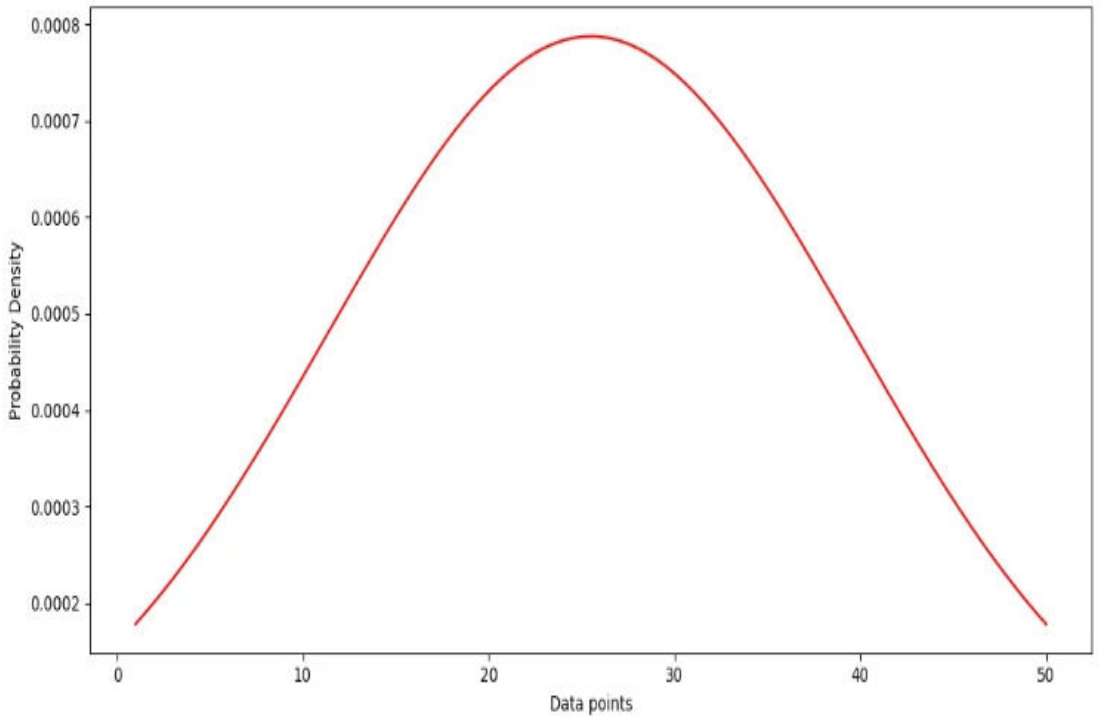
#Creating a Function.
def normal_dist(x , mean , sd):
    prob_density=(np.pi*sd) *np.exp(-0.5*((x-mean)/sd)**2)
    return prob_density

#Calculate mean and Standard deviation.
mean =np.mean(x)
sd=np.std(x)

#Apply function to the data.
pdf =normal_dist(x,mean,sd)

#Plotting the Results
plt.plot(x,pdf , color ='red')
plt.xlabel('Data points')
```

```
plt.ylabel('Probability Density')
```



Normal Curve

2. Properties of Normal Distribution

The normal distribution density function simply accepts a data point along with a mean value and a standard deviation and throws a value which we call **probability density**.

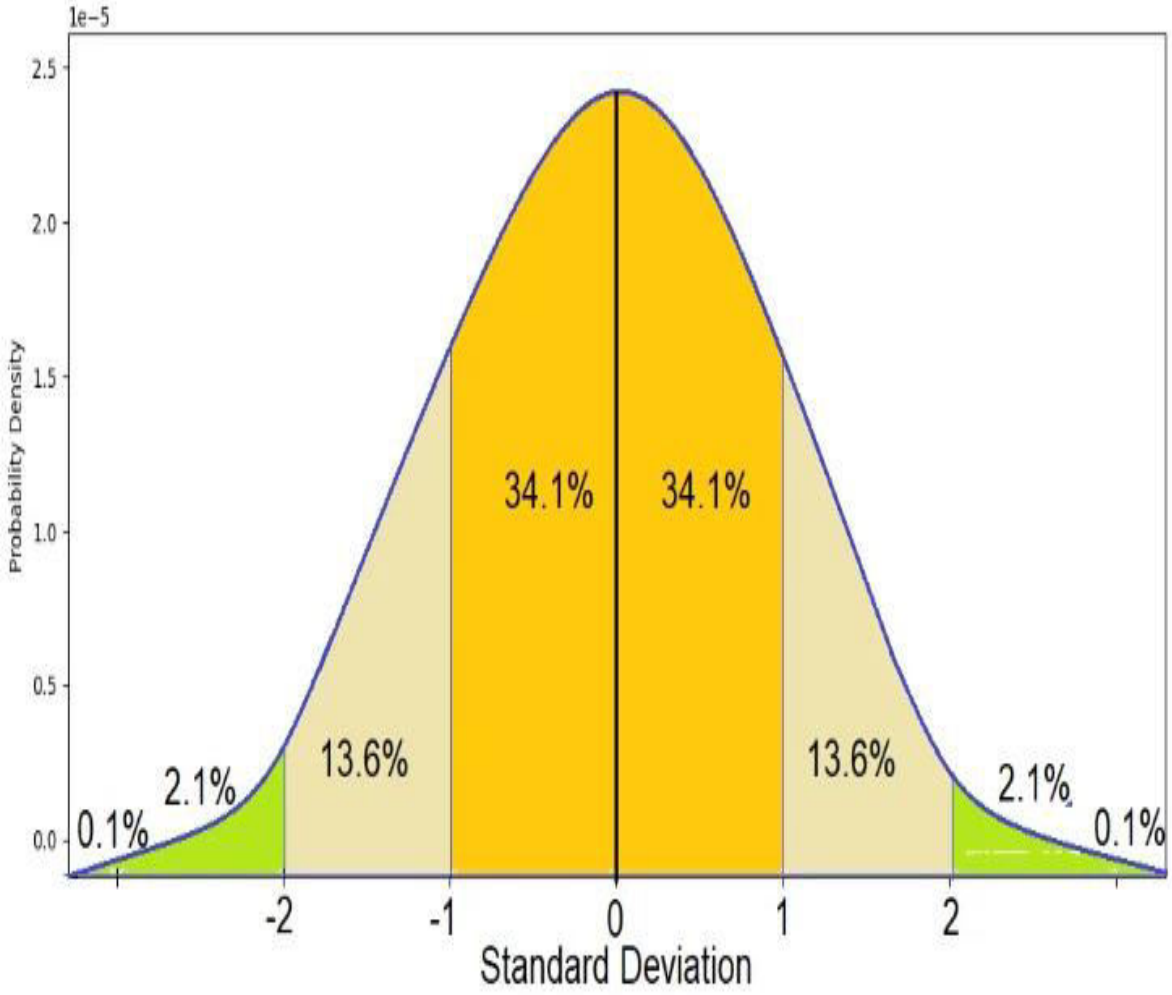
We can alter the shape of the bell curve by changing the mean and standard deviation.

Changing the mean will shift the curve towards that mean value, this means we can change the position of the curve by altering the mean value while the shape of the curve remains intact.

The shape of the curve can be controlled by the value of Standard deviation. A smaller standard deviation will result in a closely bounded curve while a high value will result in a more spread out curve.

Some excellent properties of a normal distribution:

- The mean, mode, and median are all equal.
- The total area under the curve is equal to 1.
- The curve is symmetric around the mean.



Percentage Distribution of Data Around Mean

Empirical rule tells us that:

- 68% of the data falls within one standard deviation of the mean.
- 95% of the data falls within two standard deviations of the mean.
- 99.7% of the data falls within three standard deviations of the mean.

It is by far one of the most important distributions in all of the Statistics. The normal distribution is magical because most of the naturally occurring phenomenon follows a normal distribution. For example, blood pressure, IQ scores, heights follow the normal distribution.

Calculating Probabilities with Normal Distribution

To find the probability of a value occurring within a range in a normal distribution, we just need to find the area under the curve in that range. i.e. we need to integrate the density function.
Since the normal distribution is a continuous distribution, the area under the curve represents the probabilities.
Before getting into details first let's just know what a Standard Normal Distribution is.

A standard normal distribution is just similar to a normal distribution with mean = 0 and standard deviation = 1.
$Z = (x-\mu)/ \sigma$
The z value above is also known as a **z-score**. A z-score gives you an idea of how far from the mean a data point is.
If we intend to calculate the probabilities manually we will need to lookup our z-value in a z-table to see the cumulative percentage value. Python provides us with modules to do this work for us. Let's get into it.
1. Creating the Normal Curve
We'll use scipy.norm class function to calculate probabilities from the normal distribution.
Suppose we have data of the heights of adults in a town and the data follows a normal distribution, we have a sufficient sample size with mean equals 5.3 and the standard deviation is

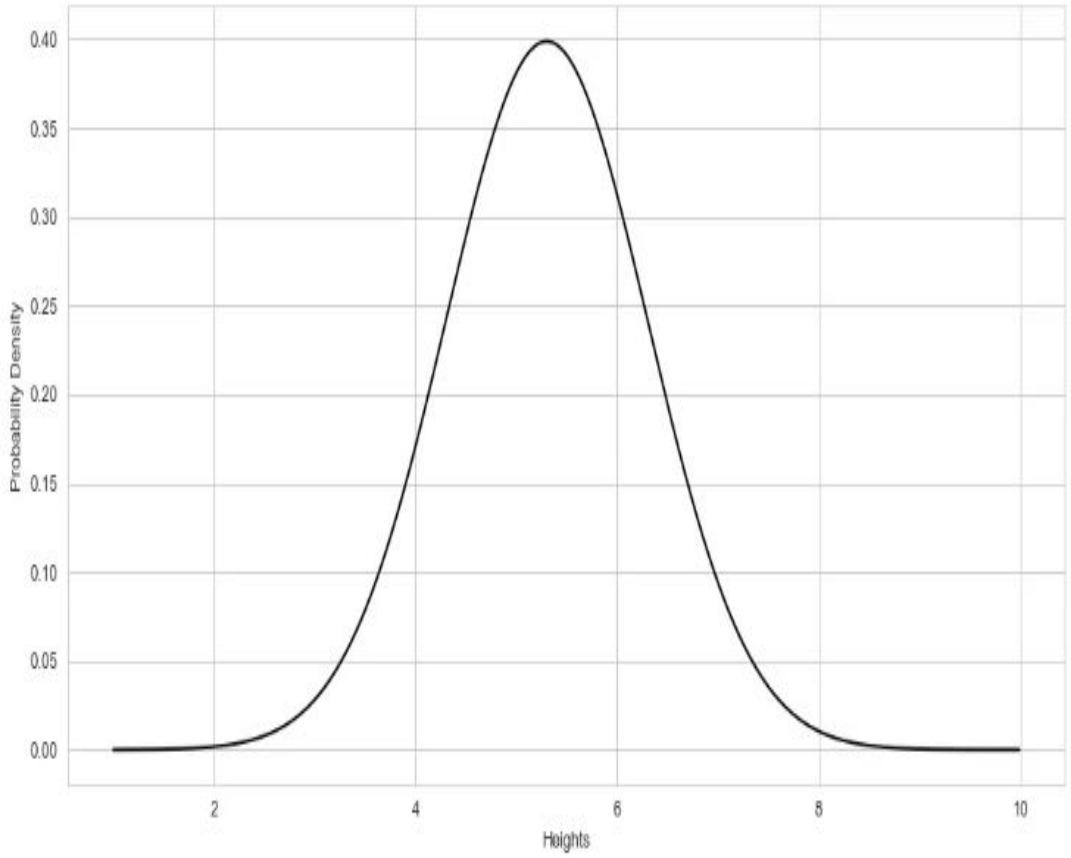
This information is sufficient to make a normal curve.

```
# import required libraries
fromscipy.statsimportnorm
importnumpy as np
importmatplotlib.pyplot as plt
importseaborn as sb

# Creating the distribution
data =np.arange(1,10,0.01)
pdf =norm.pdf(data , loc=5.3, scale =1)

#Visualizing the distribution
```

```
sb.set_style('whitegrid')
sb.lineplot(data, pdf , color='black')
plt.xlabel('Heights')
plt.ylabel('Probability Density')
```

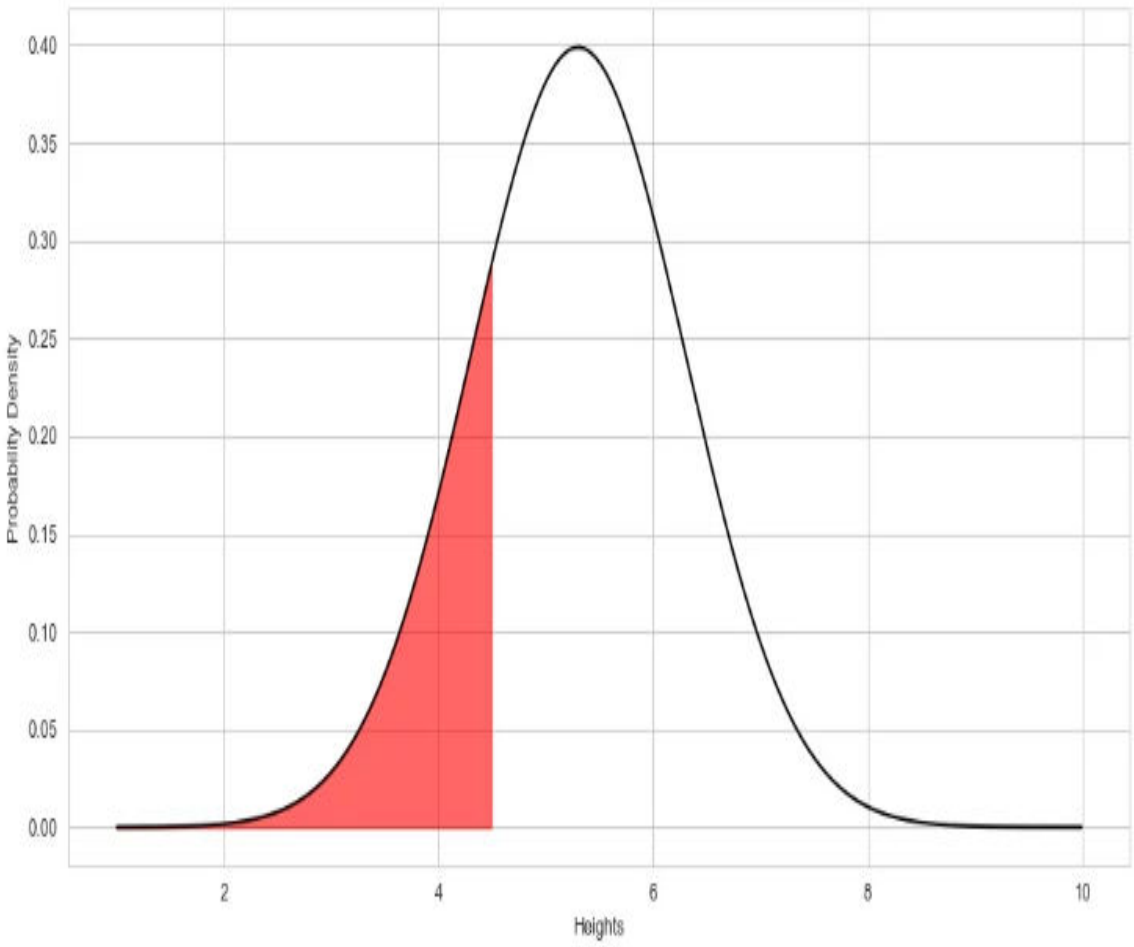


Heights Distribution

The norm.pdf( ) class method requires loc and scale along with the data as an input argument and gives the probability density value. loc is nothing but the mean and the scale is the standard deviation of data. the code is similar to what we created in the prior section but much shorter.

2. Calculating Probability of Specific Data Occurance

Now, if we were asked to pick one person randomly from this distribution, then what is the probability that the height of the person will be smaller than 4.5 ft. ?



Area Under The Curve As ProbabilityThe area under the curve as shown in the figure above will be the probability that the height of the person will be smaller than 4.5 ft if chosen randomly from the distribution. Let's see how we can calculate this in python.

The area under the curve is nothing but just the Integration of the density function with limits equals -∞ to 4.5.

```
norm(loc=5.3, scale =1).cdf(4.5)
```

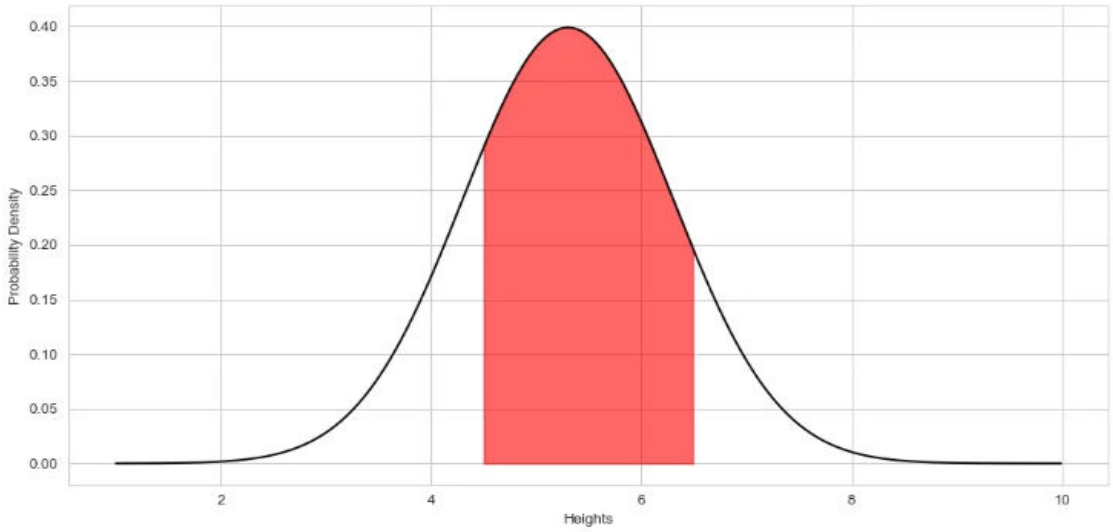
0.211855 or 21.185 %

The single line of code above finds the probability that there is a 21.18% chance that if a person is chosen randomly from the normal distribution with a mean of 5.3 and a standard deviation of 1, then the height of the person will be below 4.5 ft.

We initialize the object of class norm with mean and standard deviation, then using .cdf( ) method passing a value up to which we need to find the cumulative probability value. The cumulative distribution function (CDF) calculates the cumulative probability for a given x-value.

Cumulative probability value from -∞ to ∞ will be equal to 1.

Now, again we were asked to pick one person randomly from this distribution, then what is the probability that the height of the person will be between 6.5 and 4.5 ft. ?



Area Under The Curve Between 4.5 And 6.5 Ft

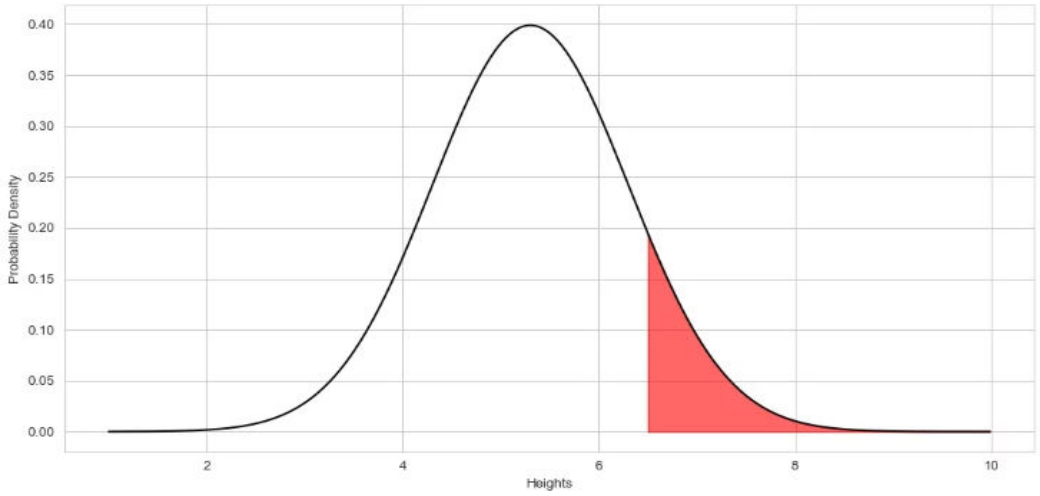
```
cdf_upper_limit=norm(loc=5.3, scale =1).cdf(6.5)
cdf_lower_limit=norm(loc=5.3, scale =1).cdf(4.5)

prob =cdf_upper_limit-cdf_lower_limit
print(prob)
```

0.673074 or 67.30 %

The above code first calculated the cumulative probability value from -∞ to 6.5 and then the cumulative probability value from -∞ to 4.5. if we subtract cdf of 4.5 from cdf of 6.5 the result we get is the area under the curve between the limits 6.5 and 4.5.

Now, what if we were asked about the probability that the height of a person chosen randomly will be above 6.5ft?



Area Under The Curve Between 6.5ft and Infinity

It's simple, as we know the total area under the curve equals 1, and if we calculate the cumulative probability value from -∞ to 6.5 and subtract it from 1, the result will be the probability that the height of a person chosen randomly will be above 6.5ft.

```
cdf_value=norm(loc=5.3, scale =1).cdf(6.5)
prob =1-cdf_value
print(prob)
```

0.115069 or 11.50 %.

That's a lot to sink in, but I encourage all to keep practicing this essential concept along with the implementation using python.

Conclusion: Thus we have studied and implemented normal distribution for given data.

ASSIGNMENT NO.B03

Demonstrates how to do data correlation, rescaling and dimensionality reduction is done through PCA

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers,Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers, Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: To study and implement PCA Algorithm using Python

Objectives: To study and implement data correlation, rescaling and dimensionality reduction using PCA.

Aim/ Problem Statement: Demonstrates how to do data correlation, rescaling and dimensionality reduction is done through PCA**.**

Apparatus: Computer

Theory:

In the modern age of technology, increasing amounts of data are produced and collected. In machine learning, however, too much data can be a bad thing. At a certain point, more features or dimensions can decrease a model's accuracy since there is more data that needs to be generalized — this is known as the **curse of dimensionality**.

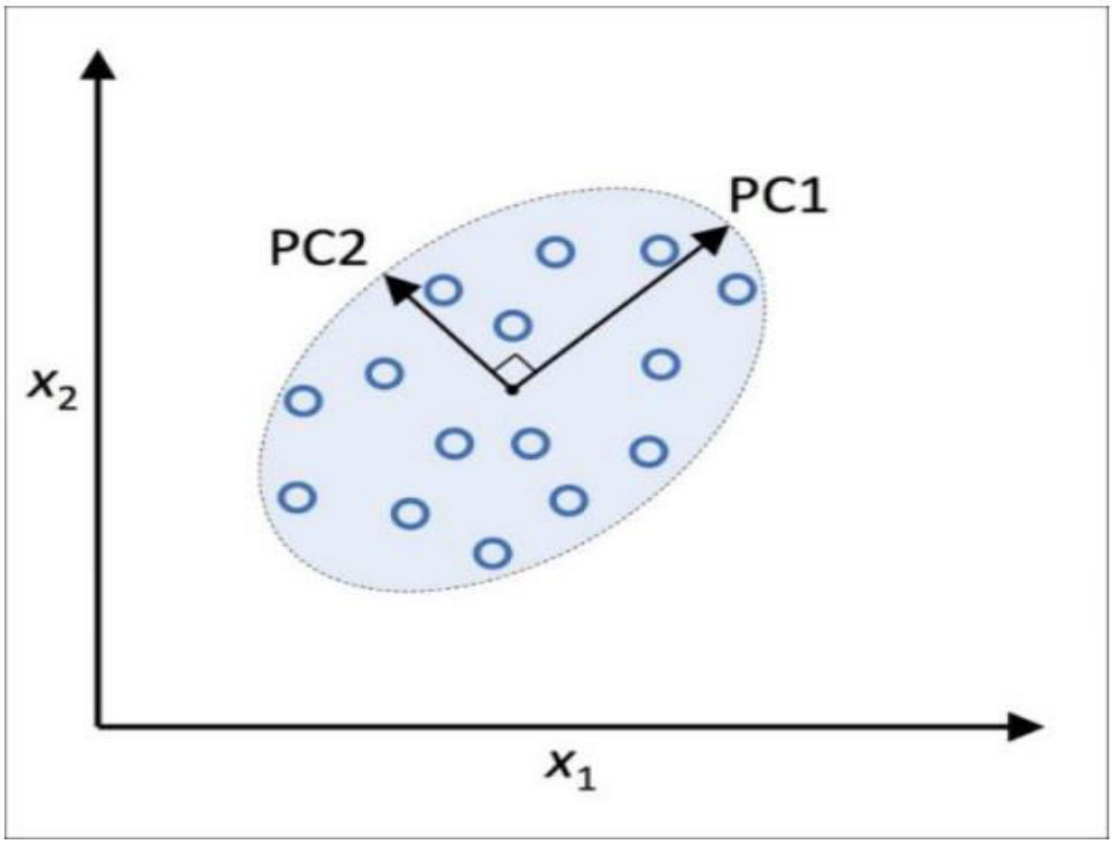
Dimensionality reduction is way to reduce the complexity of a model and avoid overfitting. There are two main categories of dimensionality reduction: feature selection and feature extraction. Via feature selection, we select a subset of the original features, whereas in feature extraction, we derive information from the feature set to construct a new feature subspace.

Introduction to Principal Component Analysis

Principal Component Analysis (**PCA**) is an unsupervised linear transformation technique that is widely used across different fields, most prominently for feature extraction and dimensionality reduction. Other popular applications of PCA include exploratory data analyses and de-noising of signals in stock market trading, and the analysis of genome data and gene expression levels in the field of bioinformatics.

PCA helps us to identify patterns in data based on the correlation between features. In a nutshell, PCA aims to find the directions of maximum variance in high-dimensional data and projects it onto a new subspace with equal or fewer dimensions than the original one.

The orthogonal axes (**principal components**) of the new subspace can be interpreted as the directions of maximum variance given the constraint that the new feature axes are orthogonal to each other, as illustrated in the following figure:



In the preceding figure, *x1* and *x2* are the original feature axes, and **PC1** and **PC2** are the principal components.

If we use PCA for dimensionality reduction, we construct a *d* x *k*–dimensional transformation matrix ***W*** that allows us to map a sample vector ***x*** onto a new *k*–dimensional feature subspace that has fewer dimensions than the original *d*–dimensional feature space:

$$\boldsymbol{x} = [x_1, x_2, \ldots, x_d], \quad \boldsymbol{x} \in \mathbb{R}^d$$
$$\downarrow \boldsymbol{x}\boldsymbol{W}, \quad \boldsymbol{W} \in \mathbb{R}^{d \times k}$$
$$\boldsymbol{z} = [z_1, z_2, \ldots, z_k], \quad \boldsymbol{z} \in \mathbb{R}^k$$

As a result of transforming the original *d*-dimensional data onto this new *k*-dimensional subspace (typically $k \ll d$), the first principal component will have the largest possible variance, and all consequent principal components will have the largest variance given the constraint that these components are uncorrelated (orthogonal) to the other principal components — even if the input features are correlated, the resulting principal components will be mutually orthogonal (uncorrelated).

Note that the PCA directions are highly sensitive to data scaling, and we need to standardize the features *prior* to PCA if the features were measured on different scales and we want to assign equal importance to all features.

Before looking at the PCA algorithm for dimensionality reduction in more detail, let's summarize the approach in a few simple steps:

1. Standardize the d-dimensional dataset.
2. Construct the covariance matrix.
3. Decompose the covariance matrix into its eigenvectors and eigenvalues.
4. Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors.
5. Select k eigenvectors which correspond to the k largest eigenvalues, where k is the dimensionality of the new feature subspace ($k \leq d$).
6. Construct a projection matrix $\boldsymbol{W}$ from the "top" k eigenvectors.
7. Transform the d-dimensional input dataset $\boldsymbol{X}$ using the projection matrix $\boldsymbol{W}$ to obtain the new k-dimensional feature subspace.

Let's perform a PCA step by step, using Python as a learning exercise. Then, we will see how to perform a PCA more conveniently using `scikit-learn`.

Extracting the Principal Components Step By Step

We will be using the *Wine* dataset from The UCI Machine Learning Repository in our example. This dataset consists of 178 wine samples with 13 features describing their different chemical properties. You can find out more here.

In this section we will tackle the first four steps of a PCA; later we will go over the last three. You can follow along with the code in this tutorial by using a Next Tech sandbox, which has all the necessary libraries pre-installed, or if you'd prefer, you can run the snippets in your own local environment.

Once your sandbox loads, we will start by loading the *Wine* dataset directly from the repository:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

Next, we will process the *Wine* data into separate training and test sets — using a 70:30 split — and standardize it to unit variance:

After completing the mandatory preprocessing, let's advance to the second step: constructing the covariance matrix. The symmetric *d* x *d*-dimensional covariance matrix, where *d* is the number of dimensions in the dataset, stores the pairwise covariances between the different features. For example, the covariance between two features *x_j* and *x_k* on the population level can be calculated via the following equation:

$$\sigma_{jk} = \frac{1}{n}\sum_{i=1}^{n}(x_j^{(i)} - \mu_j)(x_k^{(i)} - \mu_k)$$

Here, *μ_j* and *μ_k* are the sample means of features *j* and *k*, respectively.

Note that the sample means are zero if we standardized the dataset. A positive covariance between two features indicates that the features increase or decrease together, whereas a negative covariance indicates that the features vary in opposite directions. For example, the covariance matrix of three features can then be written as follows (note that **Σ** stands for the Greek uppercase letter **sigma**, which is not to be confused with the **sum** symbol):

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 \end{bmatrix}$$

The eigenvectors of the covariance matrix represent the principal components (the directions of maximum variance), whereas the corresponding eigenvalues will define their magnitude. In the case of the *Wine* dataset, we would obtain 13 eigenvectors and eigenvalues from the 13 x 13-dimensional covariance matrix.

Now, for our third step, let's obtain the eigenpairs of the covariance matrix. An eigenvector ***v*** satisfies the following condition:

$$\Sigma \boldsymbol{v} = \lambda \boldsymbol{v}$$

Here, **λ** is a scalar: the eigenvalue. Since the manual computation of eigenvectors and eigenvalues is a somewhat tedious and elaborate task, we will use the `linalg.eig` function from `NumPy` to obtain the eigenpairs of the *Wine* covariance matrix:

Using the `numpy.cov` function, we computed the covariance matrix of the standardized training dataset. Using the `linalg.eig` function, we performed the eigendecomposition, which yielded a vector (`eigen_vals`) consisting of 13 eigenvalues and the corresponding eigenvectors stored as columns in a 13 x 13-dimensional matrix (`eigen_vecs`).

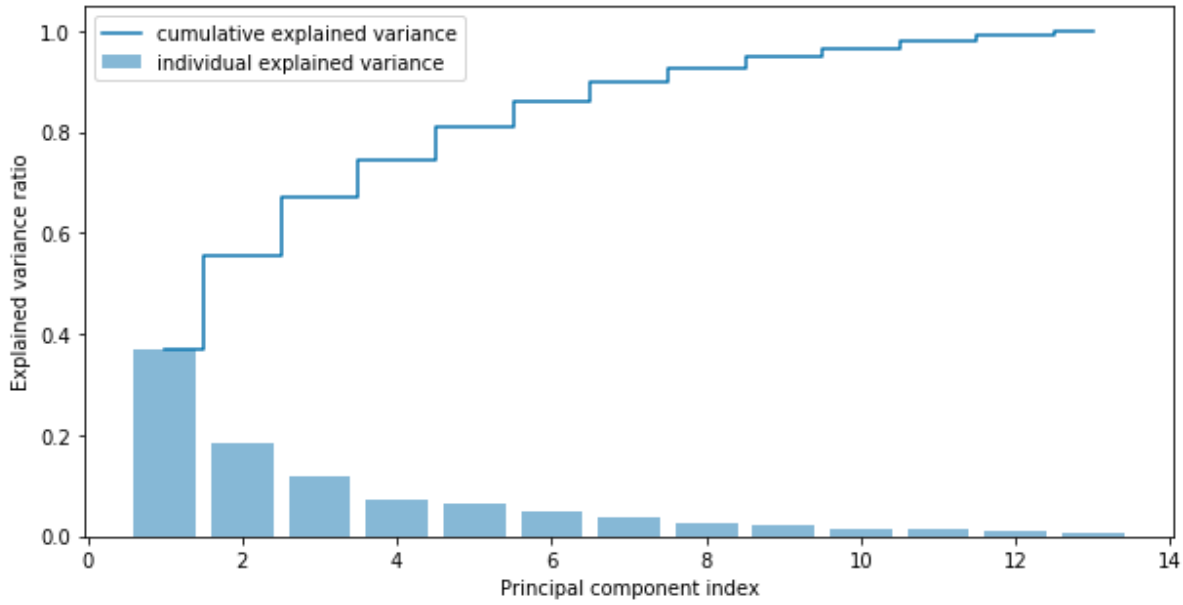
Total and Explained Variance

Since we want to reduce the dimensionality of our dataset by compressing it onto a new feature subspace, we only select the subset of the eigenvectors (principal components) that contains most of the information (variance). The eigenvalues define the magnitude of the eigenvectors, so we have to sort the eigenvalues by decreasing magnitude; we are interested in the top *k* eigenvectors based on the values of their corresponding eigenvalues.

But before we collect those *k* most informative eigenvectors, let's plot the **variance explained ratios** of the eigenvalues. The variance explained ratio of an eigenvalue λ_j is simply the fraction of an eigenvalue λ_j and the total sum of the eigenvalues:

$$\frac{\lambda_j}{\sum_{j=1}^{d} \lambda_j}$$

Using the NumPy `cumsum` function, we can then calculate the cumulative sum of explained variances, which we will then plot via `matplotlib`'s `step` function:



The resulting plot indicates that the first principal component alone accounts for approximately 40% of the variance. Also, we can see that the first two principal components combined explain almost 60% of the variance in the dataset.

Feature Transformation

After we have successfully decomposed the covariance matrix into eigenpairs, let's now proceed with the last three steps of PCA to transform the *Wine* dataset onto the new principal component axes.

We will sort the eigenpairs by descending order of the eigenvalues, construct a projection matrix from the selected eigenvectors, and use the projection matrix to transform the data onto the lower-dimensional subspace.

We start by sorting the eigenpairs by decreasing order of the eigenvalues:

Next, we collect the two eigenvectors that correspond to the two largest eigenvalues, to capture about 60% of the variance in this dataset. Note that we only chose two eigenvectors for the purpose of illustration, since we are going to plot the data via a two-dimensional scatter plot later in this subsection. In practice, the number of principal components has to be determined by a trade-off between computational efficiency and the performance of the classifier:

```
[Out:]
Matrix W:
[[-0.13724218 0.50303478]
[ 0.24724326 0.16487119]
[-0.02545159 0.24456476]
[ 0.20694508 -0.11352904]
[-0.15436582 0.28974518]
[-0.39376952 0.05080104]
[-0.41735106 -0.02287338]
[ 0.30572896 0.09048885]
[-0.30668347 0.00835233]
[ 0.07554066 0.54977581]
[-0.32613263 -0.20716433]
[-0.36861022 -0.24902536]
[-0.29669651  0.38022942]]
```

By executing the preceding code, we have created a 13 x 2-dimensional projection matrix ***W*** from the top two eigenvectors.

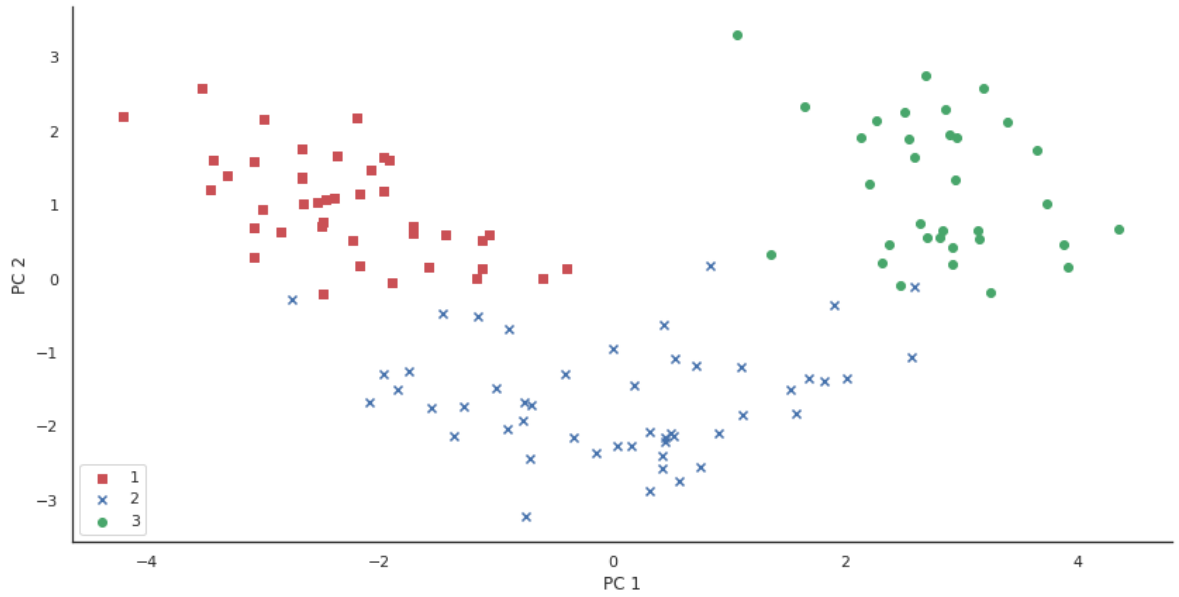
Using the projection matrix, we can now transform a sample ***x*** (represented as a 1 x 13-dimensional row vector) onto the PCA subspace (the principal components one and two) obtaining ***x′***, now a two-dimensional sample vector consisting of two new features:

$$\boldsymbol{x}' = \boldsymbol{x}\boldsymbol{W}$$

Similarly, we can transform the entire 124 x 13-dimensional training dataset onto the two principal components by calculating the matrix dot product:

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W}$$

Lastly, let's visualize the transformed *Wine* training set, now stored as an 124 x 2-dimensional matrix, in a two-dimensional scatterplot:



As we can see in the resulting plot, the data is more spread along the *x*-axis — the first principal component — than the second principal component (*y*-axis), which is consistent with the explained variance ratio plot that we created previously. However, we can intuitively see that a linear classifier will likely be able to separate the classes well.

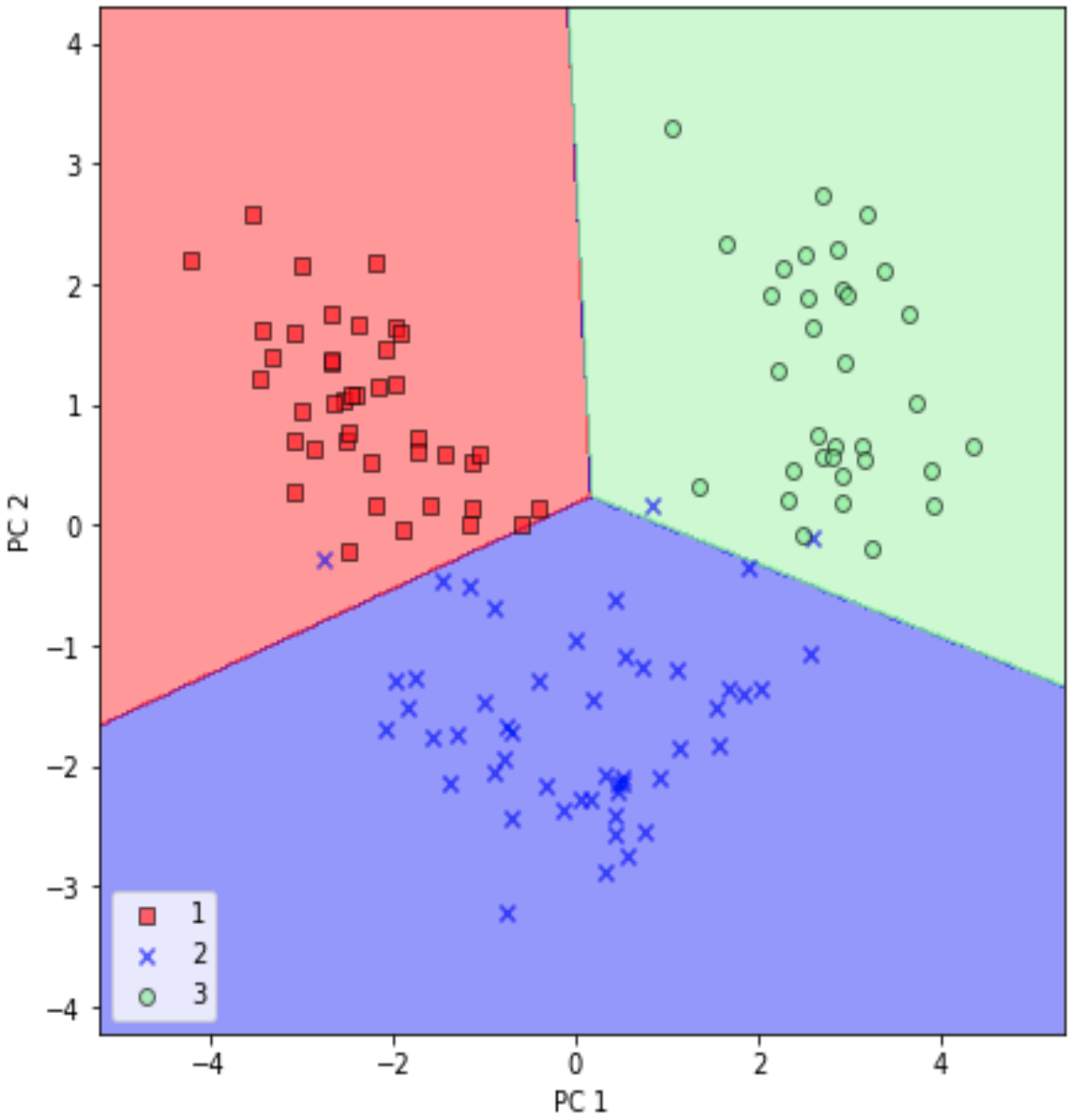
Although we encoded the class label information for the purpose of illustration in the preceding scatter plot, we have to keep in mind that PCA is an unsupervised technique that does not use any class label information.

PCA in scikit-learn

Although the verbose approach in the previous subsection helped us to follow the inner workings of PCA, we will now discuss how to use the `PCA` class implemented in `scikit-learn`. The `PCA` class is another one of `scikit-learn`'s transformer classes, where we first fit the model using the training data before we transform both the training data and the test dataset using the same model parameters.

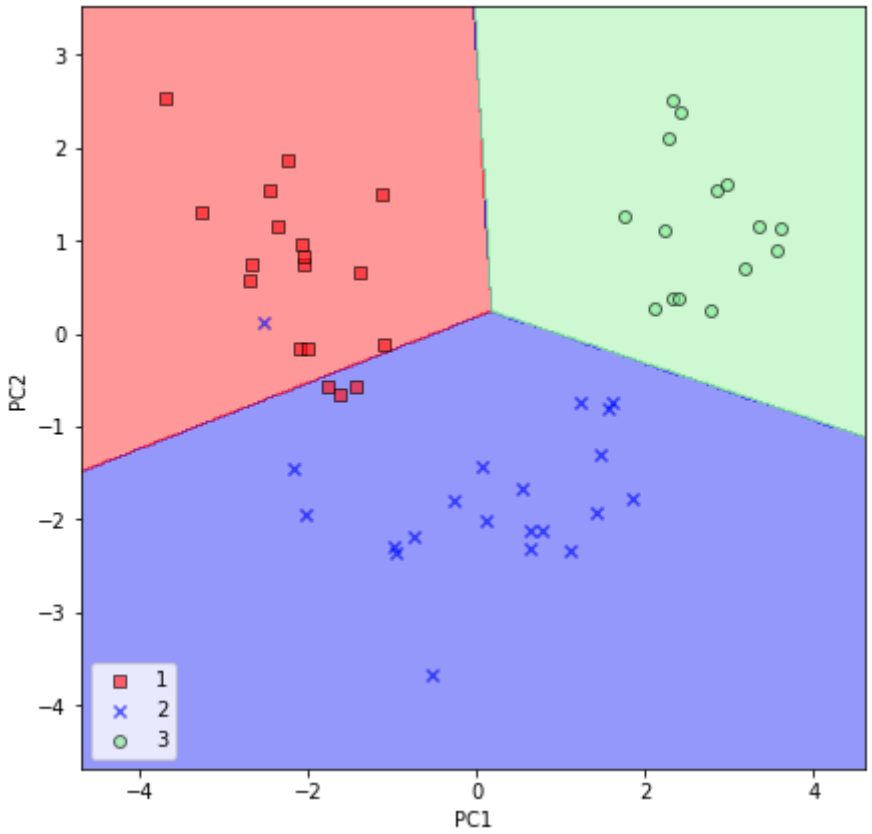
Let's use the `PCA` class on the *Wine* training dataset, classify the transformed samples via logistic regression:

Now, using a custom `plot_decision_regions` function, we will visualize the decision regions:



By executing the preceding code, we should now see the decision regions for the training data reduced to two principal component axes.

For the sake of completeness, let's plot the decision regions of the logistic regression on the transformed test dataset as well to see if it can separate the classes well:



After we plotted the decision regions for the test set by executing the preceding code, we can see that logistic regression performs quite well on this small two-dimensional feature subspace and only misclassifies very few samples in the test dataset.

If we are interested in the explained variance ratios of the different principal components, we can simply initialize the `PCA` class with the `n_components` parameter set to `None`, so all principal components are kept and the explained variance ratio can then be accessed via the `explained_variance_ratio_` attribute:

Note that we set `n_components=None` when we initialized the `PCA` class so that it will return all principal components in a sorted order instead of performing a dimensionality reduction.

Conclusion: Thus, we have studied and implemented PCA algorithm for data transformation and dimension reduction.

ASSIGNMENT NO.B04

Using a suitable tool, perform data visualization, feature extraction analysis

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
20	Explanation of Assignment	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
10	Preparation of Algorithm and logic development for program	Paper work	Introduces, Facilitates, Explains	Discuss	Knowledge, Communication, intrapersonal, Application
60	Coding of program	Keyboard, mouse, computers,Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers, Internet.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Using a suitable tool, perform data visualization, feature extraction analysis**Objectives:** To study and implement data data visualization, feature extraction analysis**.**

Aim/ Problem Statement: Using a suitable tool, perform data visualization, feature extraction analysis.

Apparatus: Computer

Theory:

Titanic DataSet

The data science solutions workflow solves for seven major goals.

Classifying. We may want to classify or categorize our samples. We may also want to understand the implications or correlation of different classes with our solution goal.

Correlating. One can approach the problem based on available features within the training dataset. Which features within the dataset contribute significantly to our solution goal? Statistically speaking is there a correlation among a feature and solution goal? As the feature values change does the solution state change as well, and visa-versa? This can be tested both for numerical and categorical features in the given dataset. We may also want to determine correlation among features other than survival for subsequent goals and workflow stages. Correlating certain features may help in creating, completing, or correcting features.

Converting. For modeling stage, one needs to prepare the data. Depending on the choice of model algorithm one may require all features to be converted to numerical equivalent values. So for instance converting text categorical values to numeric values.

Completing. Data preparation may also require us to estimate any missing values within a feature. Model algorithms may work best when there are no missing values.

Correcting. We may also analyze the given training dataset for errors or possibly innacurate values within features and try to corrent these values or exclude the samples containing the errors. One way to do this is to detect any outliers among our samples or features. We may also completely discard a feature if it is not contribting to the analysis or may significantly skew the results.

Creating. Can we create new features based on an existing feature or a set of features, such that the new feature follows the correlation, conversion, completeness goals.

Charting. How to select the right visualization plots and charts depending on nature of the data and the solution goals

In [1]:

```
# data analysis and wrangling
import pandas as pd
import numpy as np
import random as rnd

# visualization
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

# machine learning
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC, LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import Perceptron
from sklearn.linear_model import SGDClassifier
from sklearn.tree import DecisionTreeClassifier
```

Acquire data

The Python Pandas packages helps us work with our datasets. We start by acquiring the training and testing datasets into Pandas DataFrames. We also combine these datasets to run certain operations on both datasets together.

In [2]:

```
train_df = pd.read_csv('../input/train.csv')
test_df = pd.read_csv('../input/test.csv')
combine = [train_df, test_df]
```

Analyze by describing data

Pandas also helps describe the datasets answering following questions early in our project.

Which features are available in the dataset?

Noting the feature names for directly manipulating or analyzing these.

In [3]:
```
print(train_df.columns.values)
```

```
['PassengerId' 'Survived' 'Pclass' 'Name' 'Sex' 'Age' 'SibSp' 'Parch'
 'Ticket' 'Fare' 'Cabin' 'Embarked']
```

Which features are categorical?

These values classify the samples into sets of similar samples. Within categorical features are the values nominal, ordinal, ratio, or interval based? Among other things this helps us select the appropriate plots for visualization.

- Categorical: Survived, Sex, and Embarked. Ordinal: Pclass.

Which features are numerical?

Which features are numerical? These values change from sample to sample. Within numerical features are the values discrete, continuous, or timeseries based? Among other things this helps us select the appropriate plots for visualization.

- Continous: Age, Fare. Discrete: SibSp, Parch.

In [4]:
```
# preview the data
train_df.head()
```

Out[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Which features are mixed data types?

Numerical, alphanumeric data within same feature. These are candidates for correcting goal.

- Ticket is a mix of numeric and alphanumeric data types. Cabin is alphanumeric.

Which features may contain errors or typos?

This is harder to review for a large dataset, however reviewing a few samples from a smaller dataset may just tell us outright, which features may require correcting.

- Name feature may contain errors or typos as there are several ways used to describe a name including titles, round brackets, and quotes used for alternative or short names.

```
In [5]: train_df.tail()
```

Out[5]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Which features contain blank, null or empty values?

These will require correcting.

- Cabin > Age > Embarked features contain a number of null values in that order for the training dataset.
- Cabin > Age are incomplete in case of test dataset.

What are the data types for various features?

Helping us during converting goal.

- Seven features are integer or floats. Six in case of test dataset.
- Five features are strings (object).

```
In [6]:
train_df.info()
print('_'*40)
test_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
PassengerId    891 non-null int64
Survived       891 non-null int64
Pclass         891 non-null int64
Name           891 non-null object
Sex            891 non-null object
Age            714 non-null float64
SibSp          891 non-null int64
Parch          891 non-null int64
Ticket         891 non-null object
Fare           891 non-null float64
```

What is the distribution of numerical feature values across the samples?

This helps us determine, among other early insights, how representative is the training dataset of the actual problem domain.

- Total samples are 891 or 40% of the actual number of passengers on board the Titanic (2,224).
- Survived is a categorical feature with 0 or 1 values.

- Around 38% samples survived representative of the actual survival rate at 32%.
- Most passengers (> 75%) did not travel with parents or children.
- Nearly 30% of the passengers had siblings and/or spouse aboard.
- Fares varied significantly with few passengers (<1%) paying as high as $512.
- Few elderly passengers (<1%) within age range 65-80.

In [7]:

```
train_df.describe()
# Review survived rate using `percentiles=[.61, .62]` knowing our problem description mentions 38% survival rate.
# Review Parch distribution using `percentiles=[.75, .8]`
# SibSp distribution `[.68, .69]`
# Age and Fare `[.1, .2, .3, .4, .5, .6, .7, .8, .9, .99]`
```

Out[7]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

What is the distribution of categorical features?

- Names are unique across the dataset (count=unique=891)
- Sex variable as two possible values with 65% male (top=male, freq=577/count=891).
- Cabin values have several dupicates across samples. Alternatively several passengers shared a cabin.
- Embarked takes three possible values. S port used by most passengers (top=S)
- Ticket feature has high ratio (22%) of duplicate values (unique=681).

In [8]:
```
train_df.describe(include=['O'])
```

Out[8]:

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Panula, Master. Juha Niilo	male	1601	C23 C25 C27	S
freq	1	577	7	4	644

Assumtions based on data analysis

We arrive at following assumptions based on data analysis done so far. We may validate these assumptions further before taking appropriate actions.

Correlating.

We want to know how well does each feature correlate with Survival. We want to do this early in our project and match these quick correlations with modelled correlations later in the project.

Completing.

1. We may want to complete Age feature as it is definitely correlated to survival.
2. We may want to complete the Embarked feature as it may also correlate with survival or another important feature.

Correcting.

1. Ticket feature may be dropped from our analysis as it contains high ratio of duplicates (22%) and there may not be a correlation between Ticket and survival.
2. Cabin feature may be dropped as it is highly incomplete or contains many null values both in training and test dataset.
3. PassengerId may be dropped from training dataset as it does not contribute to survival.

4. Name feature is relatively non-standard, may not contribute directly to survival, so maybe dropped.

Creating.

1. We may want to create a new feature called Family based on Parch and SibSp to get total count of family members on board.
2. We may want to engineer the Name feature to extract Title as a new feature.
3. We may want to create new feature for Age bands. This turns a continous numerical feature into an ordinal categorical feature.
4. We may also want to create a Fare range feature if it helps our analysis.

Classifying.

We may also add to our assumptions based on the problem description noted earlier.

1. Women (Sex=female) were more likely to have survived.
2. Children (Age<?) were more likely to have survived.
3. The upper-class passengers (Pclass=1) were more likely to have survived.

Analyze by pivoting features

To confirm some of our observations and assumptions, we can quickly analyze our feature correlations by pivoting features against each other. We can only do so at this stage for features which do not have any empty values. It also makes sense doing so only for features which are categorical (Sex), ordinal (Pclass) or discrete (SibSp, Parch) type.

- **Pclass** We observe significant correlation (>0.5) among Pclass=1 and Survived (classifying #3). We decide to include this feature in our model.
- **Sex** We confirm the observation during problem definition that Sex=female had very high survival rate at 74% (classifying #1).
- **SibSp and Parch** These features have zero correlation for certain values. It may be best to derive a feature or a set of features from these individual features (creating #1).

In [9]:
```
train_df[['Pclass', 'Survived']].groupby(['Pclass'], as_index=False).mean().sort_values(by='Survived', ascending=False)
```

Out[9]:

	Pclass	Survived
0	1	0.629630
1	2	0.472826
2	3	0.242363

In [10]:
```
train_df[["Sex", "Survived"]].groupby(['Sex'], as_index=False).mean().sort_values(by='Survived', ascending=False)
```

Out[10]:

	Sex	Survived
0	female	0.742038
1	male	0.188908

In [11]:
```
train_df[["SibSp", "Survived"]].groupby(['SibSp'], as_index=False).mean().sort_values(by='Survived', ascending=False)
```

Out[11]:

	SibSp	Survived
1	1	0.535885
2	2	0.464286
0	0	0.345395
3	3	0.250000
4	4	0.166667
5	5	0.000000
6	8	0.000000

In [12]:
```
train_df[["Parch", "Survived"]].groupby(['Parch'], as_index=False).mean().sort_values(by='Survived', ascending=False)
```

Out[12]:

	Parch	Survived
3	3	0.600000
1	1	0.550847
2	2	0.500000
0	0	0.343658
5	5	0.200000
4	4	0.000000
6	6	0.000000

Analyze by visualizing data

Now we can continue confirming some of our assumptions using visualizations for analyzing the data.

Correlating numerical features

Let us start by understanding correlations between numerical features and our solution goal (Survived).

A histogram chart is useful for analyzingcontinous numerical variables like Age where banding or ranges will help identify useful patterns. The histogram can indicate distribution of samples using automatically defined bins or equally ranged bands. This helps us answer questions relating to specific bands (Did infants have better survival rate?)

Note that x-axis in historgram visualizations represents the count of samples or passengers.

Observations.

- Infants (Age <=4) had high survival rate.
- Oldest passengers (Age = 80) survived.
- Large number of 15-25 year olds did not survive.
- Most passengers are in 15-35 age range.

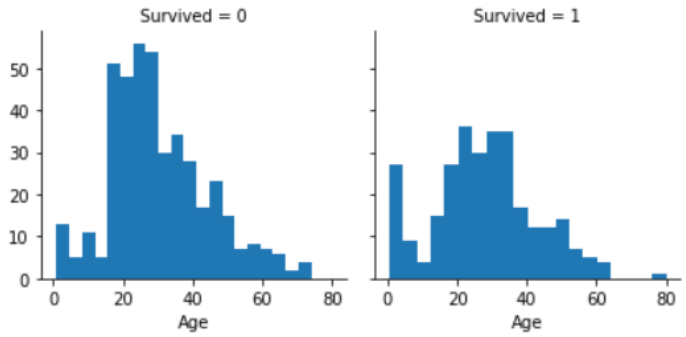
Decisions.

This simple analysis confirms our assumptions as decisions for subsequent workflow stages.

- We should consider Age (our assumption classifying #2) in our model training.
- Complete the Age feature for null values (completing #1).
- We should band age groups (creating #3).

In [13]:
```
g = sns.FacetGrid(train_df, col='Survived')
g.map(plt.hist, 'Age', bins=20)
```

Out[13]:
```
<seaborn.axisgrid.FacetGrid at 0x7fa19ed21390>
```



Correlating numerical and ordinal features

We can combine multiple features for identifying correlations using a single plot. This can be done with numerical and categorical features which have numeric values.

Observations

- Pclass=3 had most passengers, however most did not survive. Confirms our classifying assumption #2.
- Infant passengers in Pclass=2 and Pclass=3 mostly survived. Further qualifies our classifying assumption #2.
- Most passengers in Pclass=1 survived. Confirms our classifying assumption #3.
- Pclass varies in terms of Age distribution of passengers.

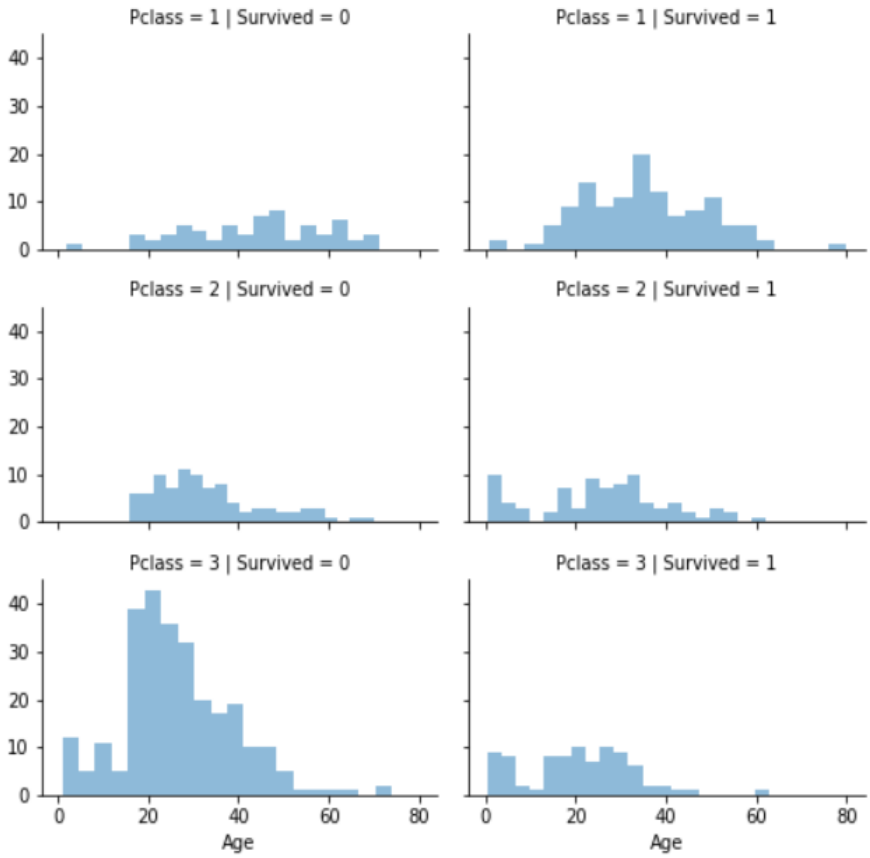
Decisions

- Consider Pclass for model training.

In [14]:

```
# grid = sns.FacetGrid(train_df, col='Pclass', hue='Survived')
grid = sns.FacetGrid(train_df, col='Survived', row='Pclass', size=2.2, aspect=1.6)
grid.map(plt.hist, 'Age', alpha=.5, bins=20)
grid.add_legend();
```

```
/opt/conda/lib/python3.6/site-packages/seaborn/axisgrid.py:230: UserWarning: The `size` paramter has been renamed to `
height`; please update your code.
  warnings.warn(msg, UserWarning)
```



Correlating categorical features

Now we can correlate categorical features with our solution goal.

Observations

- Female passengers had much better survival rate than males. Confirms classifying (#1).
- Exception in Embarked=C where males had higher survival rate. This could be a correlation between Pclass and Embarked and in turn Pclass and Survived, not necessarily direct correlation between Embarked and Survived.
- Males had better survival rate in Pclass=3 when compared with Pclass=2 for C and Q ports. Completing (#2).

- Ports of embarkation have varying survival rates for Pclass=3 and among male passengers. Correlating (#1).

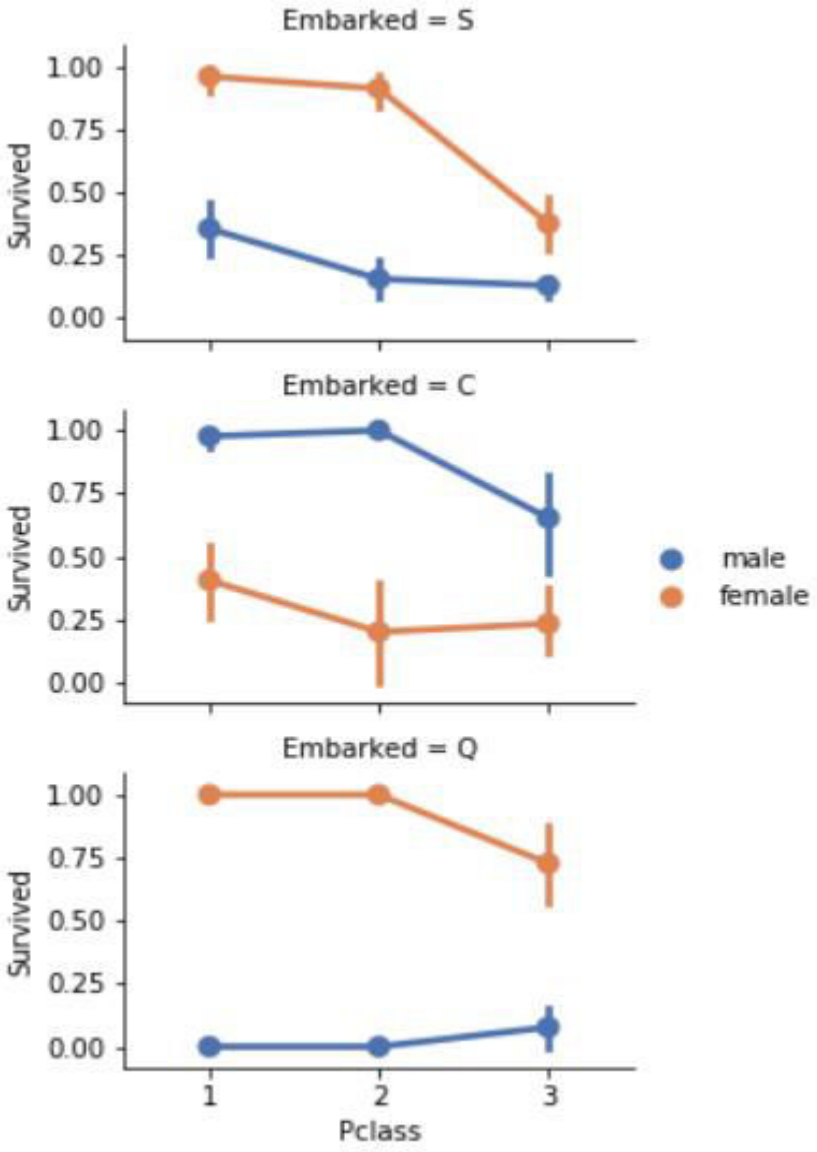
Decisions

- Add Sex feature to model training.
- Complete and add Embarked feature to model training.

In [15]:

```
# grid = sns.FacetGrid(train_df, col='Embarked')
grid = sns.FacetGrid(train_df, row='Embarked', size=2.2, aspect=1.6)
grid.map(sns.pointplot, 'Pclass', 'Survived', 'Sex', palette='deep')
grid.add_legend()
```

```
/opt/conda/lib/python3.6/site-packages/seaborn/axisgrid.py:230: UserWarning: The `size` paramter has been renamed to `height`; please update your code.
  warnings.warn(msg, UserWarning)
/opt/conda/lib/python3.6/site-packages/seaborn/axisgrid.py:715: UserWarning: Using the pointplot function without specifying `order` is likely to produce an incorrect plot.
  warnings.warn(warning)
/opt/conda/lib/python3.6/site-packages/seaborn/axisgrid.py:720: UserWarning: Using the pointplot function without specifying `hue_order` is likely to produce an incorrect plot.
  warnings.warn(warning)
/opt/conda/lib/python3.6/site-packages/scipy/stats/stats.py:1713: FutureWarning: Using a non-tuple sequence for multidimensional indexing is deprecated; use `arr[tuple(seq)]` instead of `arr[seq]`. In the future this will be interpreted as an array index, `arr[np.array(seq)]`, which will result either in an error or a different result.
```



Correlating categorical and numerical features

We may also want to correlate categorical features (with non-numeric values) and numeric features. We can consider correlating Embarked (Categorical non-numeric), Sex (Categorical non-numeric), Fare (Numeric continuous), with Survived (Categorical numeric).

Observations.

- Higher fare paying passengers had better survival. Confirms our assumption for creating (#4) fare ranges.

- Port of embarkation correlates with survival rates. Confirms correlating (#1) and completing (#2).

Decisions.

- Consider banding Fare feature.

In [16]:
```
# grid = sns.FacetGrid(train_df, col='Embarked', hue='Survived', palette={0: 'k', 1: 'w'})
grid = sns.FacetGrid(train_df, row='Embarked', col='Survived', size=2.2, aspect=1.6)
grid.map(sns.barplot, 'Sex', 'Fare', alpha=.5, ci=None)
grid.add_legend()
```

```
/opt/conda/lib/python3.6/site-packages/seaborn/axisgrid.py:230: UserWarning: The `size` paramter has been renamed to `
height`; please update your code.
  warnings.warn(msg, UserWarning)
/opt/conda/lib/python3.6/site-packages/seaborn/axisgrid.py:715: UserWarning: Using the barplot function without specif
ying `order` is likely to produce an incorrect plot.
  warnings.warn(warning)
```

Wrangle data

We have collected several assumptions and decisions regarding our datasets and solution requirements. So far we did not have to change a single feature or value to arrive at these. Let us now execute our decisions and assumptions for correcting, creating, and completing goals.

Correcting by dropping features

This is a good starting goal to execute. By dropping features we are dealing with fewer data points. Speeds up our notebook and eases the analysis.

Based on our assumptions and decisions we want to drop the Cabin (correcting #2) and Ticket (correcting #1) features.

Note that where applicable we perform operations on both training and testing datasets together to stay consistent.

In [17]:
```
print("Before", train_df.shape, test_df.shape, combine[0].shape, combine[1].shape)

train_df = train_df.drop(['Ticket', 'Cabin'], axis=1)
test_df = test_df.drop(['Ticket', 'Cabin'], axis=1)
combine = [train_df, test_df]

"After", train_df.shape, test_df.shape, combine[0].shape, combine[1].shape
```

```
Before (891, 12) (418, 11) (891, 12) (418, 11)
```

Out[17]:
```
('After', (891, 10), (418, 9), (891, 10), (418, 9))
```

Creating new feature extracting from existing

We want to analyze if Name feature can be engineered to extract titles and test correlation between titles and survival, before dropping Name and PassengerId features.

In the following code we extract Title feature using regular expressions. The RegEx pattern (\w+\.) matches the first word which ends with a dot character within Name feature. The expand=False flag returns a DataFrame.

Observations

When we plot Title, Age, and Survived, we note the following observations.

- Most titles band Age groups accurately. For example: Master title has Age mean of 5 years.
- Survival among Title Age bands varies slightly.
- Certain titles mostly survived (Mme, Lady, Sir) or did not (Don, Rev, Jonkheer).

Decision

- We decide to retain the new Title feature for model training.

Conclusion: Thus we have studied

In [18]:

```
for dataset in combine:
    dataset['Title'] = dataset.Name.str.extract(' ([A-Za-z]+)\.', expand=False)

pd.crosstab(train_df['Title'], train_df['Sex'])
```

Out[18]:

Sex	female	male
Title		
Capt	0	1
Col	0	2
Countess	1	0
Don	0	1
Dr	1	6
Jonkheer	0	1
Lady	1	0
Major	0	2

We can replace many titles with a more common name or classify them as Rare.

In [19]:

In [19]:

```
for dataset in combine:
    dataset['Title'] = dataset['Title'].replace(['Lady', 'Countess','Capt', 'Col',\
        'Don', 'Dr', 'Major', 'Rev', 'Sir', 'Jonkheer', 'Dona'], 'Rare')

    dataset['Title'] = dataset['Title'].replace('Mlle', 'Miss')
    dataset['Title'] = dataset['Title'].replace('Ms', 'Miss')
    dataset['Title'] = dataset['Title'].replace('Mme', 'Mrs')

train_df[['Title', 'Survived']].groupby(['Title'], as_index=False).mean()
```

Out[19]:

	Title	Survived
0	Master	0.575000
1	Miss	0.702703
2	Mr	0.156673
3	Mrs	0.793651
4	Rare	0.347826

We can convert the categorical titles to ordinal.

In [20]:

In [20]:

```
title_mapping = {"Mr": 1, "Miss": 2, "Mrs": 3, "Master": 4, "Rare": 5}
for dataset in combine:
    dataset['Title'] = dataset['Title'].map(title_mapping)
    dataset['Title'] = dataset['Title'].fillna(0)

train_df.head()
```

Out[20]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Fare	Embarked	Title
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	7.2500	S	1
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	71.2833	C	3
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	7.9250	S	2
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	53.1000	S	3
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	8.0500	S	1

Now we can safely drop the Name feature from training and testing datasets. We also do not need the PassengerId feature in the training dataset.

In [21]:

In [21]:

```
train_df = train_df.drop(['Name', 'PassengerId'], axis=1)
test_df = test_df.drop(['Name'], axis=1)
combine = [train_df, test_df]
train_df.shape, test_df.shape
```

Out[21]:

```
((891, 9), (418, 9))
```

Converting a categorical feature

Now we can convert features which contain strings to numerical values. This is required by most model algorithms. Doing so will also help us in achieving the feature completing goal.

Let us start by converting Sex feature to a new feature called Gender where female=1 and male=0.

In [22]:
```
for dataset in combine:
    dataset['Sex'] = dataset['Sex'].map( {'female': 1, 'male': 0} ).astype(int)

train_df.head()
```

Out[22]:

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Title
0	0	3	0	22.0	1	0	7.2500	S	1
1	1	1	1	38.0	1	0	71.2833	C	3
2	1	3	1	26.0	0	0	7.9250	S	2
3	1	1	1	35.0	1	0	53.1000	S	3
4	0	3	0	35.0	0	0	8.0500	S	1

Completing a numerical continuous feature

Now we should start estimating and completing features with missing or null values. We will first do this for the Age feature.

We can consider three methods to complete a numerical continuous feature.

1. A simple way is to generate random numbers between mean and standard deviation.
2. More accurate way of guessing missing values is to use other correlated features. In our case we note correlation among Age, Gender, and Pclass. Guess Age values using median values for Age across sets of Pclass and Gender feature combinations. So, median Age for Pclass=1 and Gender=0, Pclass=1 and Gender=1, and so on...
3. Combine methods 1 and 2. So instead of guessing age values based on median, use random numbers between mean and standard deviation, based on sets of Pclass and Gender combinations.

Conclusion:- Thus we have studied and implemented data visualization and feature extraction on Titanic dataset.

ASSIGNMENT NO.B05 (Content Beyond Syllabus)

Study and implement Convolutional Neural Networks and Recurrent Neural Networks

Practical Session Plan

Time (min)	**Content**	**Learning Aid / Methodology**	**Faculty Approach**	**Typical Student Activity**	**Skill / Competency Developed**
40	Explanation of Assignment and concepts	Chalk & Talk , Presentation	Introduces, Facilitates, Explains	Listens	Knowledge, Communication, intrapersonal, Application
50	Coding of CNN,RNN	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
20	Testing of program	Keyboard, mouse, computers.	Introduces, Facilitates, Explains	Discuss, participates	Knowledge, Communication, intrapersonal, Application
10	Results and conclusions	Keywords	Lists, Facilitates	Listens, Participates, Discusses	Knowledge, Communication, intrapersonal, Comprehension

Title: Study and implement Convolutional Neural Networks and Recurrent Neural Networks

Objectives: To demonstrate Convolutional Neural Networks and Recurrent Neural Networks

Aim/ Problem Statement: Write program to demonstrate Convolutional Neural Networks and Recurrent Neural Networks

Apparatus: Computer

Theory:

- TensorFlow - Convolutional Neural Networks
- TensorFlow - Recurrent Neural Networks

TensorFlow - Convolutional Neural Networks

Convolutional Neural networks are designed to process data through multiple layers of arrays. This type of neural networks is used in applications like image recognition or face recognition. The primary difference between CNN and any other ordinary neural network is that CNN takes input as a two-dimensional array and operates directly on the images rather than focusing on feature extraction which other neural networks focus on.

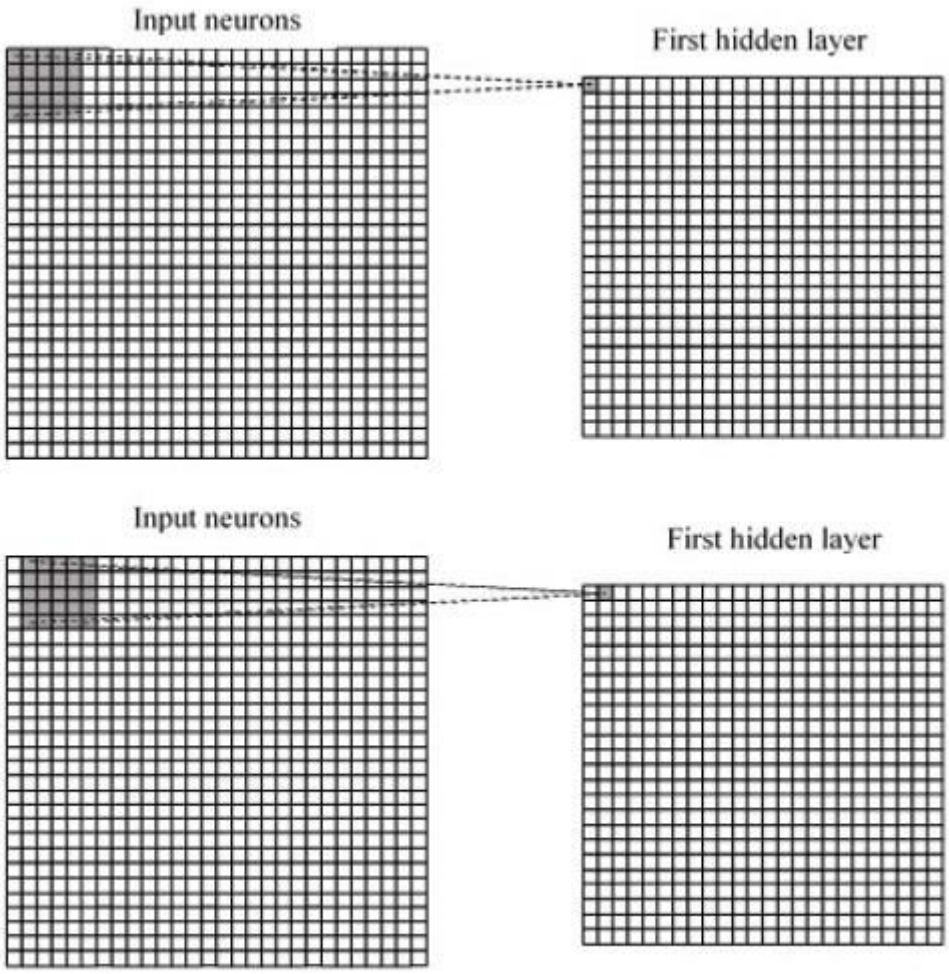
The dominant approach of CNN includes solutions for problems of recognition. Top companies like Google and Facebook have invested in research and development towards recognition projects to get activities done with greater speed.

A convolutional neural network uses three basic ideas −

- Local respective fields
- Convolution
- Pooling

CNN utilizes spatial correlations that exist within the input data. Each concurrent layer of a neural network connects some input neurons. This specific region is called local receptive field. Local receptive field focusses on the hidden neurons. The hidden neurons process the input data inside the mentioned field not realizing the changes outside the specific boundary.

Diagram representation of generating local respective fields is as follows



Local Respective Fields with input neurons

If we observe the above representation, each connection learns a weight of the hidden neuron with an associated connection with movement from one layer to another. Here, individual neurons perform a shift from time to time. This process is called “convolution”.

The mapping of connections from the input layer to the hidden feature map is defined as “shared weights” and bias included is called “shared bias”.

CNN or convolutional neural networks use pooling layers, which are the layers, positioned immediately after CNN declaration. It takes the input from the user as a feature map that comes out of convolutional networks and prepares a condensed feature map. Pooling layers helps in creating layers with neurons of previous layers.

TensorFlow Implementation of CNN

Step 1 − Include the necessary modules for TensorFlow and the data set modules, which are needed to compute the CNN model.

```
import tensorflow as tf
import numpy as np
from tensorflow.examples.tutorials.mnist import input_data
```

Step 2 − Declare a function called **run_cnn()**, which includes various parameters and optimization variables with declaration of data placeholders. These optimization variables will declare the training pattern.

```
def run_cnn():
mnist = input_data.read_data_sets("MNIST_data/", one_hot = True)
learning_rate = 0.0001
epochs = 10
batch_size = 50
```

Step 3 − In this step, we will declare the training data placeholders with input parameters - for 28 x 28 pixels = 784. This is the flattened image data that is drawn from **mnist.train.nextbatch()**.

We can reshape the tensor according to our requirements. The first value (-1) tells function to dynamically shape that dimension based on the amount of data passed to it. The two middle dimensions are set to the image size (i.e. 28 x 28).

```
x = tf.placeholder(tf.float32, [None, 784])
x_shaped = tf.reshape(x, [-1, 28, 28, 1])
y = tf.placeholder(tf.float32, [None, 10])
```

Step 4 − Now it is important to create some convolutional layers −

```
layer1 = create_new_conv_layer(x_shaped, 1, 32, [5, 5], [2, 2], name = 'layer1')
layer2 = create_new_conv_layer(layer1, 32, 64, [5, 5], [2, 2], name = 'layer2')
```

Step 5 − Let us flatten the output ready for the fully connected output stage - after two layers of stride 2 pooling with the dimensions of 28 x 28, to dimension of 14 x 14 or minimum 7 x 7 x,y co-ordinates, but with 64 output channels. To create the fully connected with "dense" layer, the new shape needs to be [-1, 7 x 7 x 64]. We can set up some weights and bias values for this layer, then activate with ReLU.

```
flattened = tf.reshape(layer2, [-1, 7 * 7 * 64])

wd1 = tf.Variable(tf.truncated_normal([7 * 7 * 64, 1000], stddev = 0.03), name = 'wd1')

bd1 = tf.Variable(tf.truncated_normal([1000], stddev = 0.01), name = 'bd1')

dense_layer1 = tf.matmul(flattened, wd1) + bd1

dense_layer1 = tf.nn.relu(dense_layer1)
```

Step 6 − Another layer with specific softmax activations with the required optimizer defines the accuracy assessment, which makes the setup of initialization operator.

```
wd2 = tf.Variable(tf.truncated_normal([1000, 10], stddev = 0.03), name = 'wd2')

bd2 = tf.Variable(tf.truncated_normal([10], stddev = 0.01), name = 'bd2')

dense_layer2 = tf.matmul(dense_layer1, wd2) + bd2

y_ = tf.nn.softmax(dense_layer2)

cross_entropy = tf.reduce_mean(

   tf.nn.softmax_cross_entropy_with_logits(logits = dense_layer2, labels = y))

optimiser = tf.train.AdamOptimizer(learning_rate = learning_rate).minimize(cross_entropy)

correct_prediction = tf.equal(tf.argmax(y, 1), tf.argmax(y_, 1))

accuracy = tf.reduce_mean(tf.cast(correct_prediction, tf.float32))

init_op = tf.global_variables_initializer()
```

Step 7 − We should set up recording variables. This adds up a summary to store the accuracy of data.

```
tf.summary.scalar('accuracy', accuracy)
```

```
merged = tf.summary.merge_all()
writer = tf.summary.FileWriter('E:\TensorFlowProject')
  with tf.Session() as sess:
  sess.run(init_op)
  total_batch = int(len(mnist.train.labels) / batch_size)
      for epoch in range(epochs):
    avg_cost = 0
  for i in range(total_batch):
    batch_x, batch_y = mnist.train.next_batch(batch_size = batch_size)
      _, c = sess.run([optimiser, cross_entropy], feed_dict = {
      x:batch_x, y: batch_y})
      avg_cost += c / total_batch
    test_acc = sess.run(accuracy, feed_dict = {x: mnist.test.images, y:
      mnist.test.labels})
      summary = sess.run(merged, feed_dict = {x: mnist.test.images, y:
      mnist.test.labels})
    writer.add_summary(summary, epoch)
print("\nTraining complete!")
writer.add_graph(sess.graph)
print(sess.run(accuracy, feed_dict = {x: mnist.test.images, y:
```

```
      mnist.test.labels}))

def create_new_conv_layer(

   input_data, num_input_channels, num_filters,filter_shape, pool_shape, name):

   conv_filt_shape = [

      filter_shape[0], filter_shape[1], num_input_channels, num_filters]

   weights = tf.Variable(

      tf.truncated_normal(conv_filt_shape, stddev = 0.03), name = name+'_W')

   bias = tf.Variable(tf.truncated_normal([num_filters]), name = name+'_b')

#Out layer defines the output

   out_layer =

      tf.nn.conv2d(input_data, weights, [1, 1, 1, 1], padding = 'SAME')

   out_layer += bias

   out_layer = tf.nn.relu(out_layer)

   ksize = [1, pool_shape[0], pool_shape[1], 1]

   strides = [1, 2, 2, 1]

   out_layer = tf.nn.max_pool(

      out_layer, ksize = ksize, strides = strides, padding = 'SAME')

   return out_layer

if __name__ == "__main__":

run_cnn()
```

Output/Result :

See @{tf.nn.softmax_cross_entropy_with_logits_v2}.

2018-09-19 17:22:58.802268: I

T:\src\github\tensorflow\tensorflow\core\platform\cpu_feature_guard.cc:140]

Your CPU supports instructions that this TensorFlow binary was not compiled to

use: AVX2

2018-09-19 17:25:41.522845: W

T:\src\github\tensorflow\tensorflow\core\framework\allocator.cc:101] Allocation

of 1003520000 exceeds 10% of system memory.

2018-09-19 17:25:44.630941: W

T:\src\github\tensorflow\tensorflow\core\framework\allocator.cc:101] Allocation

of 501760000 exceeds 10% of system memory.

Epoch: 1 cost = 0.676 test accuracy: 0.940

2018-09-19 17:26:51.987554: W

T:\src\github\tensorflow\tensorflow\core\framework\allocator.cc:101] Allocation

of 1003520000 exceeds 10% of system memory.

TensorFlow - Recurrent Neural Networks

Recurrent neural networks is a type of deep learning-oriented algorithm, which follows a sequential approach. In neural networks, we always assume that each input and output is independent of all other layers. These type of neural networks are called recurrent because they perform mathematical computations in sequential manner.

Consider the following steps to train a recurrent neural network −

Step 1 − Input a specific example from dataset.

Step 2 − Network will take an example and compute some calculations using randomly initialized variables.

Step 3 − A predicted result is then computed.

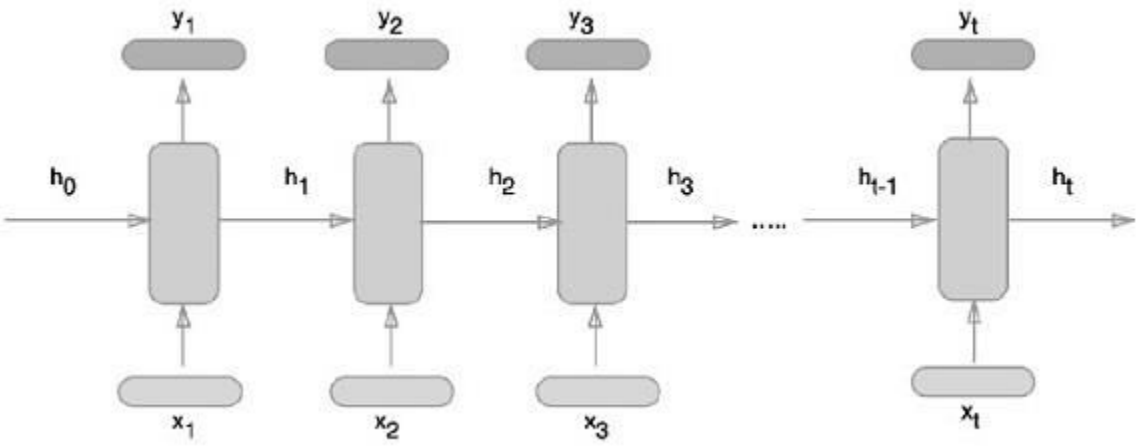
Step 4 − The comparison of actual result generated with the expected value will produce an error.

Step 5 − To trace the error, it is propagated through same path where the variables are also adjusted.

Step 6 − The steps from 1 to 5 are repeated until we are confident that the variables declared to get the output are defined properly.

Step 7 − A systematic prediction is made by applying these variables to get new unseen input.

The schematic approach of representing recurrent neural networks is described below −



Recurrent Neural Network Implementation with TensorFlow

Step 1 − TensorFlow includes various libraries for specific implementation of the recurrent neural network module.

```
#Import necessary modules

from __future__ import print_function

import tensorflow as tf

from tensorflow.contrib import rnn

from tensorflow.examples.tutorials.mnist import input_data

mnist = input_data.read_data_sets("/tmp/data/", one_hot = True)
```

As mentioned above, the libraries help in defining the input data, which forms the primary part of recurrent neural network implementation.

Step 2 − Our primary motive is to classify the images using a recurrent neural network, where we consider every image row as a sequence of pixels. MNIST image shape is specifically defined as 28*28 px. Now we will handle 28 sequences of 28 steps for each sample that is mentioned. We will define the input parameters to get the sequential pattern done.

```
n_input = 28 # MNIST data input with img shape 28*28

n_steps = 28

n_hidden = 128

n_classes = 10

# tf Graph input

x = tf.placeholder("float", [None, n_steps, n_input])

y = tf.placeholder("float", [None, n_classes]

weights = {
```

```
  'out': tf.Variable(tf.random_normal([n_hidden, n_classes]))
}
biases = {
  'out': tf.Variable(tf.random_normal([n_classes]))
}
```

Step 3 − Compute the results using a defined function in RNN to get the best results. Here, each data shape is compared with current input shape and the results are computed to maintain the accuracy rate.

```
def RNN(x, weights, biases):
  x = tf.unstack(x, n_steps, 1)
  # Define a lstm cell with tensorflow
  lstm_cell = rnn.BasicLSTMCell(n_hidden, forget_bias=1.0)

  # Get lstm cell output
  outputs, states = rnn.static_rnn(lstm_cell, x, dtype = tf.float32)
  # Linear activation, using rnn inner loop last output
  return tf.matmul(outputs[-1], weights['out']) + biases['out']
pred = RNN(x, weights, biases)
# Define loss and optimizer
cost = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits = pred, labels = y))
optimizer = tf.train.AdamOptimizer(learning_rate = learning_rate).minimize(cost)
# Evaluate model
correct_pred = tf.equal(tf.argmax(pred,1), tf.argmax(y,1))
```

```
accuracy = tf.reduce_mean(tf.cast(correct_pred, tf.float32))

# Initializing the variables

init = tf.global_variables_initializer()
```

Step 4 − In this step, we will launch the graph to get the computational results. This also helps in calculating the accuracy for test results.

```
with tf.Session() as sess:
  sess.run(init)
  step = 1
  # Keep training until reach max iterations
    while step * batch_size < training_iters:
    batch_x, batch_y = mnist.train.next_batch(batch_size)
    batch_x = batch_x.reshape((batch_size, n_steps, n_input))
    sess.run(optimizer, feed_dict={x: batch_x, y: batch_y})
        if step % display_step == 0:
      # Calculate batch accuracy
      acc = sess.run(accuracy, feed_dict={x: batch_x, y: batch_y})
          # Calculate batch loss
      loss = sess.run(cost, feed_dict={x: batch_x, y: batch_y})


      print("Iter " + str(step*batch_size) + ", Minibatch Loss= " + \
        "{:.6f}".format(loss) + ", Training Accuracy= " + \
        "{:.5f}".format(acc))
    step += 1
```

```
print("Optimization Finished!")

    test_len = 128

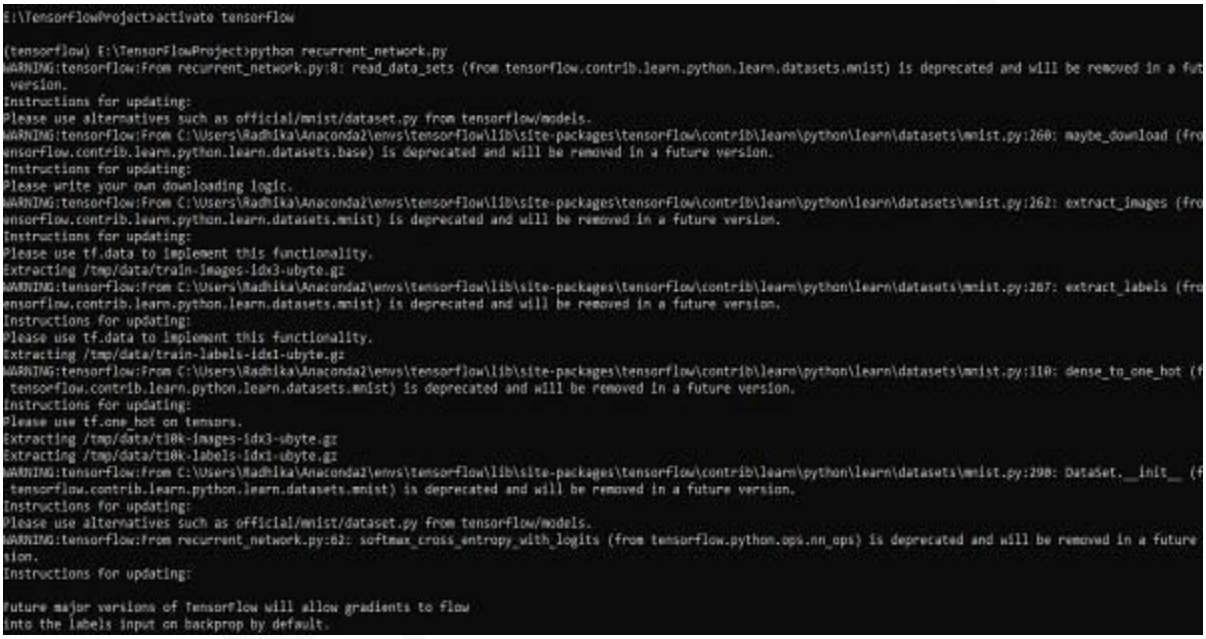
test_data = mnist.test.images[:test_len].reshape((-1, n_steps, n_input))

    test_label = mnist.test.labels[:test_len]

print("Testing Accuracy:", \

    sess.run(accuracy, feed_dict={x: test_data, y: test_label}))
```

Output/Result :



```
Iter 47360, Minibatch Loss= 0.141656, Training Accuracy= 0.96094
Iter 48640, Minibatch Loss= 0.201050, Training Accuracy= 0.92969
Iter 49920, Minibatch Loss= 0.297158, Training Accuracy= 0.93750
Iter 51200, Minibatch Loss= 0.182435, Training Accuracy= 0.95312
Iter 52480, Minibatch Loss= 0.104083, Training Accuracy= 0.96875
Iter 53760, Minibatch Loss= 0.363708, Training Accuracy= 0.90625
Iter 55040, Minibatch Loss= 0.065427, Training Accuracy= 0.98438
Iter 56320, Minibatch Loss= 0.129080, Training Accuracy= 0.96094
Iter 57600, Minibatch Loss= 0.174001, Training Accuracy= 0.96875
Iter 58880, Minibatch Loss= 0.101891, Training Accuracy= 0.97656
Iter 60160, Minibatch Loss= 0.100010, Training Accuracy= 0.96875
Iter 61440, Minibatch Loss= 0.199910, Training Accuracy= 0.93750
Iter 62720, Minibatch Loss= 0.250703, Training Accuracy= 0.92188
Iter 64000, Minibatch Loss= 0.252617, Training Accuracy= 0.93750
Iter 65280, Minibatch Loss= 0.165155, Training Accuracy= 0.94531
Iter 66560, Minibatch Loss= 0.127779, Training Accuracy= 0.95312
Iter 67840, Minibatch Loss= 0.174810, Training Accuracy= 0.92969
Iter 69120, Minibatch Loss= 0.146723, Training Accuracy= 0.96094
Iter 70400, Minibatch Loss= 0.125361, Training Accuracy= 0.95312
Iter 71680, Minibatch Loss= 0.147484, Training Accuracy= 0.96094
Iter 72960, Minibatch Loss= 0.063424, Training Accuracy= 0.98438
Iter 74240, Minibatch Loss= 0.047606, Training Accuracy= 0.98438
Iter 75520, Minibatch Loss= 0.135891, Training Accuracy= 0.95312
Iter 76800, Minibatch Loss= 0.122548, Training Accuracy= 0.97656
Iter 78080, Minibatch Loss= 0.110101, Training Accuracy= 0.93750
Iter 79360, Minibatch Loss= 0.133690, Training Accuracy= 0.95312
Iter 80640, Minibatch Loss= 0.074933, Training Accuracy= 0.96875
Iter 81920, Minibatch Loss= 0.102488, Training Accuracy= 0.96875
Iter 83200, Minibatch Loss= 0.138131, Training Accuracy= 0.96094
Iter 84480, Minibatch Loss= 0.088344, Training Accuracy= 0.96875
Iter 85760, Minibatch Loss= 0.155597, Training Accuracy= 0.96094
Iter 87040, Minibatch Loss= 0.102829, Training Accuracy= 0.94531
Iter 88320, Minibatch Loss= 0.134922, Training Accuracy= 0.95312
Iter 89600, Minibatch Loss= 0.190034, Training Accuracy= 0.96094
Iter 90880, Minibatch Loss= 0.109202, Training Accuracy= 0.96875
Iter 92160, Minibatch Loss= 0.155241, Training Accuracy= 0.97656
Iter 93440, Minibatch Loss= 0.092797, Training Accuracy= 0.97656
Iter 94720, Minibatch Loss= 0.095227, Training Accuracy= 0.96094
Iter 96000, Minibatch Loss= 0.143840, Training Accuracy= 0.97656
Iter 97280, Minibatch Loss= 0.120336, Training Accuracy= 0.96094
Iter 98560, Minibatch Loss= 0.179503, Training Accuracy= 0.96094
Iter 99840, Minibatch Loss= 0.075086, Training Accuracy= 0.97656
Optimization Finished!
```

Observations: TensorFlow is used for implementing CNN, RNN

Conclusion: Thus we have studied and implemented CNN and RNN.